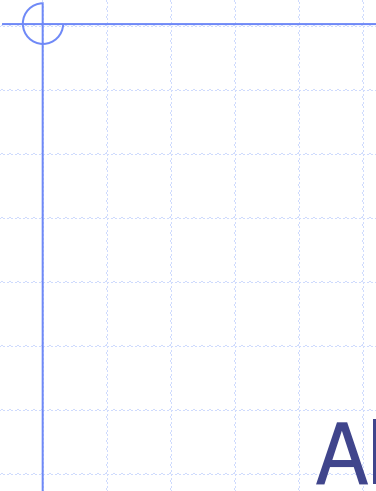




Week 7

Search Trees

Algorithms and Data Structures

COMP3506/7505

NOTE: READ CAREFULLY

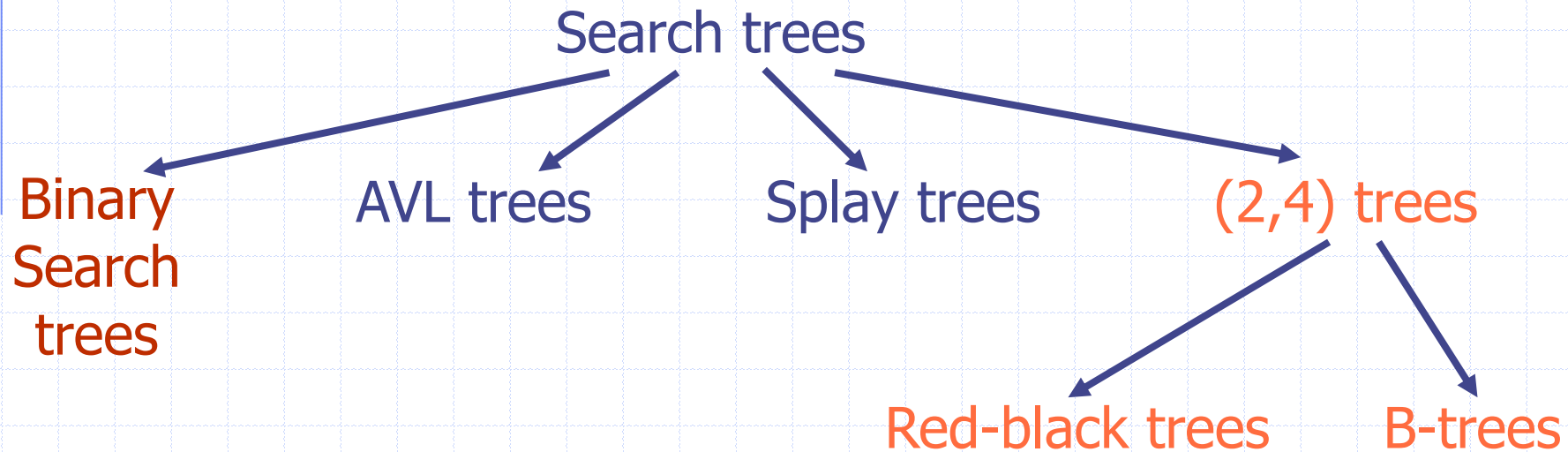
These notes cover many types of search trees.

Our key focus in COMP3506/7505 S2 2023 will be on **Binary Search Trees, AVL Trees, and Splay Trees.**

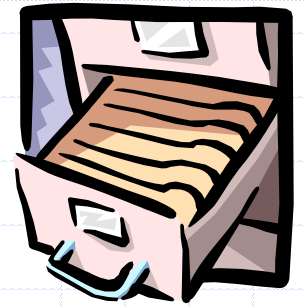
The notes on BST's and AVL trees are taken from the end of the Week 6 lecture to collect all tree material together logically.

The remaining trees may be useful for you to self study, but you are only expected to know about the existence and the basic intuition and properties of these structures, but not how they work in detail.

Different Types of Search Trees



Search Tables

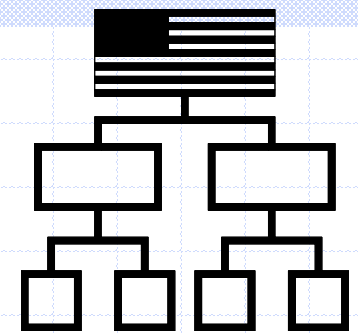


- ◆ An ordered map implemented by means of a sorted list
 - store the items in an array-based sequence
 - ◆ sorted by key
 - use an external comparator for the keys
 - use binary search to lookup keys

Search Tables: Performance

- ◆ Searches take $O(\log n)$ time
 - using binary search
- ◆ Inserting a new item takes $O(n)$ time
- ◆ Removing an item takes $O(n)$ time

Binary Search Trees



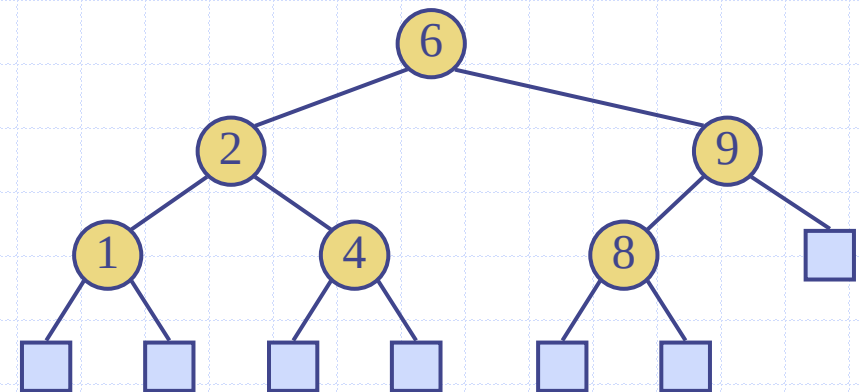
◆ Binary tree storing keys (or key-value entries) at its **internal nodes** and satisfying the following property:

- Let u , v , and w be three nodes such that u is in the left subtree of v and w is in the right subtree of v .
 $key(u) \leq key(v) \leq key(w)$

◆ **External nodes** do not store items

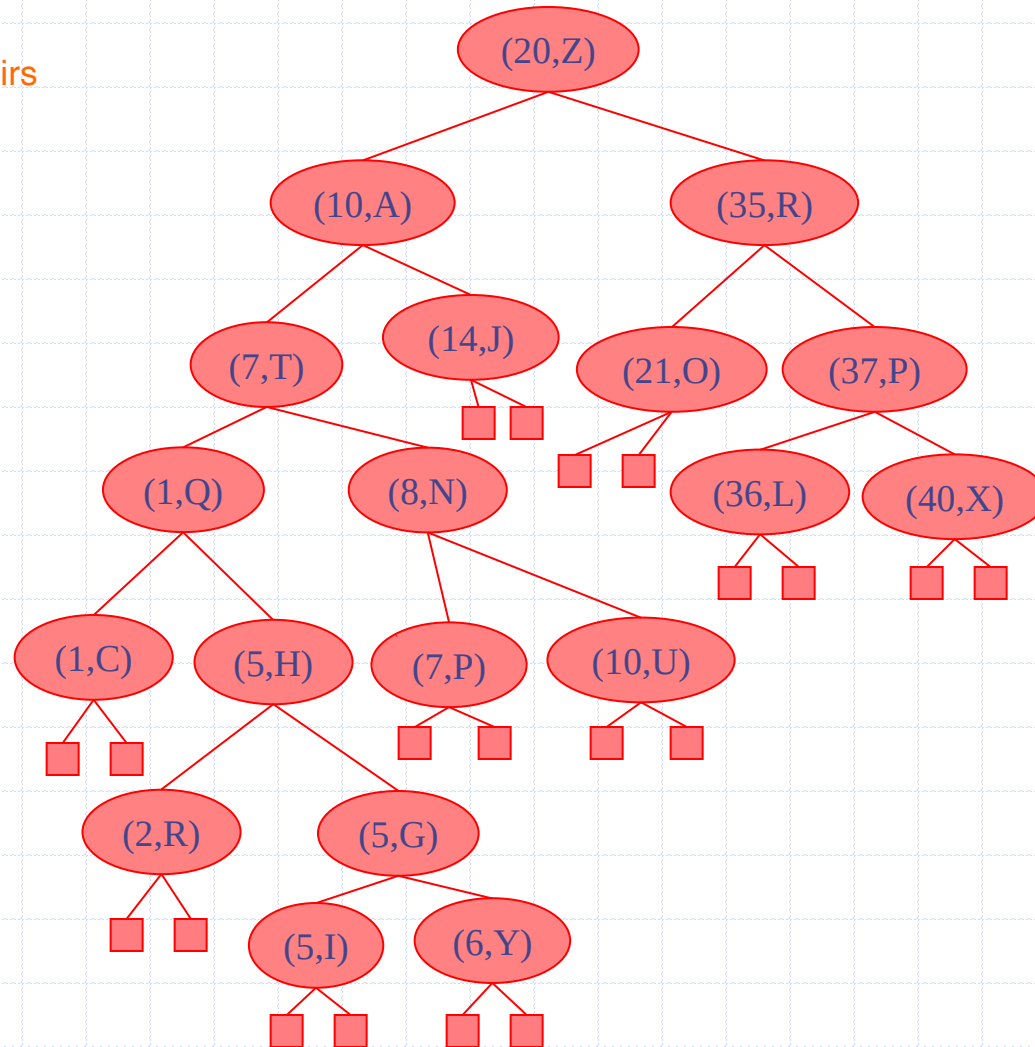
null pointers

◆ In-order traversal of a binary search tree visits the keys in increasing order

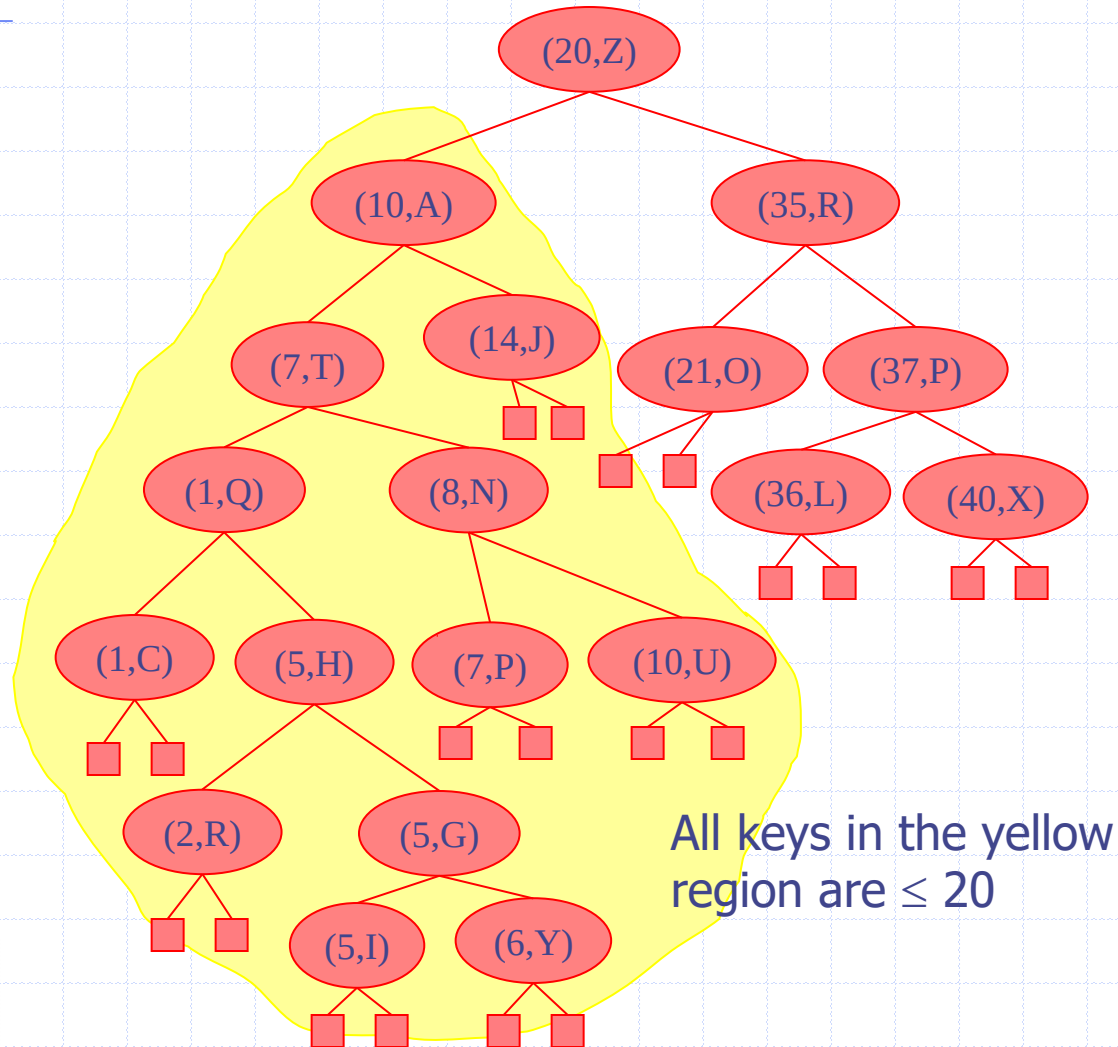


Binary Search Trees

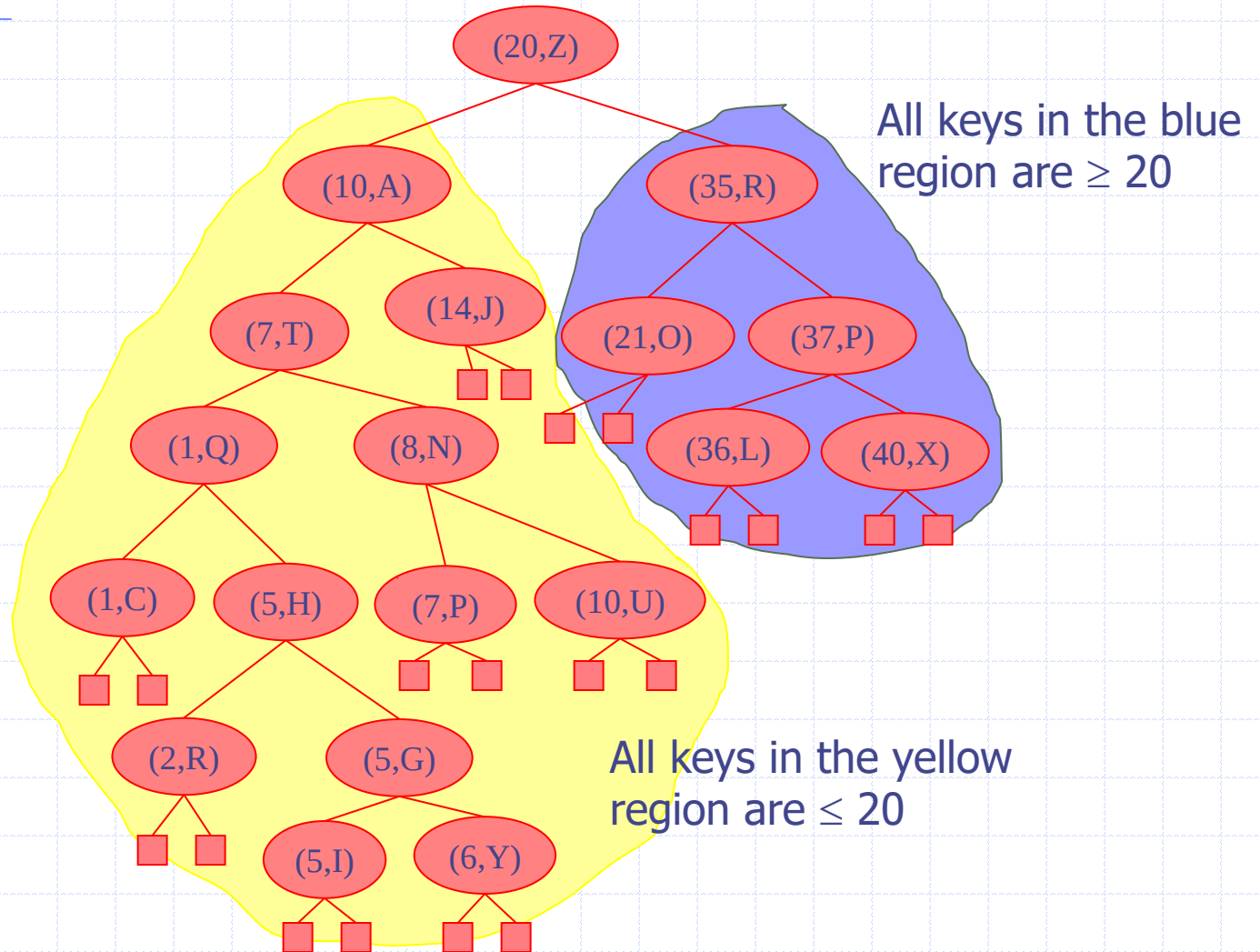
keys — tuples/ pairs



Binary Search Trees



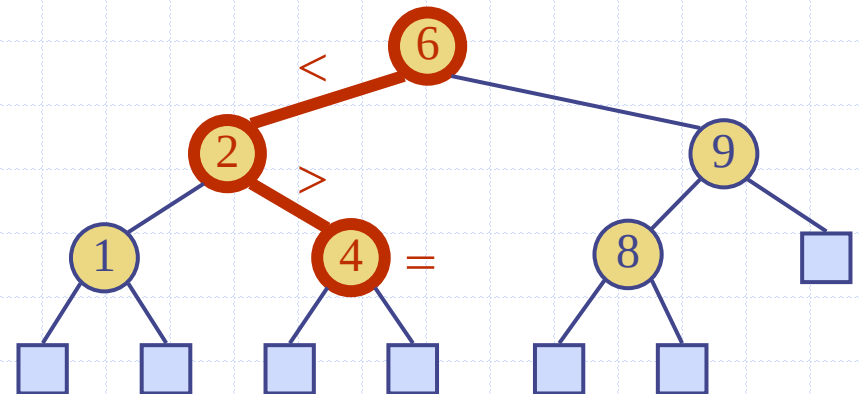
Binary Search Trees



Search for Key k

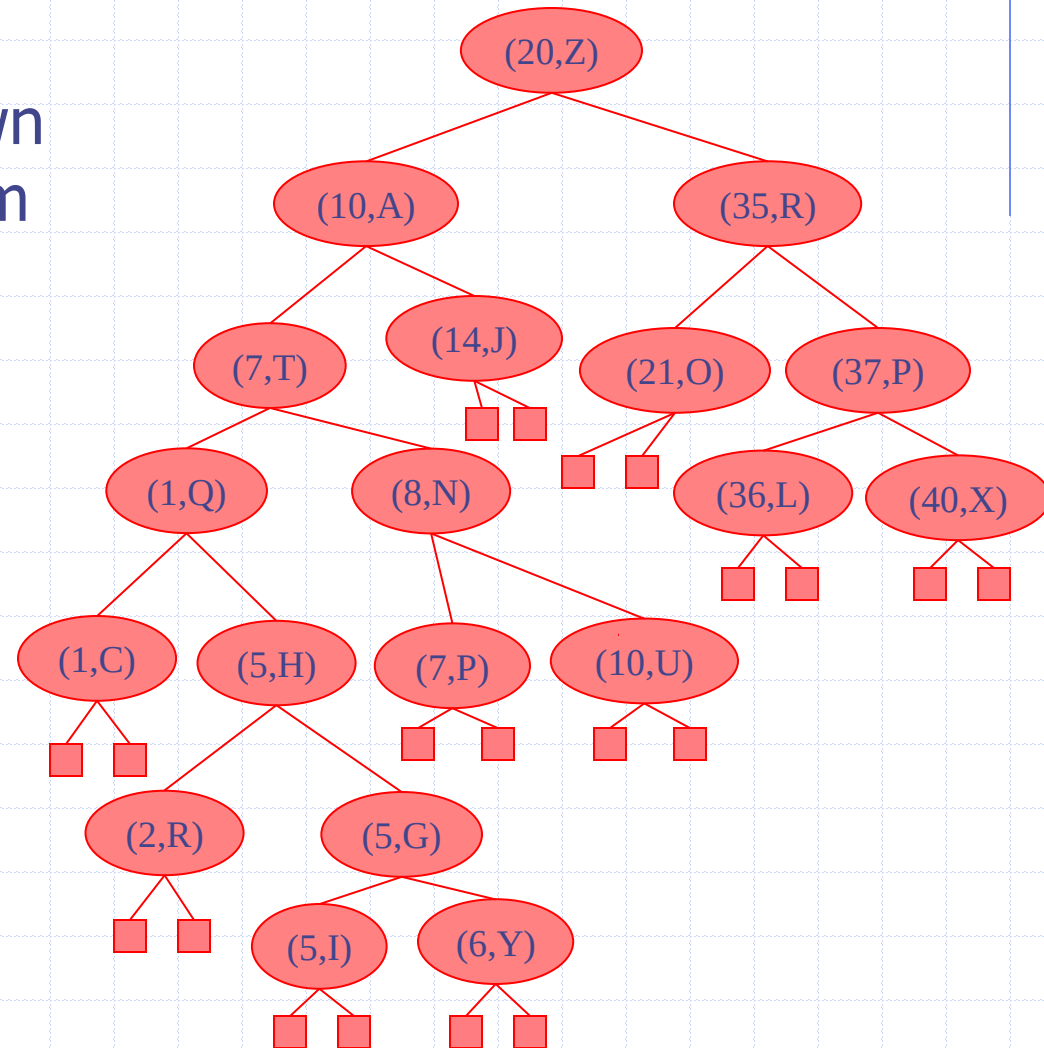
- ◆ Trace a downward path starting at the root
 - ◆ Next node visited depends on the comparison of k with the key of the current node
 - if a leaf is reached, the key is not found
 - ◆ Example: **get(4)**
 - call `TreeSearch(4, root)`
 - ◆ Algorithms for nearest neighbour queries are similar
- if key doesn't exist in the tree

```
Algorithm TreeSearch( $k, v$ ):  
  if T.isExternal ( $v$ )  
    return  $v$   
  if  $k < \text{key}(v)$   
    return TreeSearch( $k, \text{left}(v)$ )  
  else if  $k = \text{key}(v)$   
    return  $v$   
  else {  $k > \text{key}(v)$  }  
    return TreeSearch( $k, \text{right}(v)$ )
```



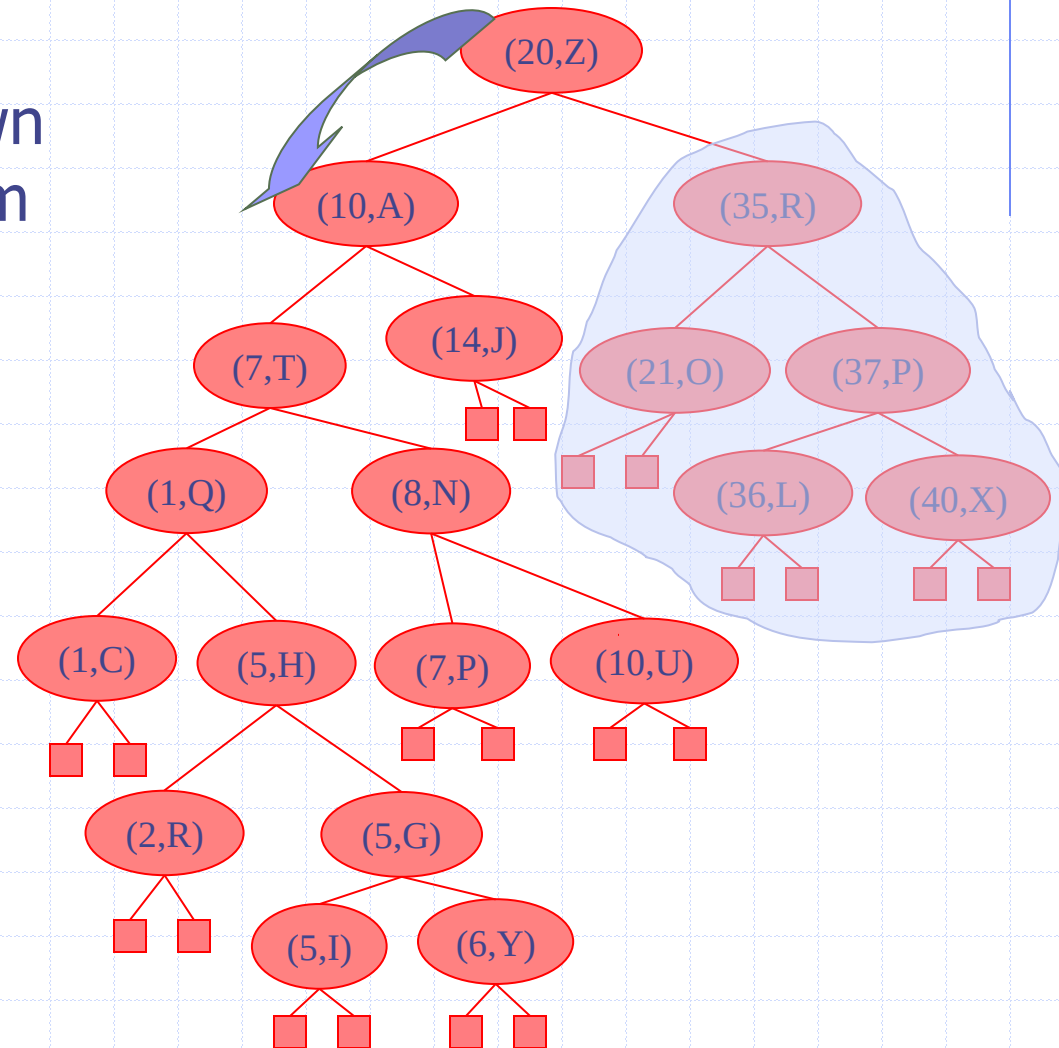
Searching

- ◆ Search proceeds down the tree to find item or an external node
- ◆ Example: Search for item with key 11



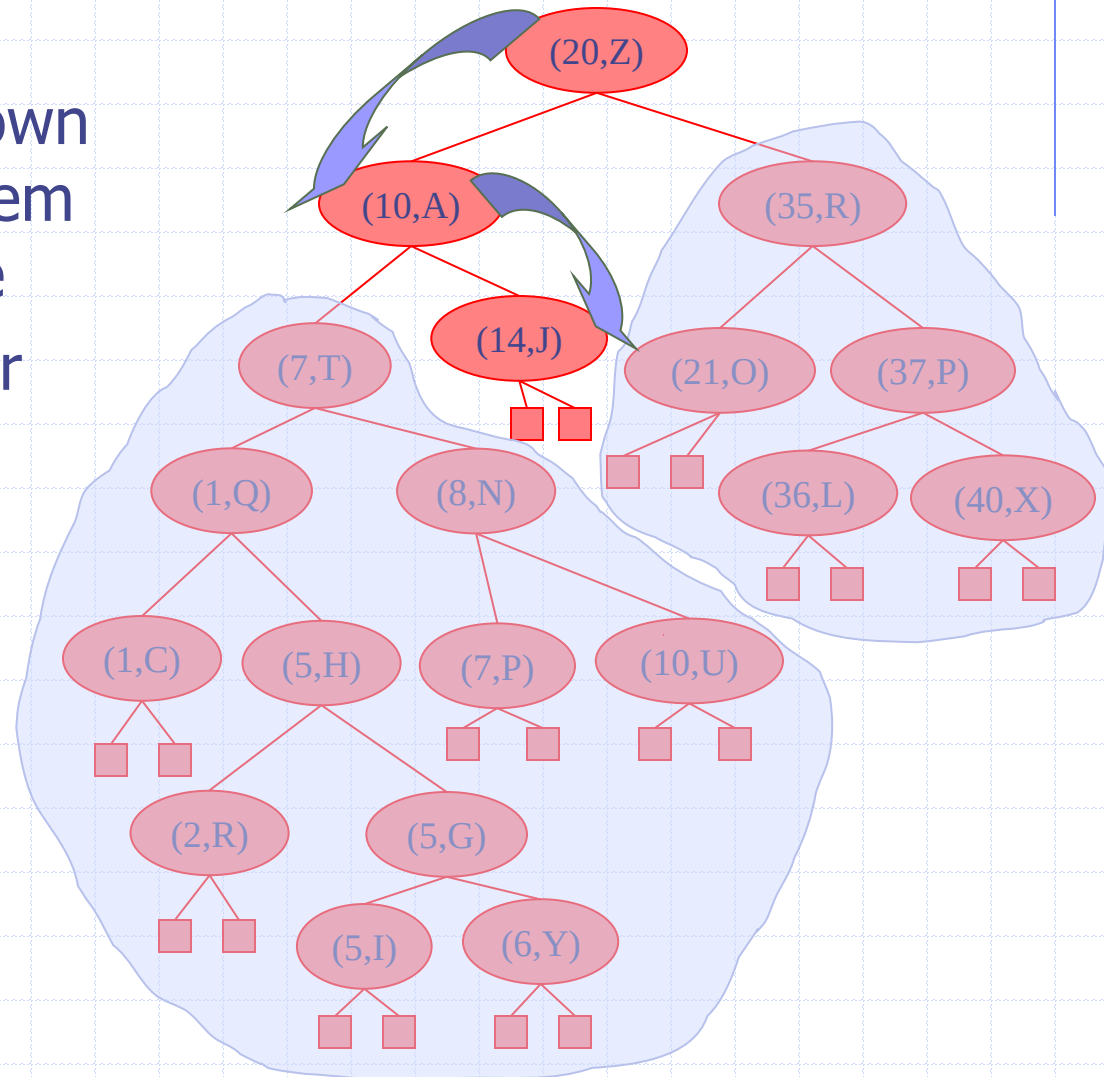
Searching

- ◆ Search proceeds down the tree to found item or an external node
- ◆ Example: Search for item with key 11



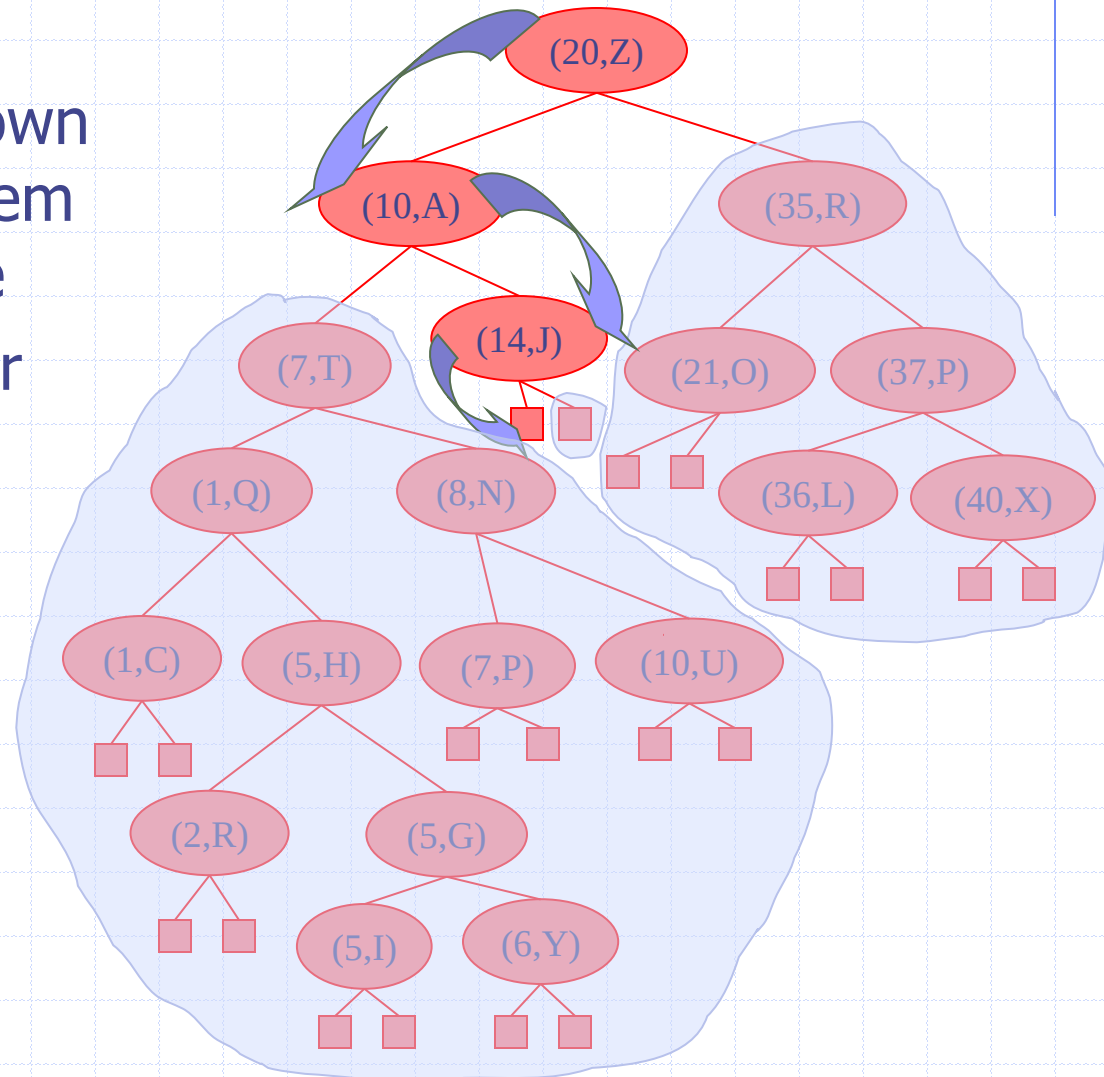
Searching

- ◆ Search proceeds down the tree to found item or an external node
- ◆ Example: Search for item with key 11



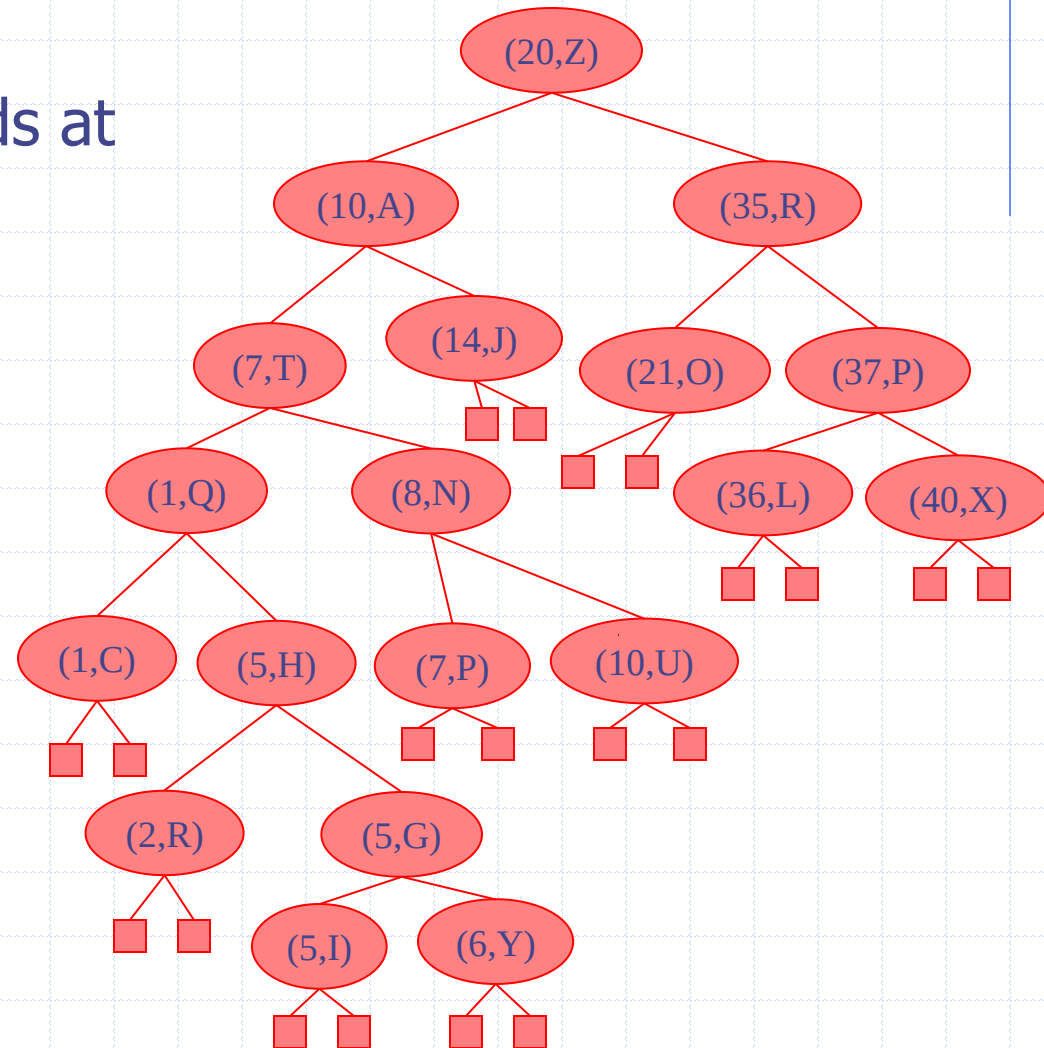
Searching

- ❖ Search proceeds down the tree to found item or an external node
- ❖ Example: Search for item with key 11



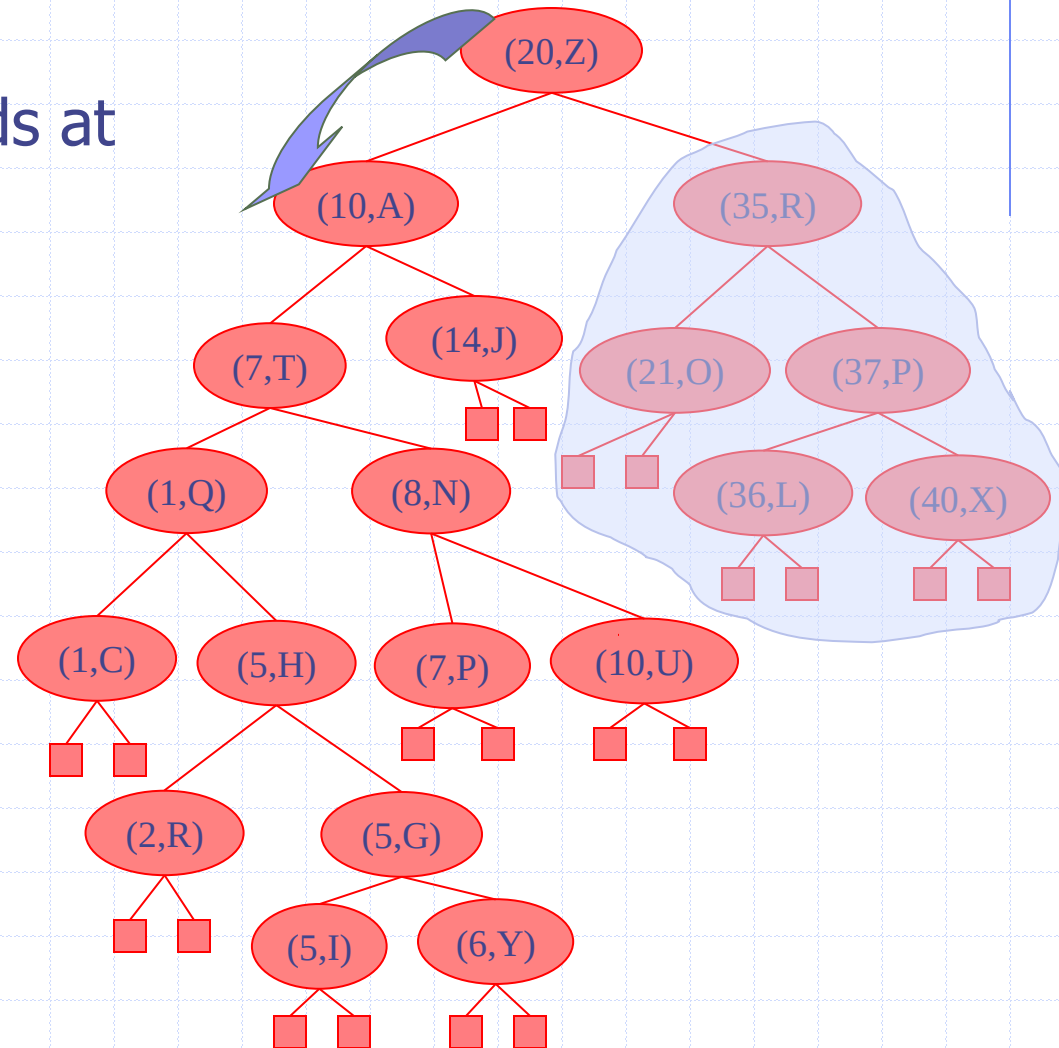
Searching

- ◆ Search for key 8, ends at an internal node



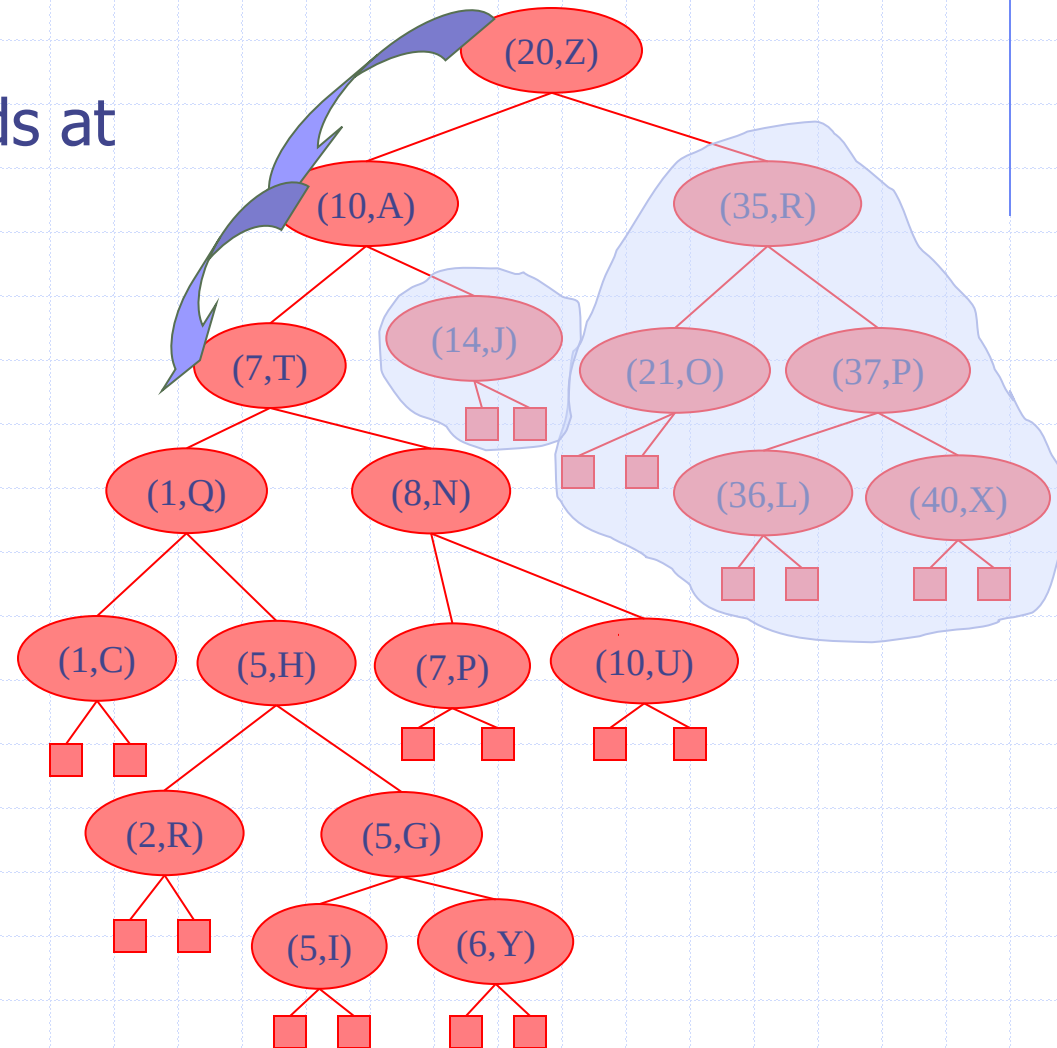
Searching

- ◆ Search for key 8, ends at an internal node



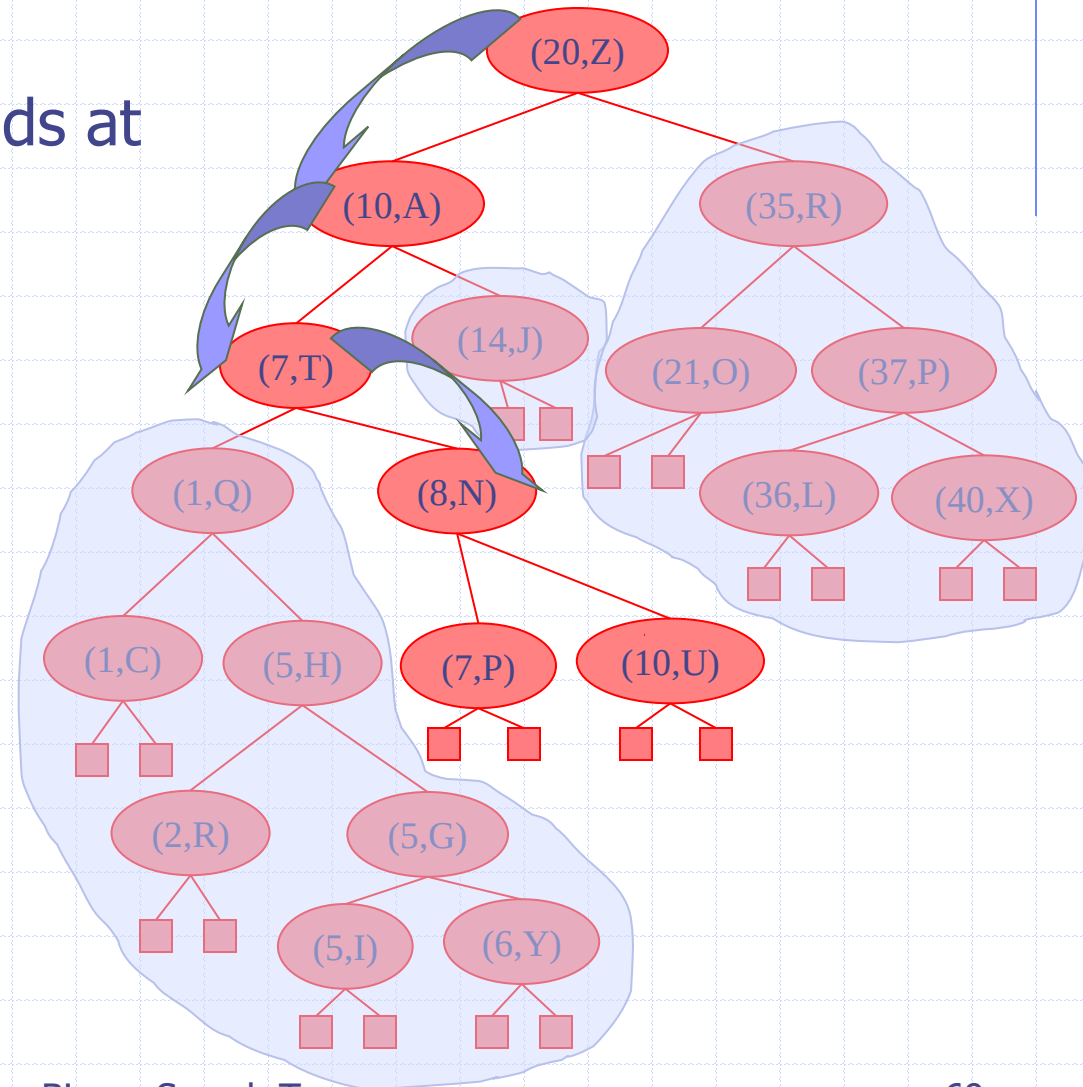
Searching

- ◆ Search for key 8, ends at an internal node



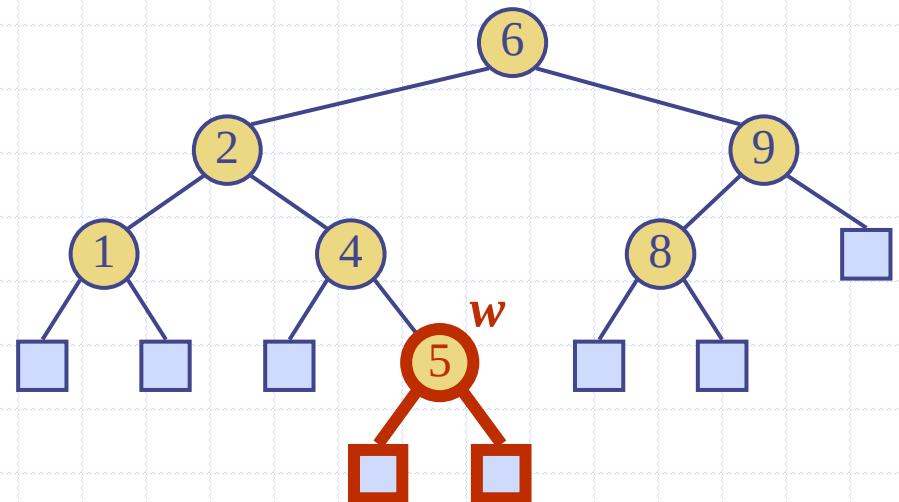
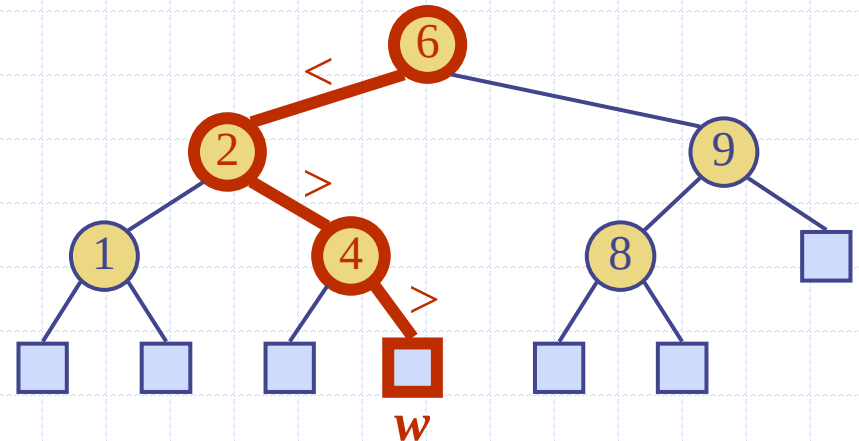
Searching

- ◆ Search for key 8, ends at an internal node



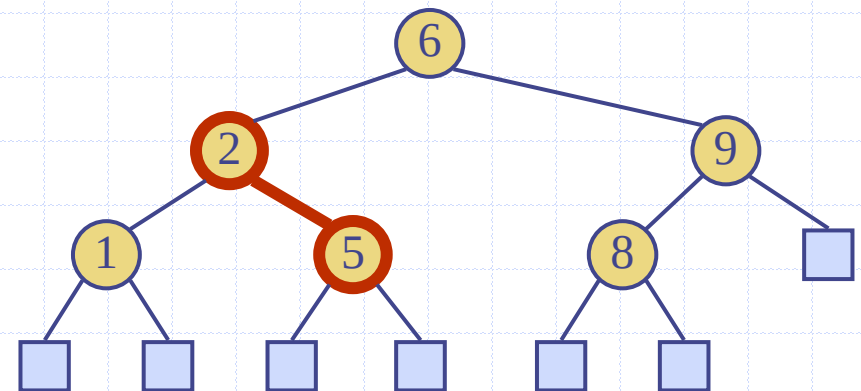
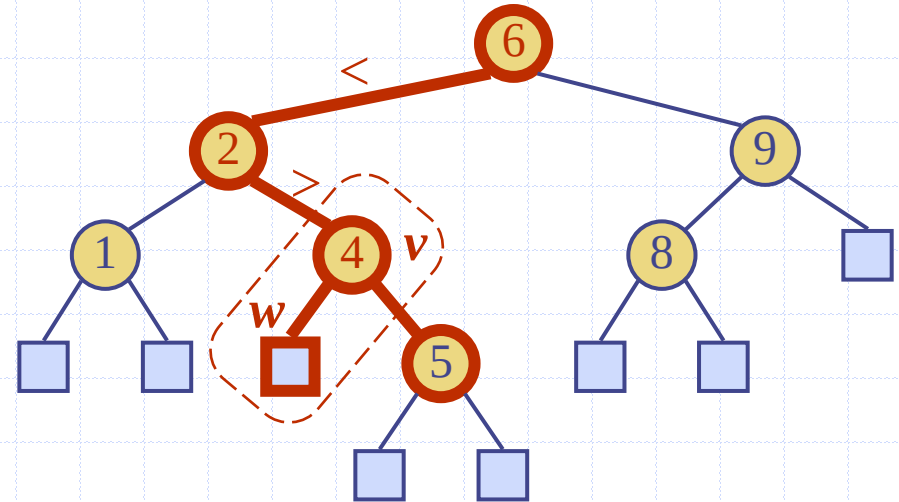
Insertion

- ◆ $\text{put}(k, o)$
 - search for key k
 - ◆ using `TreeSearch`
- ◆ Assume k is not already in the tree
 - w is the leaf reached by the search
- ◆ Insert k at node w and expand w into an internal node
- ◆ Example: insert 5



Deletion

- ◆ **remove(k)**
 - search for key k
- ◆ Assume key k is in the tree
 - v is the node storing k
- ◆ If node v has a leaf child w
 - remove v and w with **removeExternal(w)**
 - ◆ removes w and its parent
- ◆ Example: remove 4

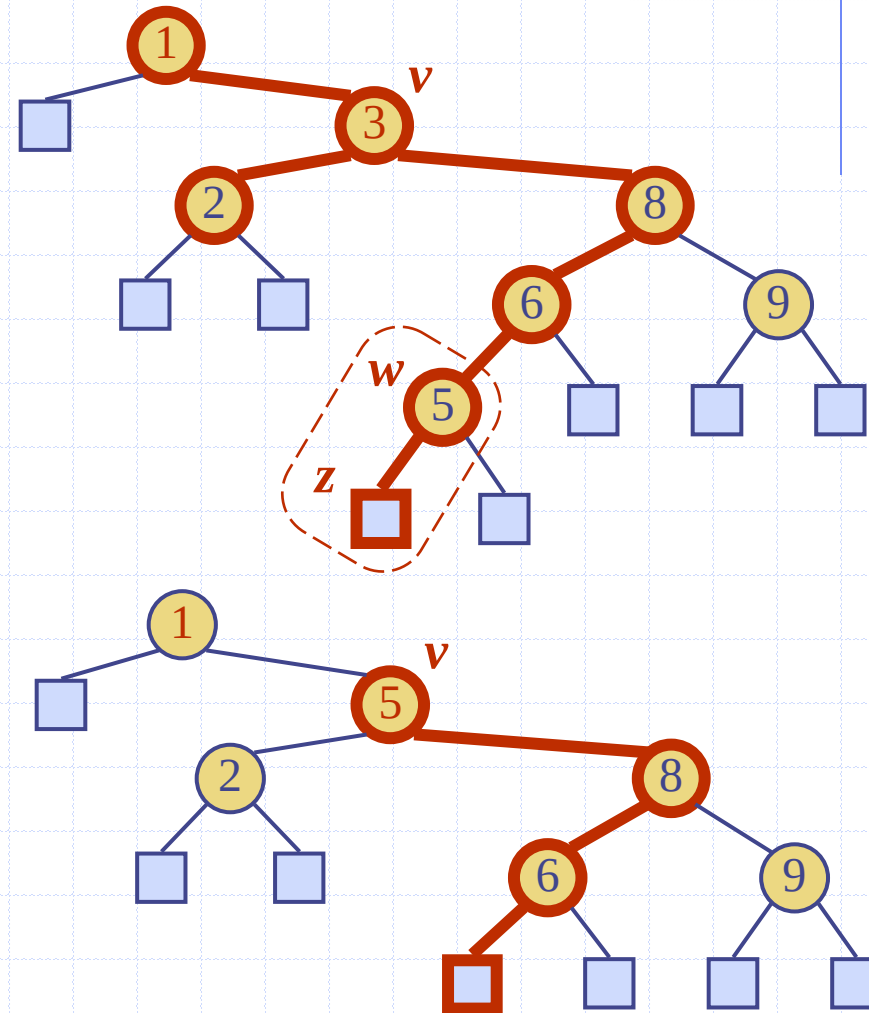


Deletion (cont.)

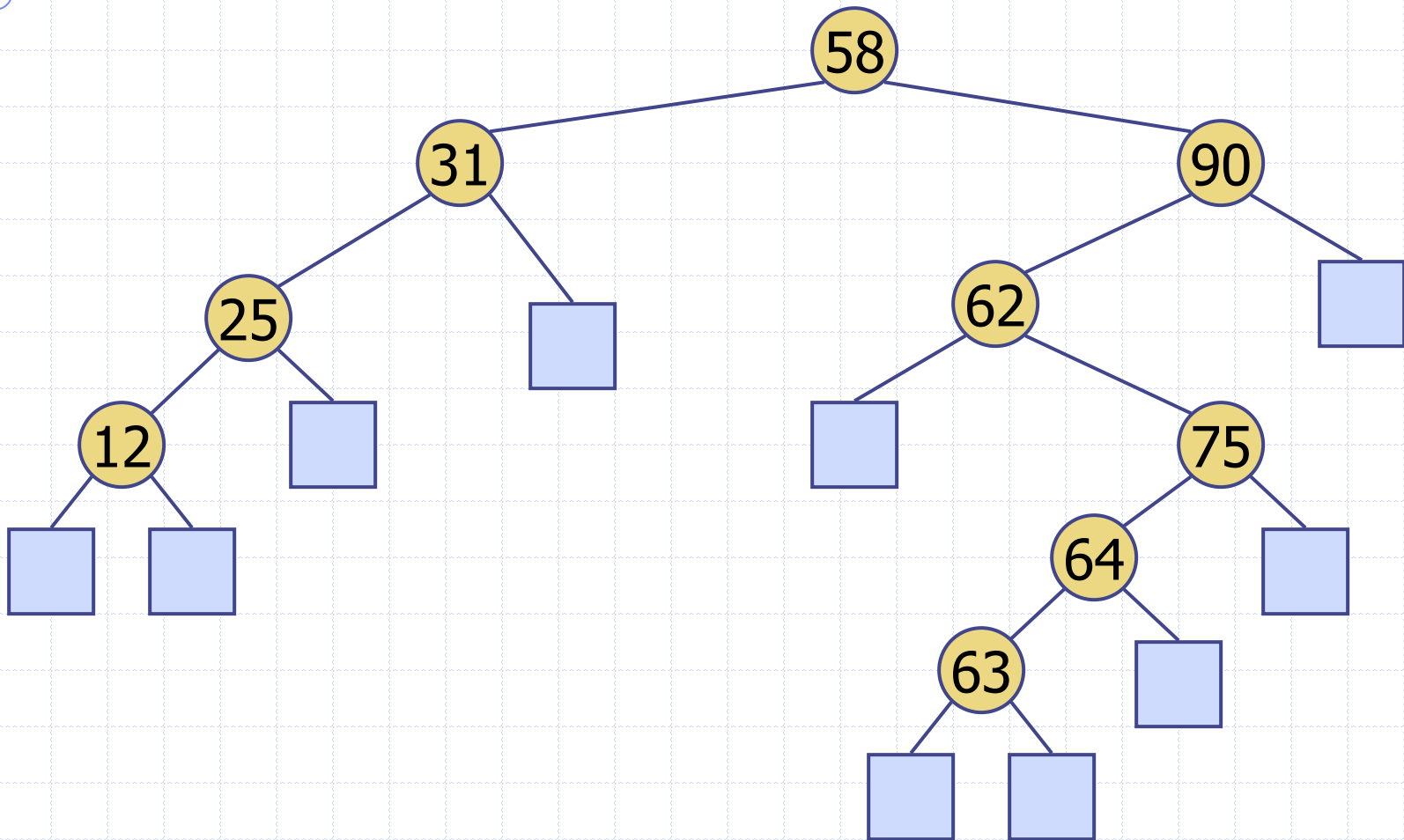
◆ Consider the case where the key k to be removed is stored at a node v whose children are both internal

- find internal node w that follows v in an **in-order** traversal
- copy $key(w)$ into node v
- remove node w and its left child z
 - ◆ must be a leaf

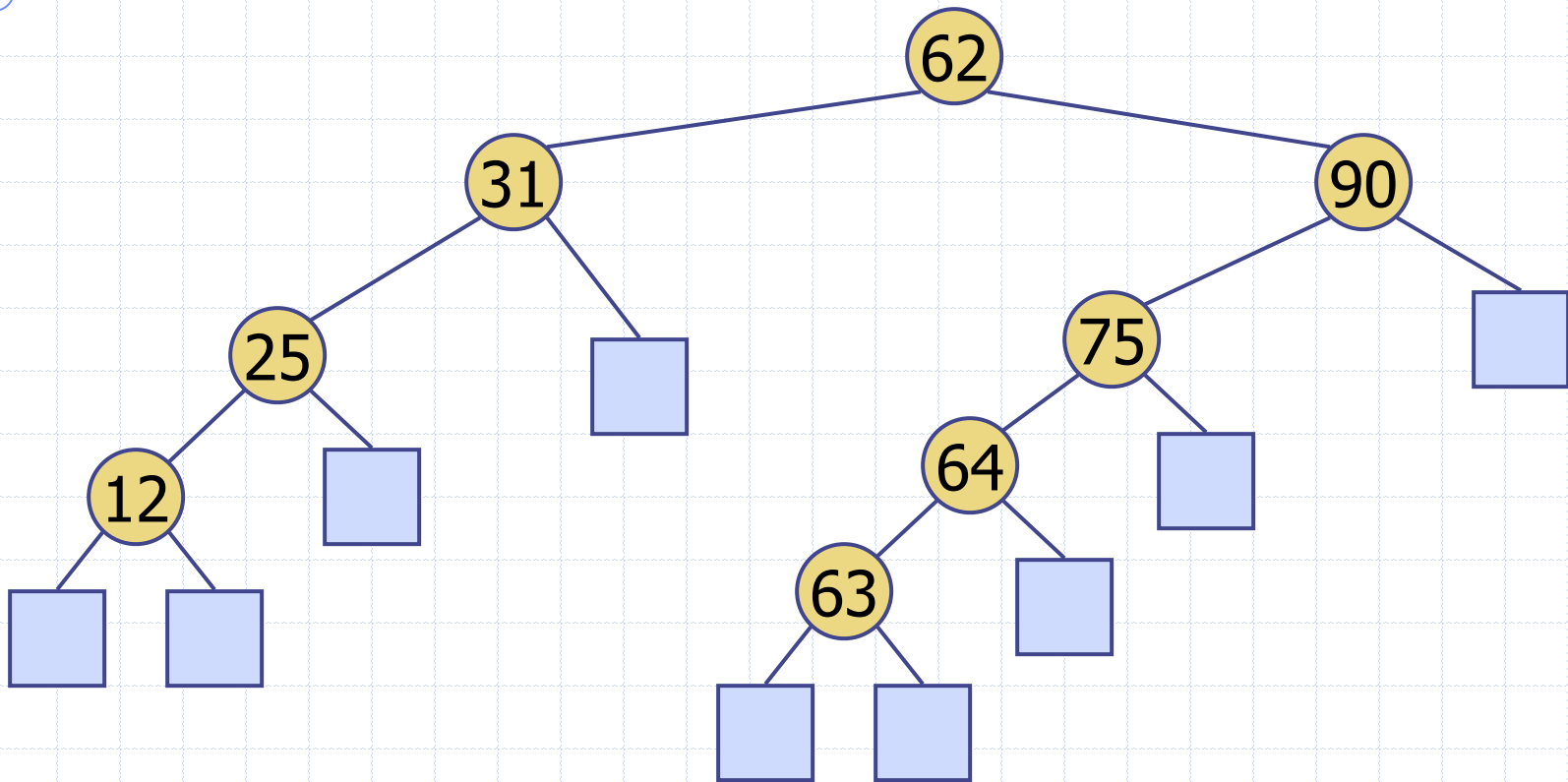
◆ Example: remove 3



Example: What steps would be involved in deleting the node with key=58?



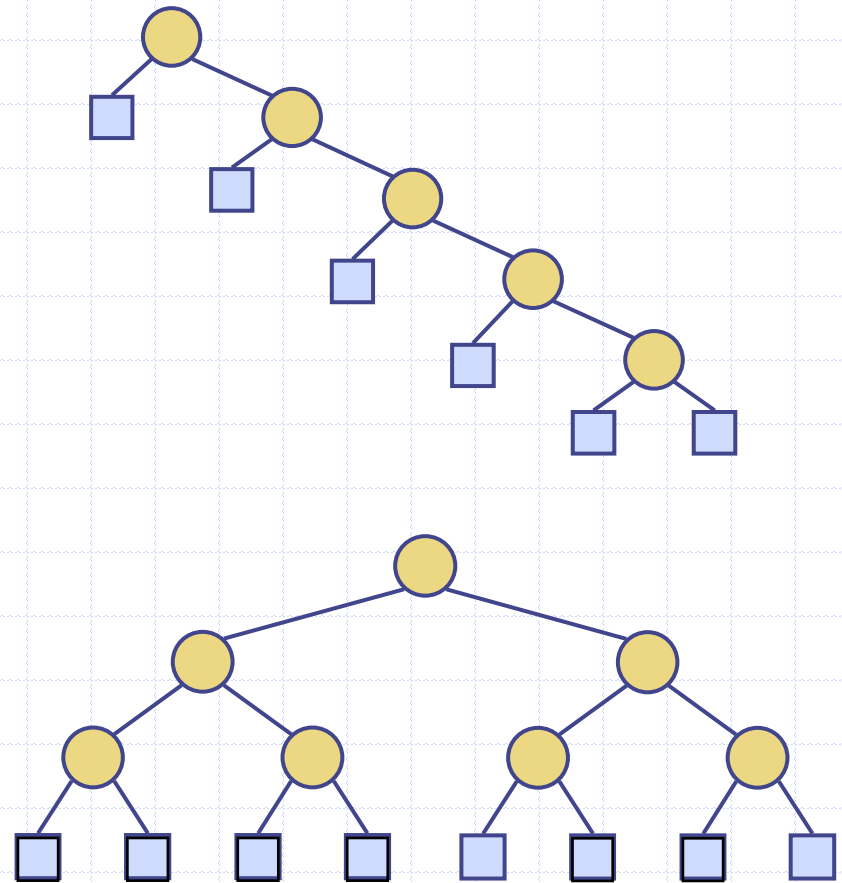
Example: What steps would be involved in deleting the node with key=58?



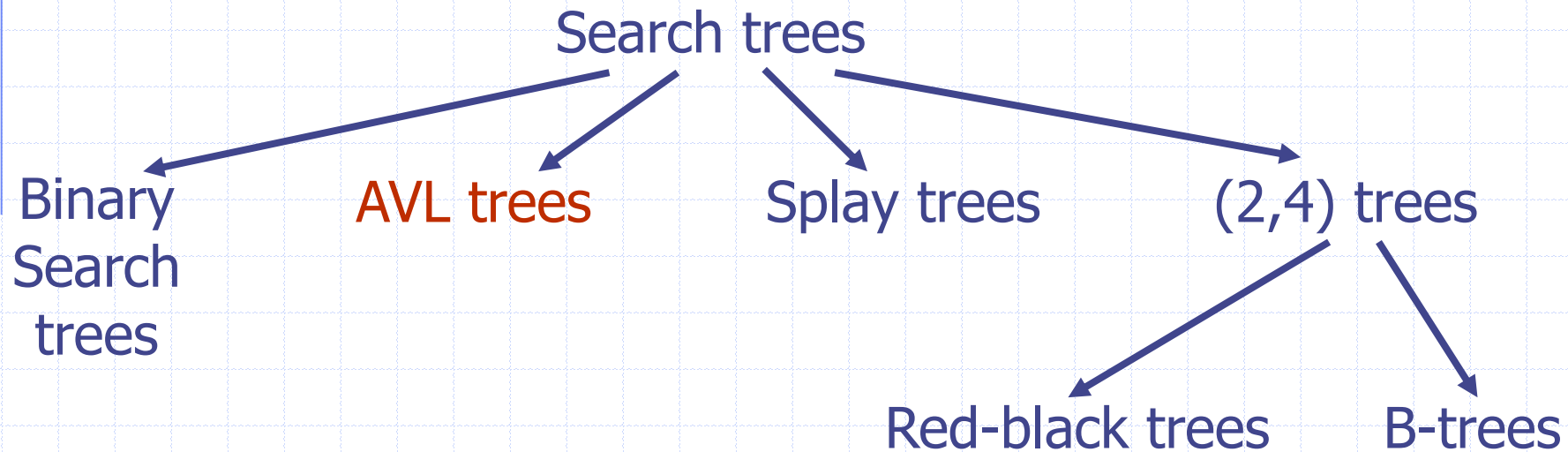
Performance

- ◆ Consider an ordered map with n items implemented by means of a binary search tree of height h
 - space used is $O(n)$
 - **get**, **put** and **remove** take $O(h)$ time
- ◆ Height h is
 - $O(n)$ in the worst case
 - $O(\log n)$ in the best case

BST corresponds to quicksort execution
pivot the tree wisely (about halfway)



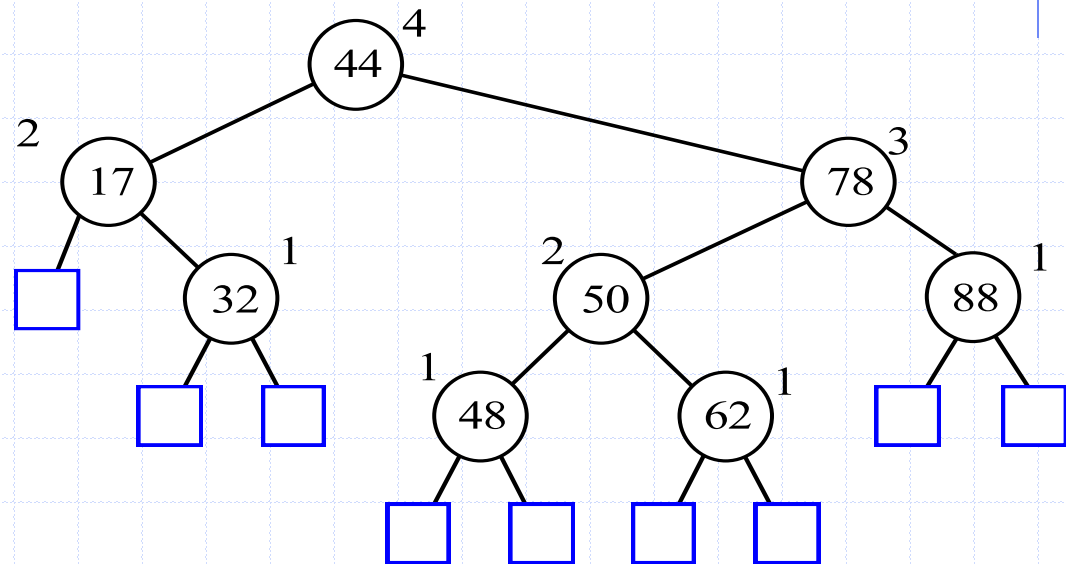
Different Types of Search Trees



AVL Tree Definition

◆ Balanced binary search tree

- for every internal node v , the heights of the children of v can differ by at most 1

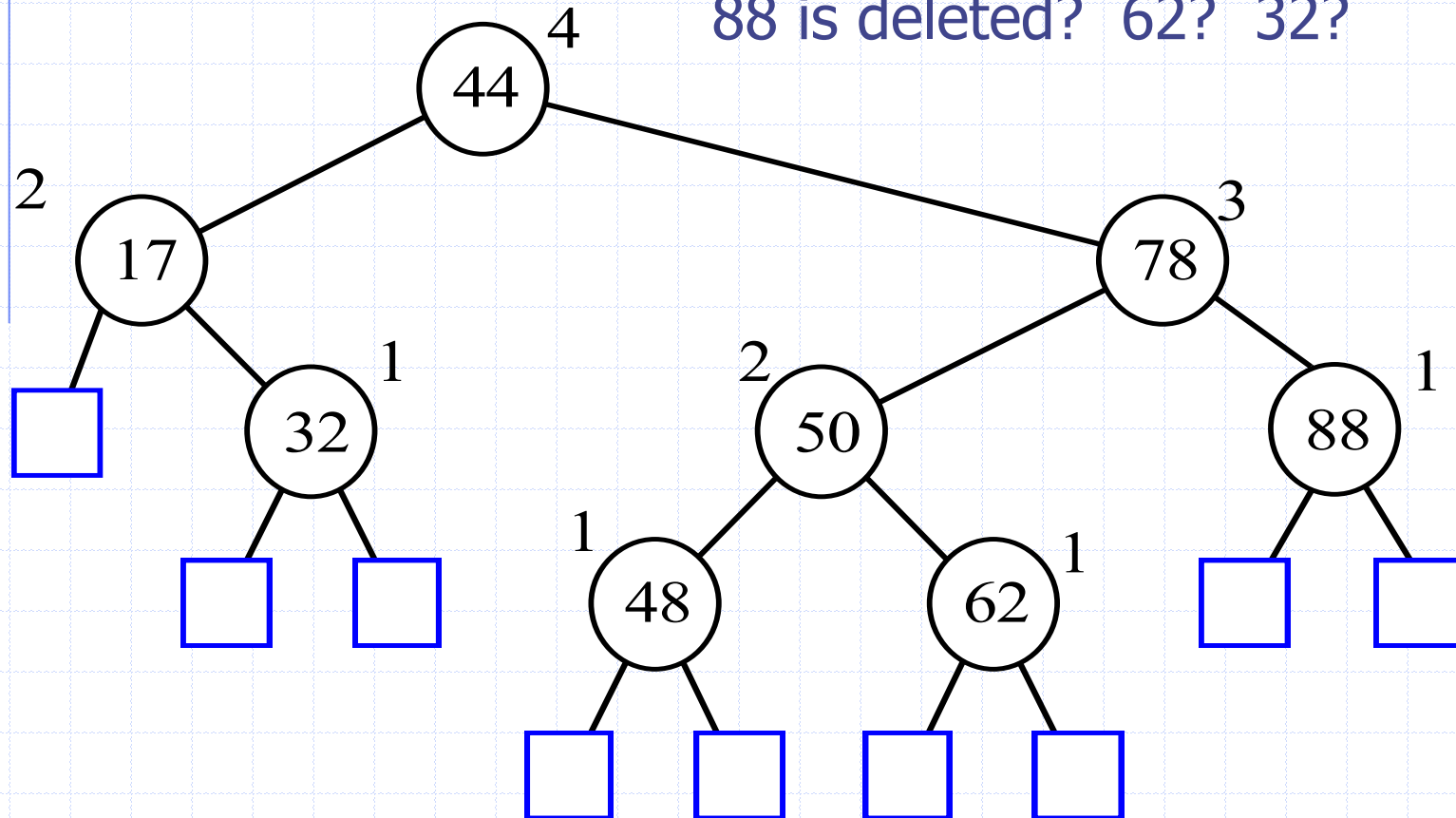


Example of an AVL tree where the heights are shown next to the nodes

Example: Height-balanced tree

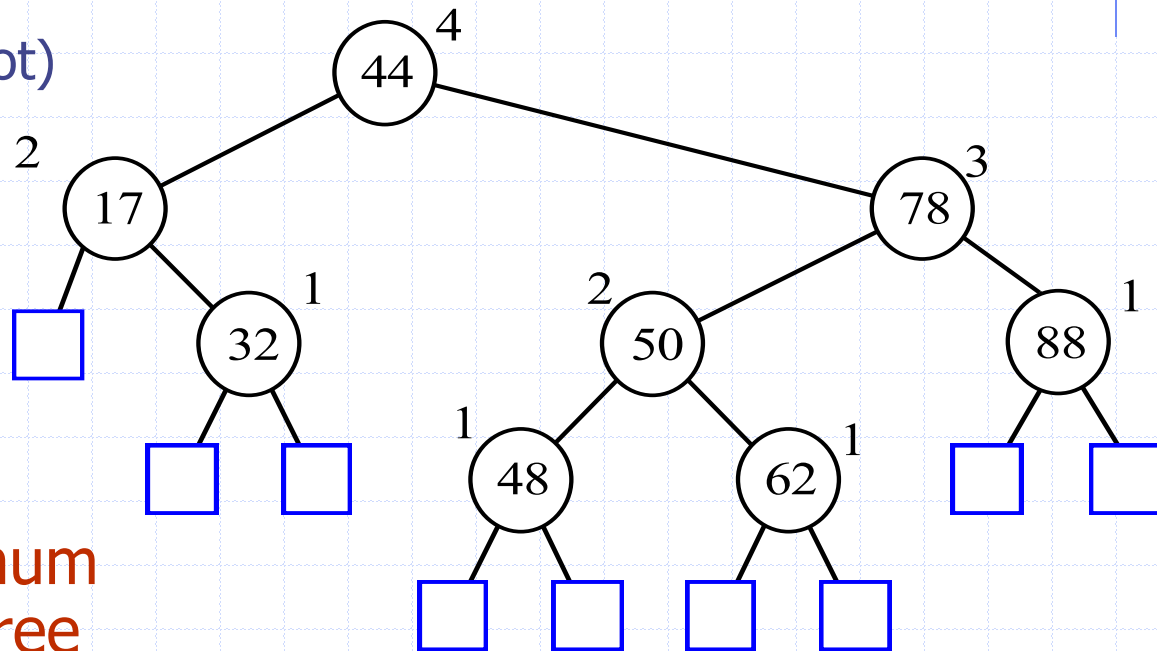
Height-Balance property: for every internal node v , the heights of the children of v differ by at most 1

Does the tree remain balanced if 88 is deleted? 62? 32?



AVL Tree Search is $O(h)$

- ◆ Example: `get(48)`
 - `TreeSearch(48, root)`



- ◆ What is the maximum height of an AVL tree with n nodes?

Equivalently, what is the minimum number of nodes in an AVL tree with height h , $n(h)$?

◆ $n(h) = 1$ if $h = 1$

◆ $n(h) = 2$ if $h = 2$

◆ $n(h) = 1 + n(h-1) + n(h-2)$ if $h \geq 3$

$> 2n(h-2)$

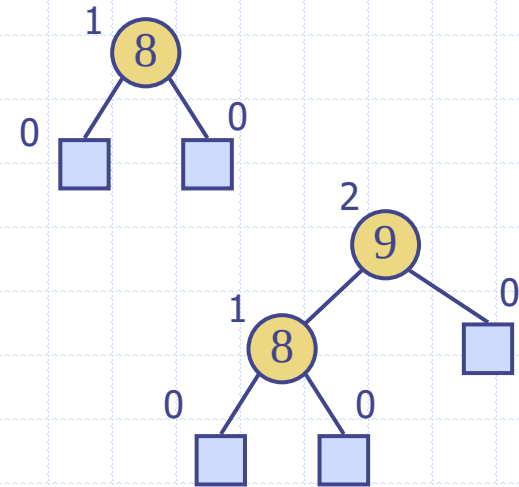
$> 4n(h-4)$ if $h \geq 5$

$> 8n(h-6)$ if $h \geq 7$

...

$> 2^i n(h-2i)$ if $h \geq 2i + 1$

$> 2^{(h-1)/2}$ for base case $h = 2i + 1$



PS: let's count the height of an empty tree as 0

Equivalently, what is the minimum number of nodes in an AVL tree with height h , $n(h)$?

◆ $n(h) = 1$ if $h = 1$

◆ $n(h) = 2$ if $h = 2$

◆ $n(h) = 1 + n(h-1) + n(h-2)$ if $h \geq 3$

$> 2n(h-2)$

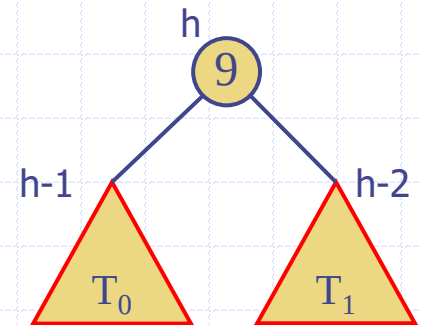
$> 4n(h-4)$ if $h \geq 5$

$> 8n(h-6)$ if $h \geq 7$

...

$> 2^i n(h-2i)$ if $h \geq 2i + 1$

$> 2^{(h-1)/2}$ for base case $h = 2i + 1 \rightarrow i = (h-1)/2$



PS: let's count the height of an empty tree as 0

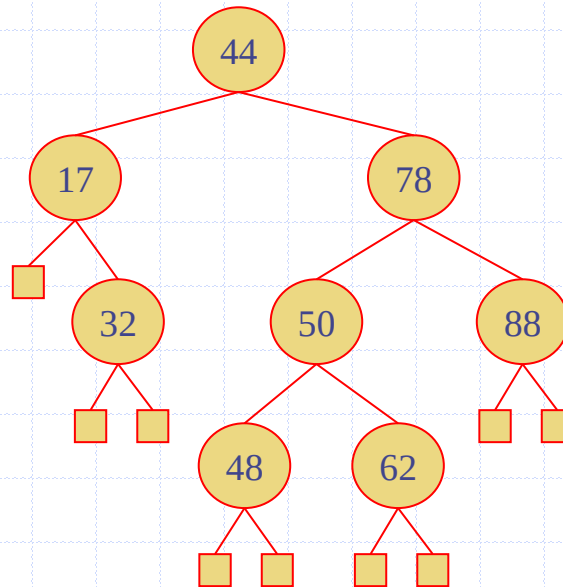
Height of an AVL Tree

- ◆ So we get: $2^{(h-1)/2} < n(h)$
- ◆ Taking logarithms: $h < 2 \log n(h) + 1$

Conclusion: height of an AVL tree storing n keys is $O(\log n)$

Insertion

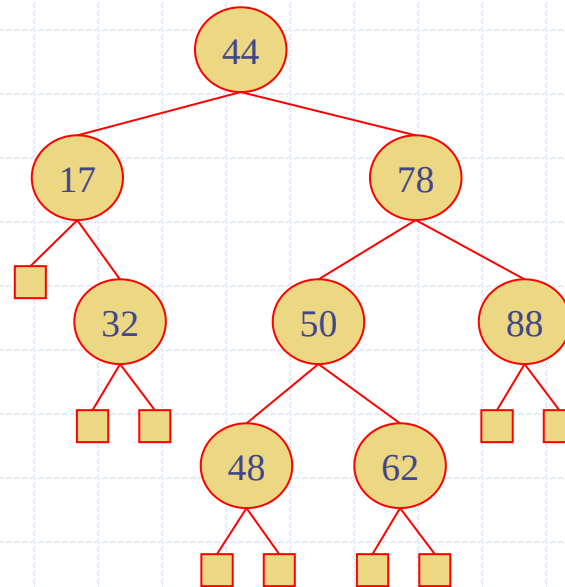
- ◆ Insertion is as in a binary search tree
 - by expanding an external node
- ◆ Example
 - insert 54



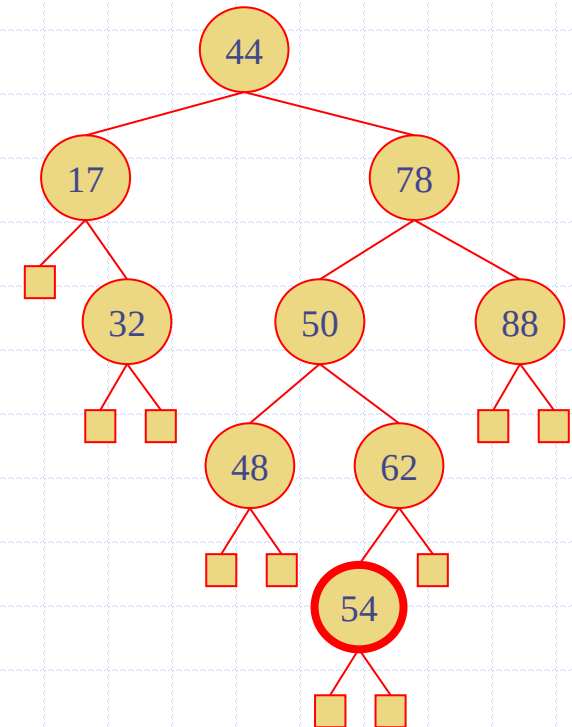
before insertion

Insertion

- ◆ Insertion is as in a binary search tree
 - by expanding an external node
- ◆ Example
 - insert 54



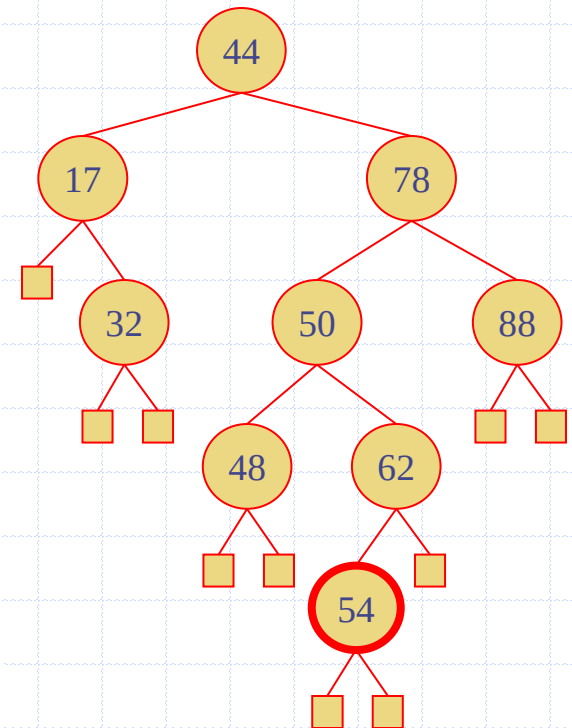
before insertion



after insertion

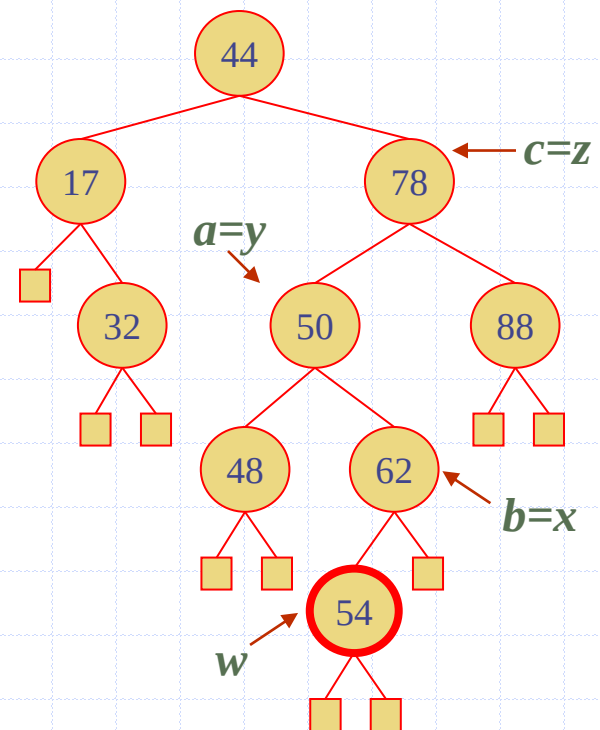
Insertion in an AVL Tree

- ◆ At worst, one or more ancestors will change from
 - having children with height difference 1 to
 - having children with height difference 2
- ◆ Rebalancing the first unbalanced ancestor will rebalance the tree
- ◆ Need to preserve in-order relation
 - using tri-node restructuring



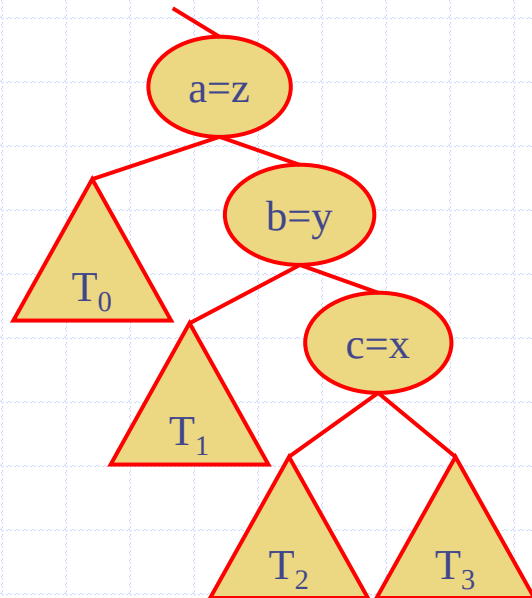
Insertion in an AVL Tree

- ◆ Insertion is as in a binary search tree, **plus rebalance**
- ◆ **z** is the **first unbalanced node** encountered while traveling up the tree from **w**
- ◆ **y** is the **child of z** with the larger height
- ◆ **x** is the **child of y** with the larger height
- ◆ **restructure(x)** to restore balance at **z**



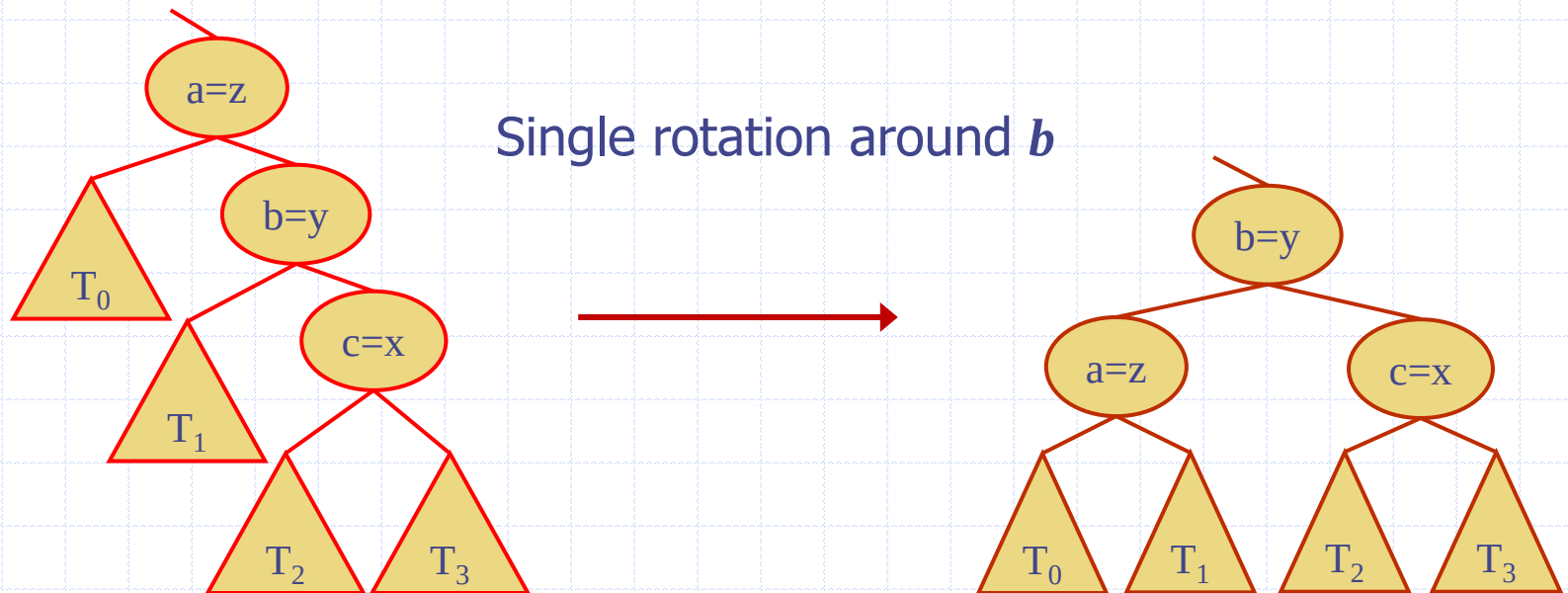
Tri-node Restructuring (Case 1)

- ◆ Let (a,b,c) be the in-order listing of x, y, z
- ◆ Perform the rotations needed to make b the topmost node of the three



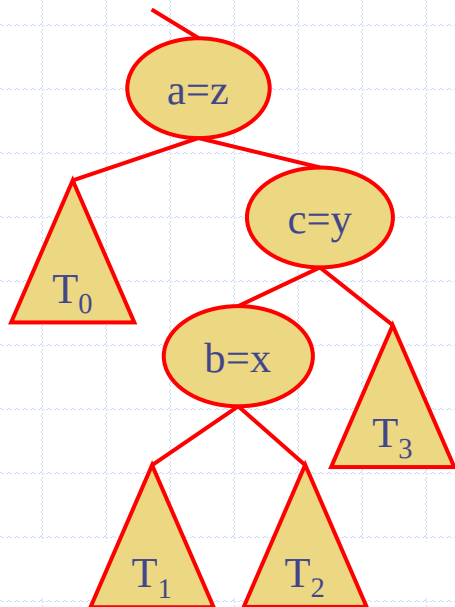
Tri-node Restructuring (Case 1)

- ◆ Let (a,b,c) be the in-order listing of x, y, z
- ◆ Perform the rotations needed to make b the topmost node of the three



Tri-node Restructuring (Case 2)

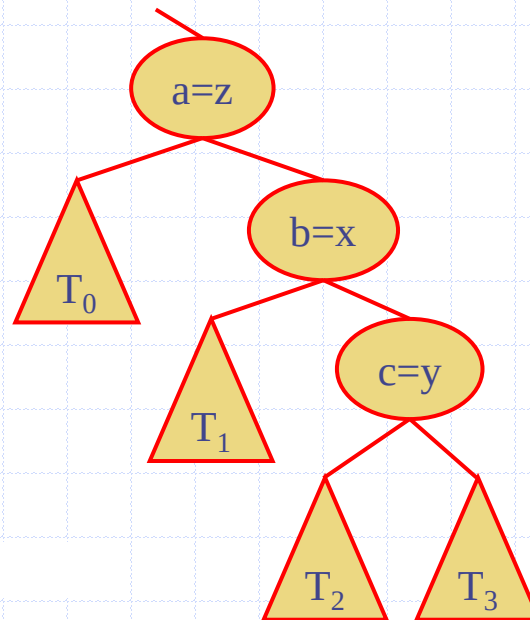
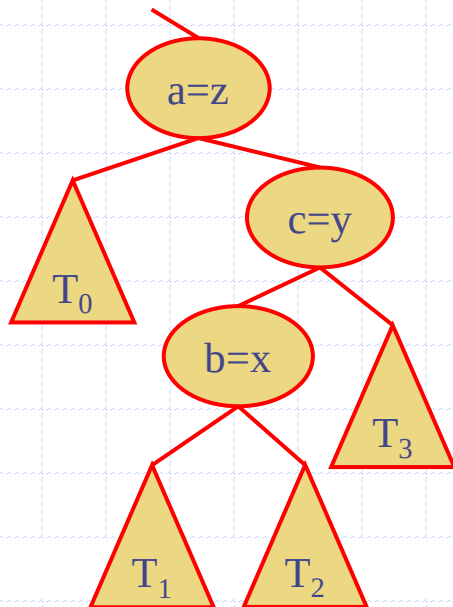
- ◆ Let (a,b,c) be the in-order listing of x, y, z
- ◆ Perform the rotations needed to make b the topmost node of the three



Tri-node Restructuring (Case 2)

- ◆ Let (a,b,c) be the in-order listing of x, y, z
- ◆ Perform the rotations needed to make b the topmost node of the three

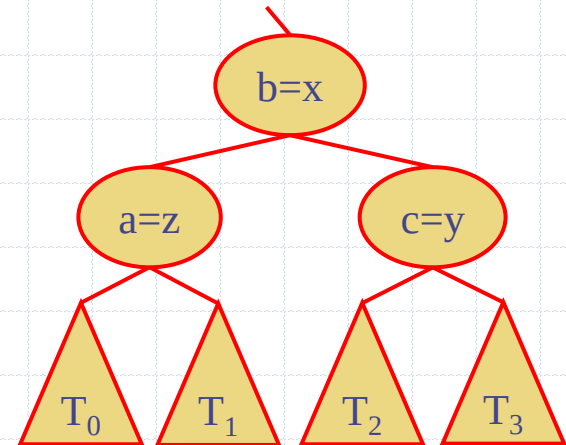
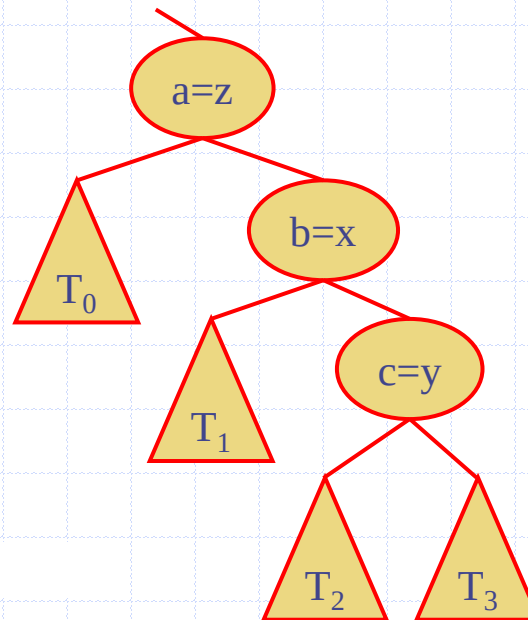
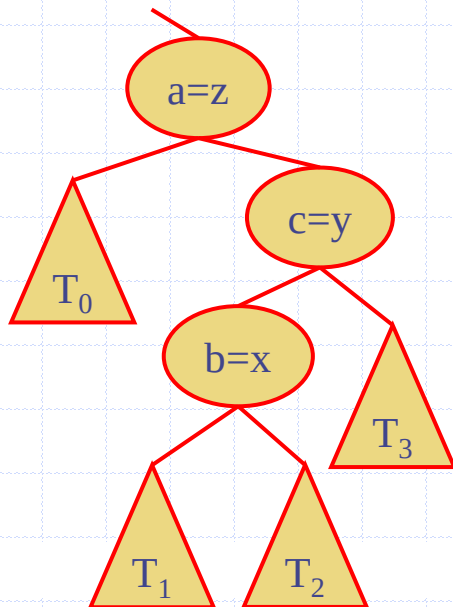
Double rotation around c and a



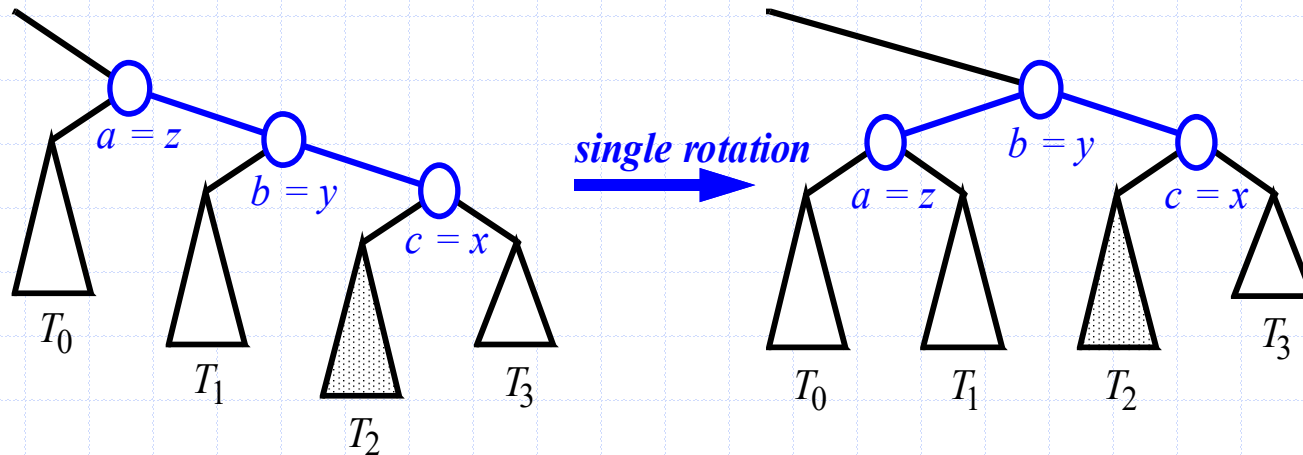
Tri-node Restructuring (Case 2)

- ◆ Let (a,b,c) be the in-order listing of x, y, z
- ◆ Perform the rotations needed to make b the topmost node of the three

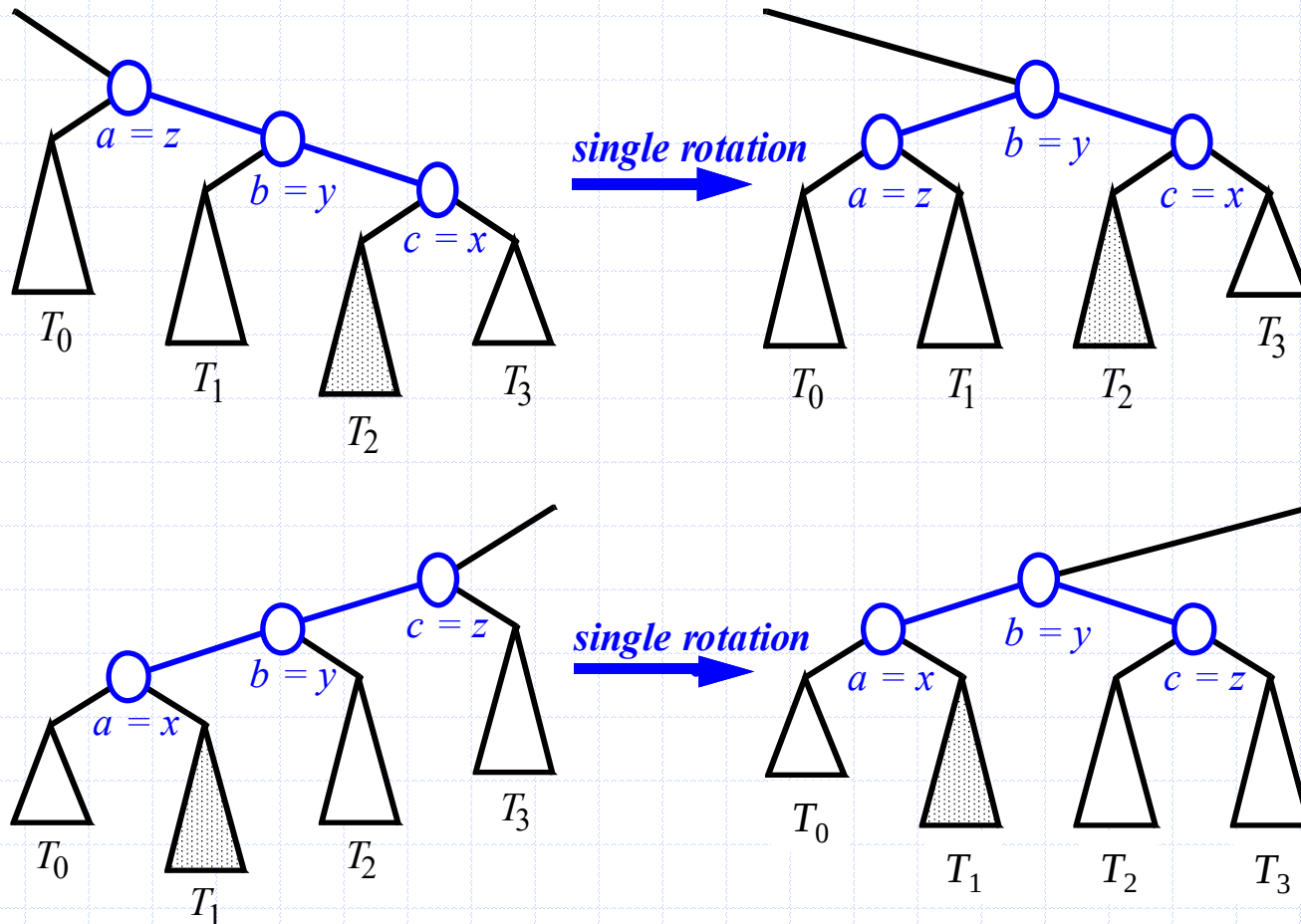
Double rotation around c and a



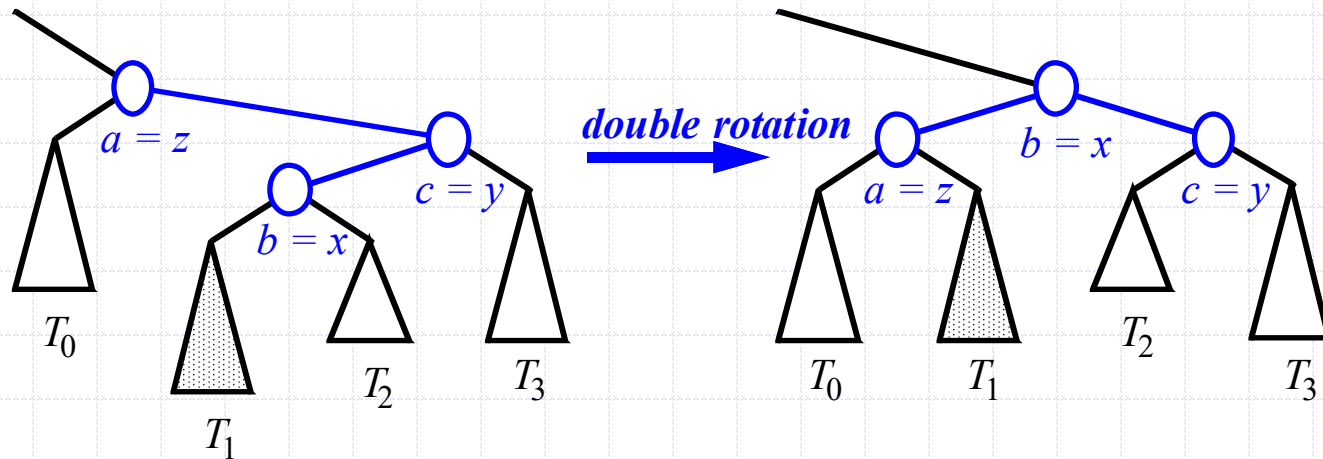
Restructuring (as Single Rotations)



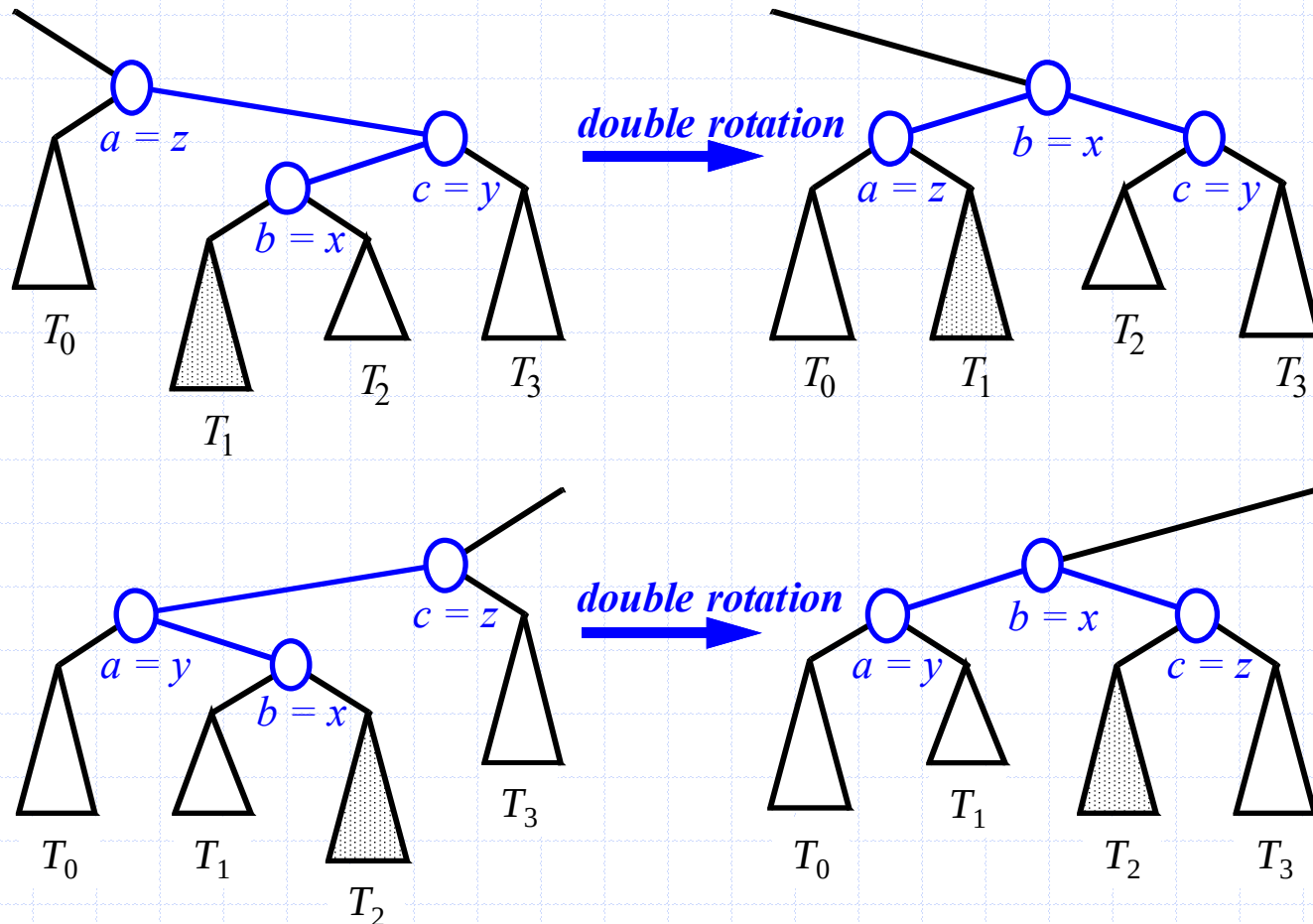
Restructuring (as Single Rotations)



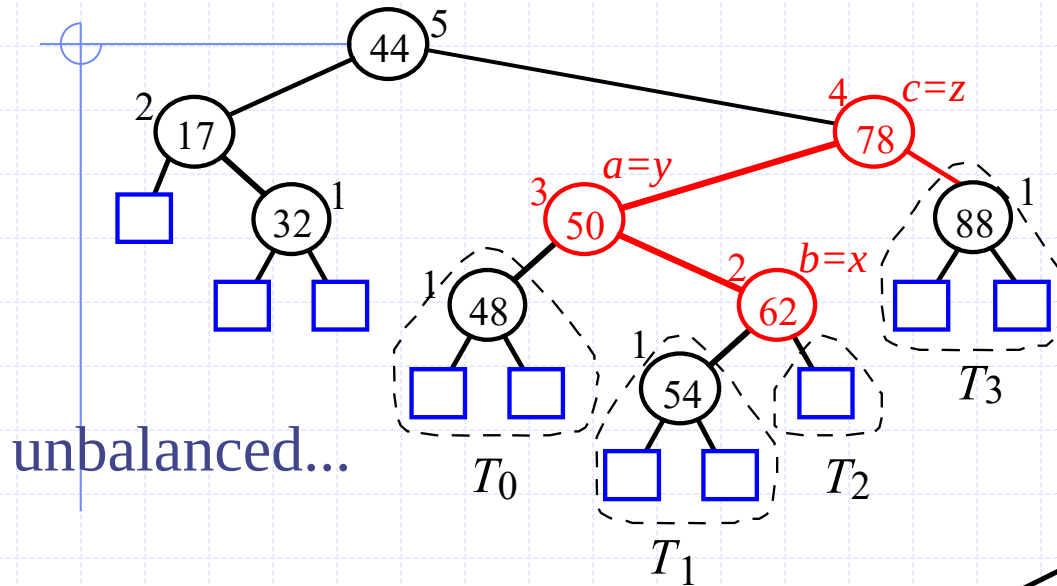
Restructuring (as Double Rotations)



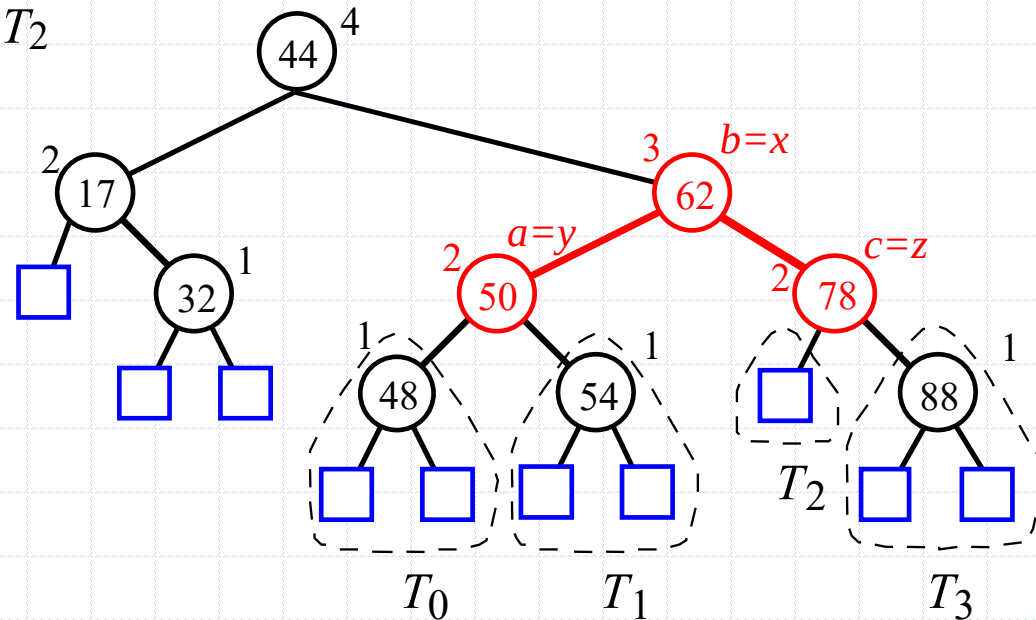
Restructuring (as Double Rotations)



Example: Insert 54, continued

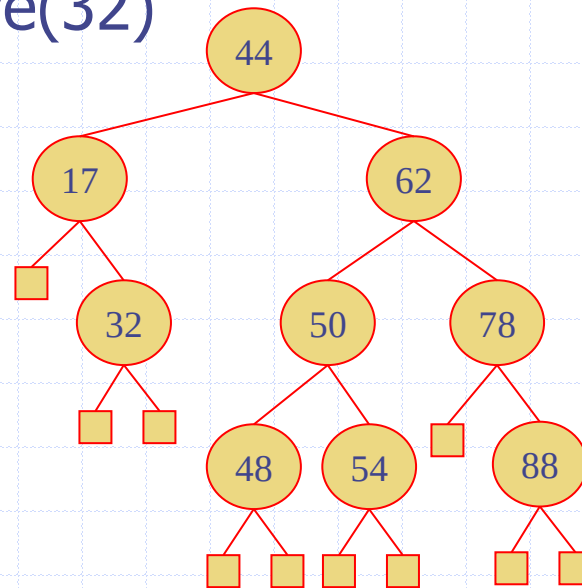


double rotation
...balanced



Removal

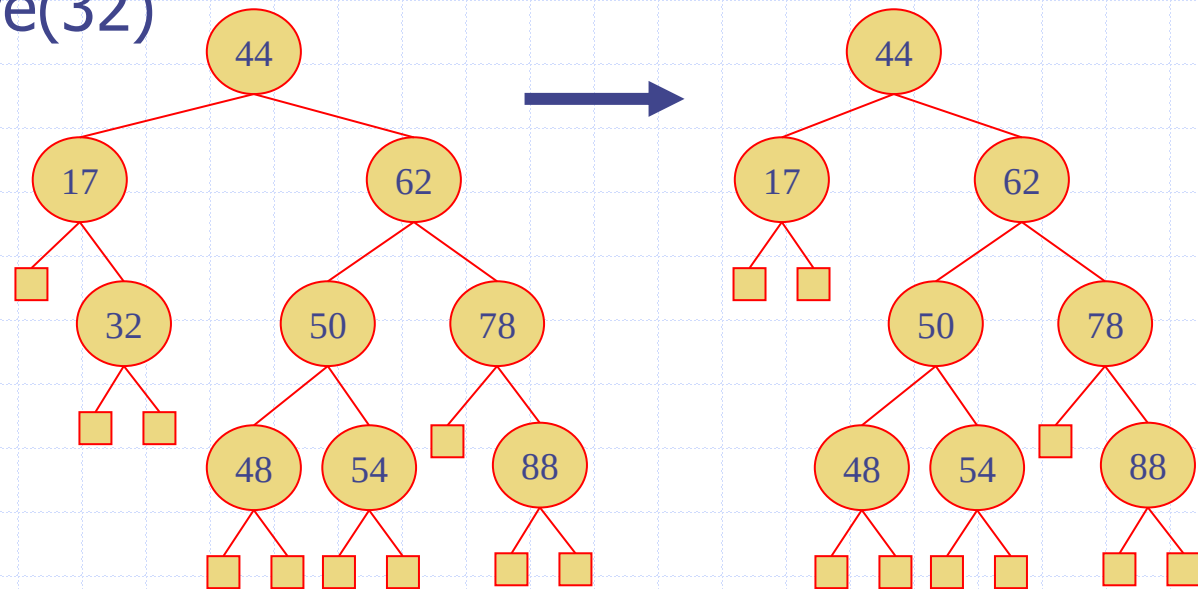
- ◆ Begins as in a binary search tree
- ◆ Removed node will become an empty external node
 - its parent, w , may cause an imbalance
- ◆ Example: remove(32)



before deletion of 32

Removal

- ◆ Begins as in a binary search tree
- ◆ Removed node will become an empty external node
 - its parent, w , may cause an imbalance
- ◆ Example: remove(32)

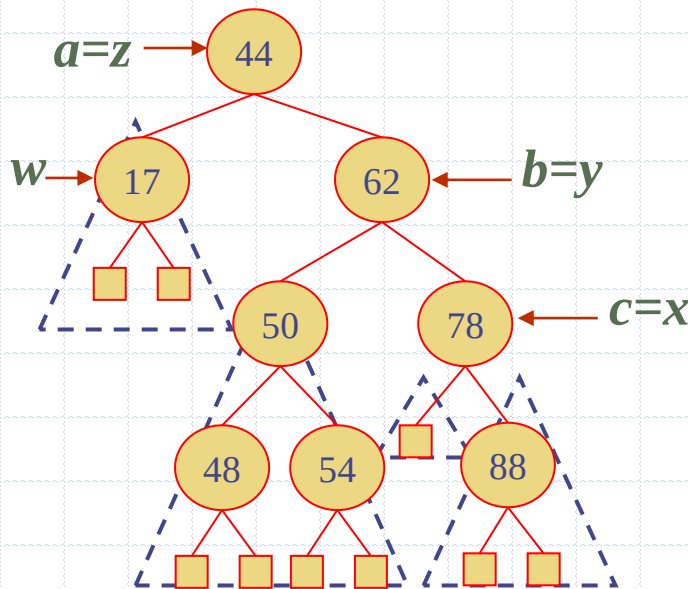


before deletion of 32

after deletion

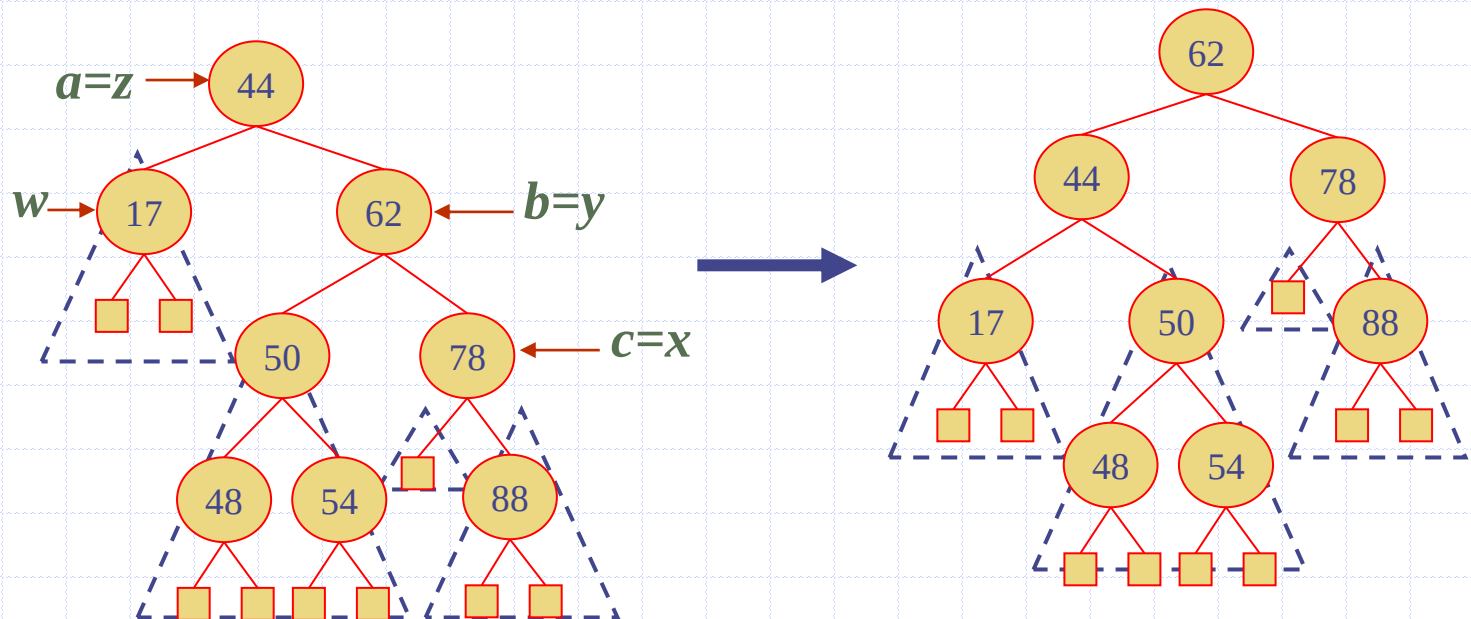
Rebalancing after a Removal

- ◆ z is the **first unbalanced** node encountered while travelling up the tree from w . Let y be the child of z with the larger height, and let x be the child of y with the larger height
- ◆ Perform **tri-node restructuring** to restore balance at z
- ◆ As this restructuring may upset the balance of another node higher in the tree, must continue checking for balance until the root of T is reached



Rebalancing after a Removal

- ◆ z is the **first unbalanced** node encountered while travelling up the tree from w . Let y be the child of z with the larger height, and let x be the child of y with the larger height
- ◆ Perform **tri-node restructuring** to restore balance at z
- ◆ As this restructuring may upset the balance of another node higher in the tree, must continue checking for balance until the root of T is reached



Tri-node Restructuring

Algorithm *restructure*(x)

Input Node x of a binary search tree T that has both a parent y and a grandparent z

Output Tree T after a tri-node restructuring (single or double rotation) involving nodes x , y and z .

1. Let (a, b, c) be a left-to-right (in-order) listing of the nodes x, y and z and let (T_0, T_1, T_2, T_3) be a left-to-right listing of the 4 subtrees rooted at (x, y, z)
2. Replace the subtree rooted at z with a new subtree rooted at b
3. Let a be the left child of b and let T_0 and T_1 be the left and right subtrees of a respectively
4. Let c be the right child of b and let T_2 and T_3 be the left and right subtrees of c respectively

Properties of Tri-node Restructure after Insertion

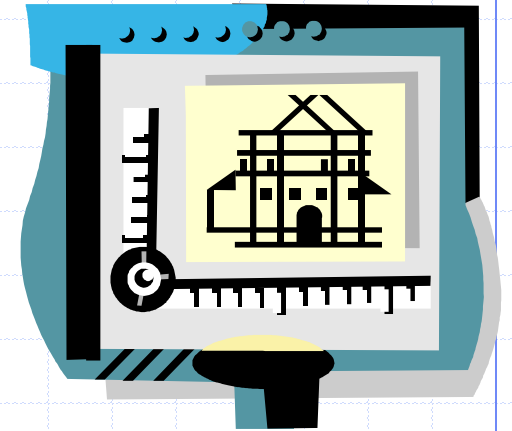
◆ Order preserving

- changes preserve the in-order traversal of all nodes

◆ Local changes

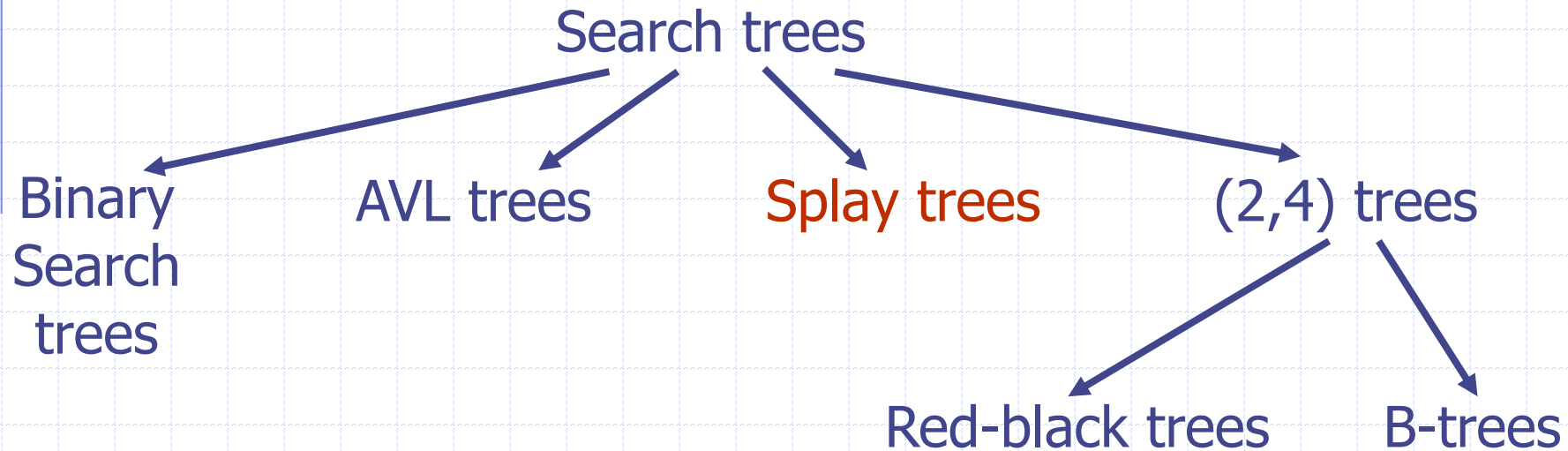
- changes are made **locally** when the grandparent is unbalanced
- only a **constant** number of nodes are altered
- **single restructure** restores the height-balance property **globally**

AVL Tree Performance



- ◆ Uses $O(n)$ space
- ◆ Single restructuring takes $O(1)$ time
 - using a linked-structure binary tree
- ◆ Searching takes $O(\log n)$ time
 - height of tree is $O(\log n)$, no restructures needed
- ◆ Insertion takes $O(\log n)$ time
 - initial find is $O(\log n)$
 - restructuring up the tree, maintaining heights is $O(\log n)$
- ◆ Removal takes $O(\log n)$ time
 - initial find is $O(\log n)$
 - restructuring up the tree, maintaining heights is $O(\log n)$

Week 7 – Search Trees



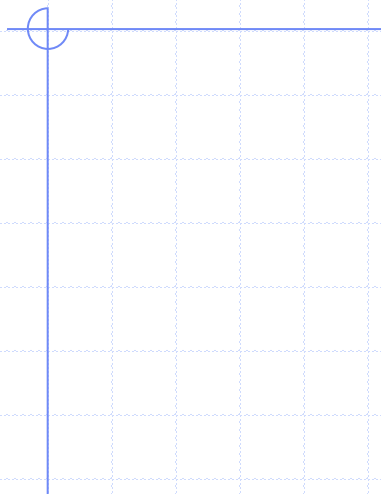
Splay Trees

- ◆ Splay tree is a binary search tree
 - nodes are **splayed** after being accessed
 - ◆ deepest internal node accessed
- ◆ Do not enforce a logarithmic upper bound on height of the tree
 - **worst-case** time complexity of search, delete and insert is $O(n)$

Splaying

- ◆ Moves a node to the root using rotations
- ◆ Performed after all operations
 - searches
 - insertions
 - Removals

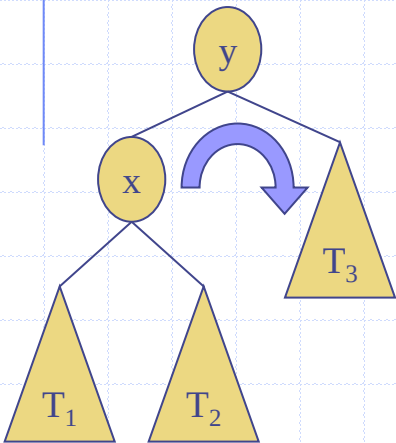
Rotations



Rotations

- right rotation

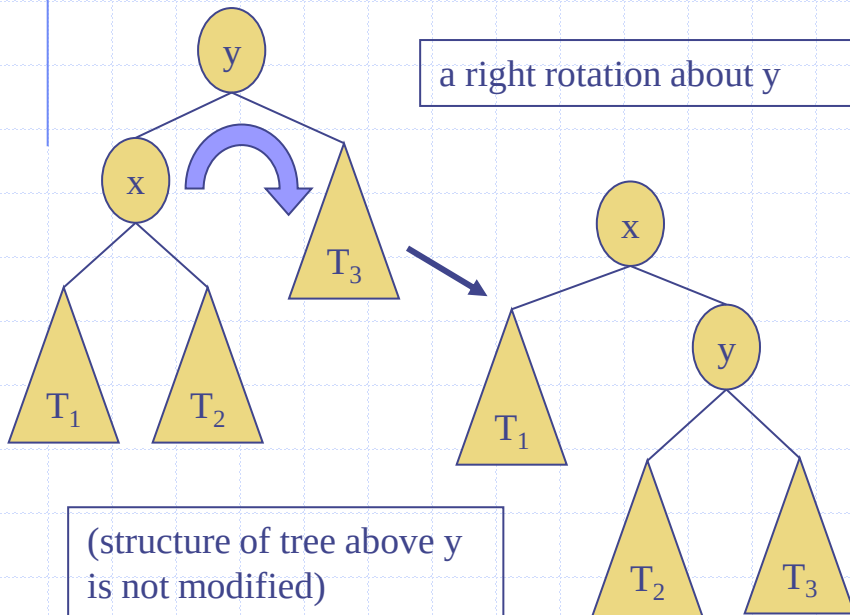
- makes the left child x of a node y into y 's parent; y becomes the right child of x



Rotations

■ right rotation

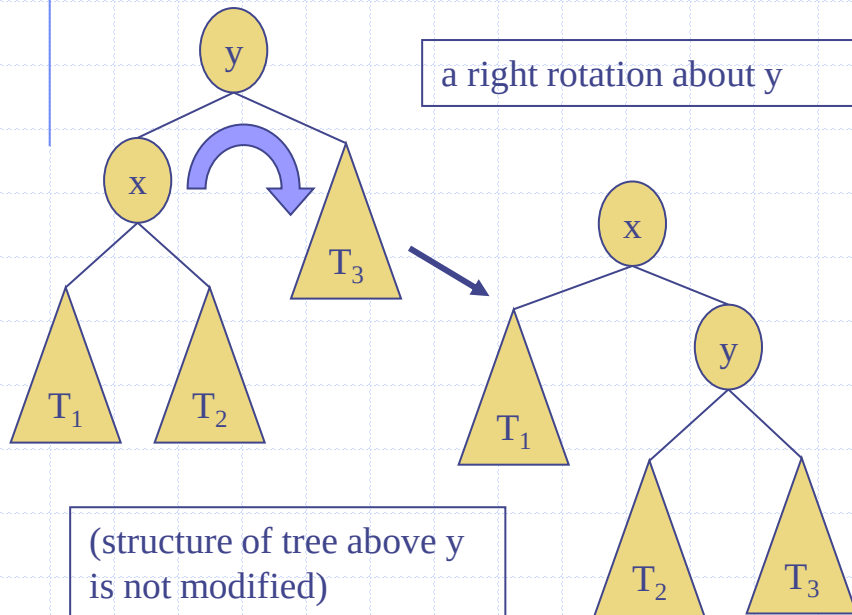
- makes the left child x of a node y into y 's parent; y becomes the right child of x



Rotations

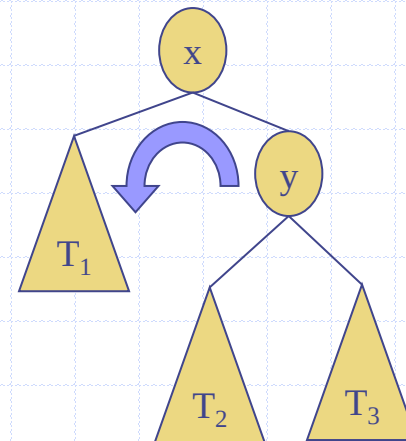
■ right rotation

- makes the left child x of a node y into y 's parent; y becomes the right child of x



■ left rotation

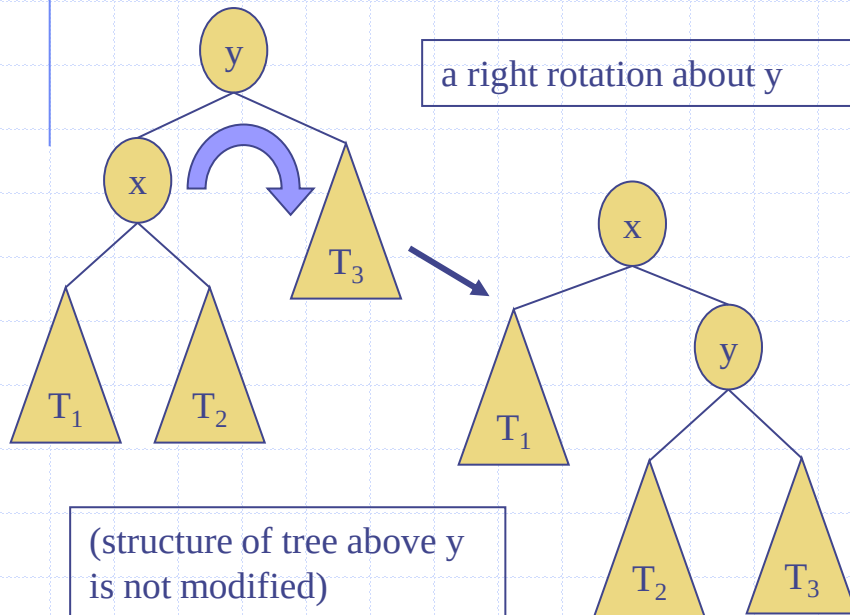
- makes the right child y of a node x into x 's parent; x becomes the left child of y



Rotations

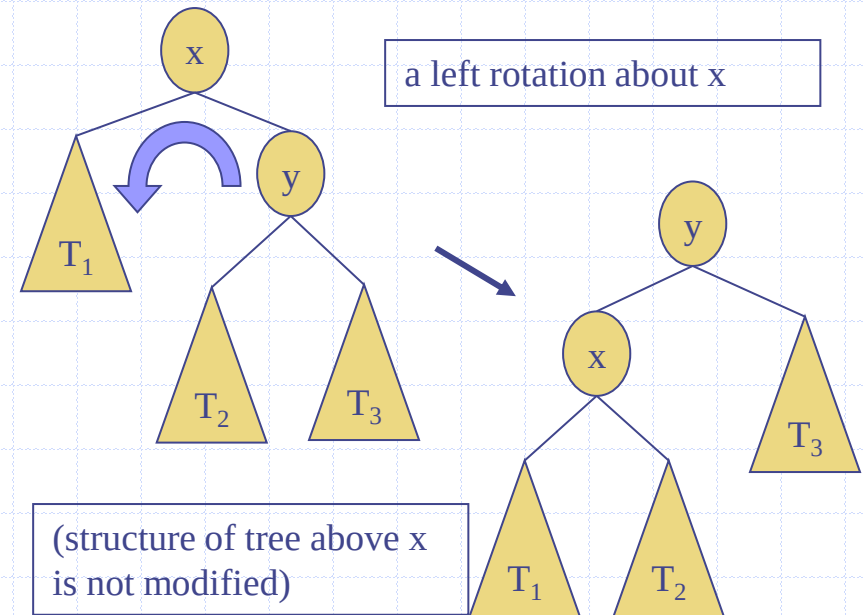
■ right rotation

- makes the left child x of a node y into y 's parent; y becomes the right child of x



■ left rotation

- makes the right child y of a node x into x 's parent; x becomes the left child of y



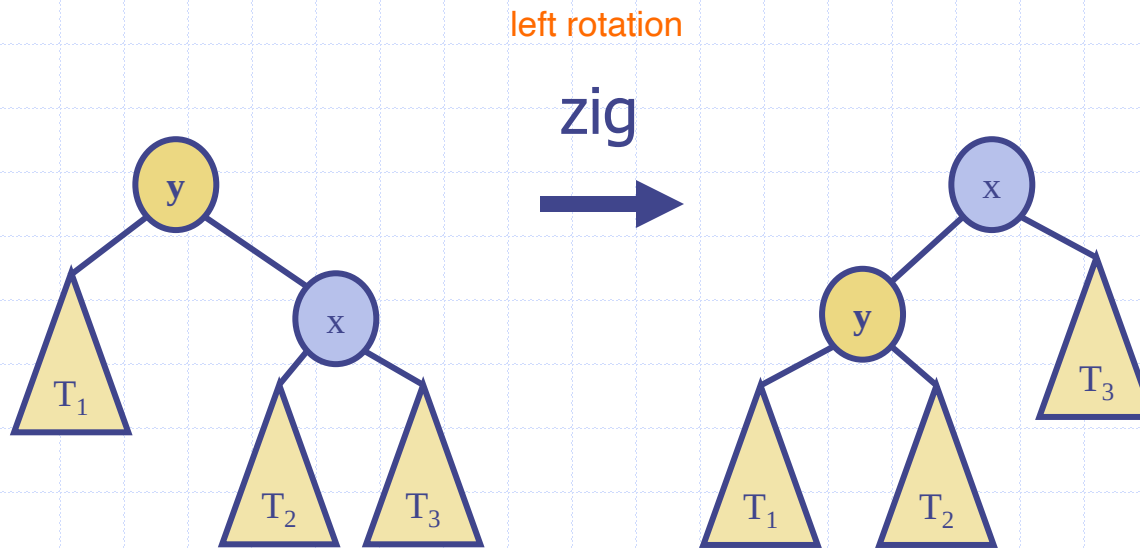
Splaying

◆ Which nodes are splayed after each operation?

method	splay node
Search for k	If key found, use that node If key not found, use parent of ending external node
Insert (k,v)	Use new node containing the entry inserted
Remove item with key k	Use parent of the internal node that was actually removed from the tree (the parent of the node that the removed item was swapped with)

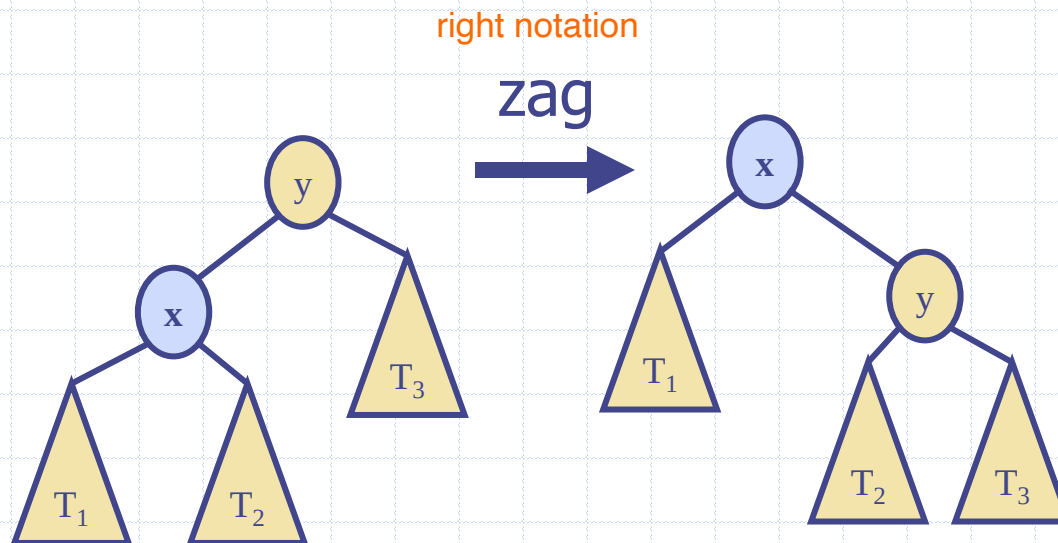
How do we Splay?

◆ Is the node a child of the root?



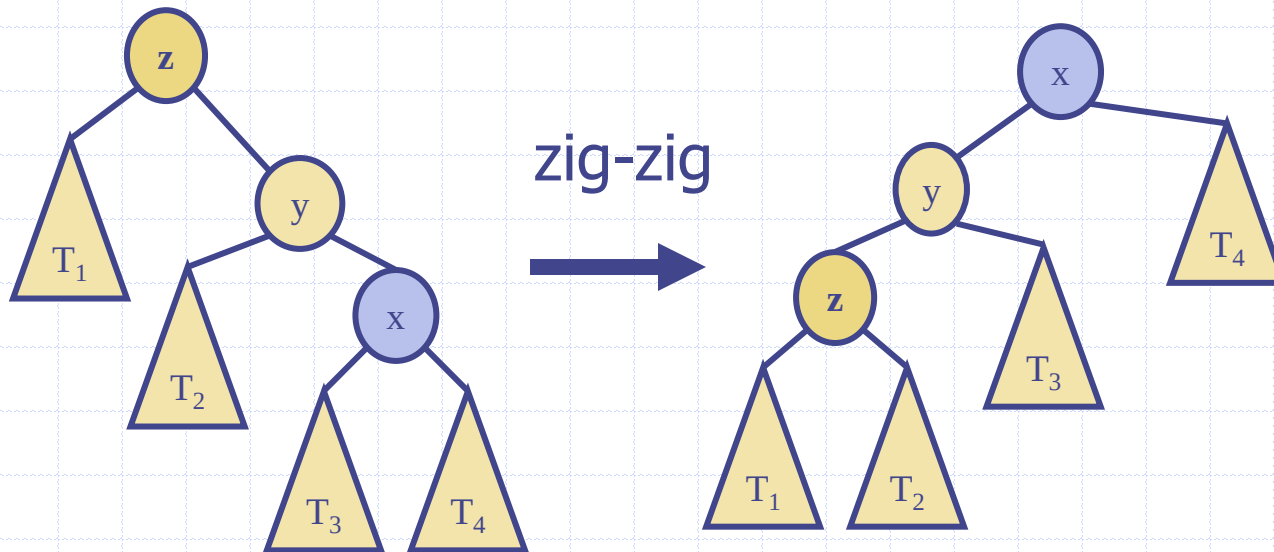
How do we Splay?

◆ Is the node a child of the root?



How do we Splay? move the node to the top (root)

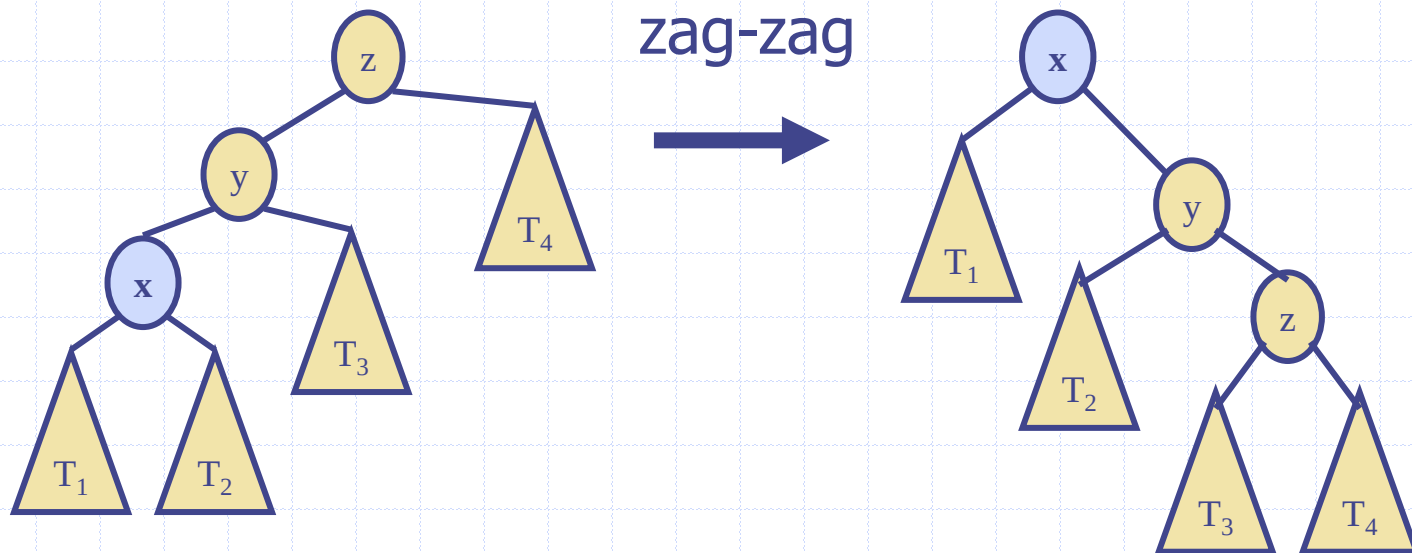
◆ What if the node isn't a child of the root?



And keep splaying until you get it to the root...

How do we Splay?

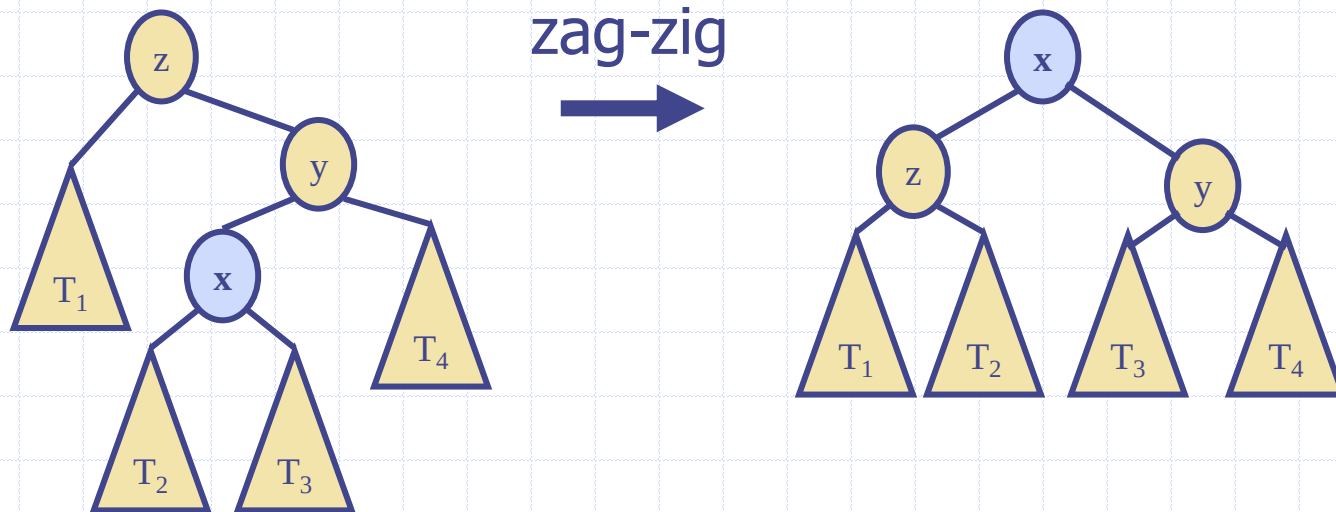
◆ What if the node isn't a child of the root?



And keep splaying until you get it to the root...

How do we Splay?

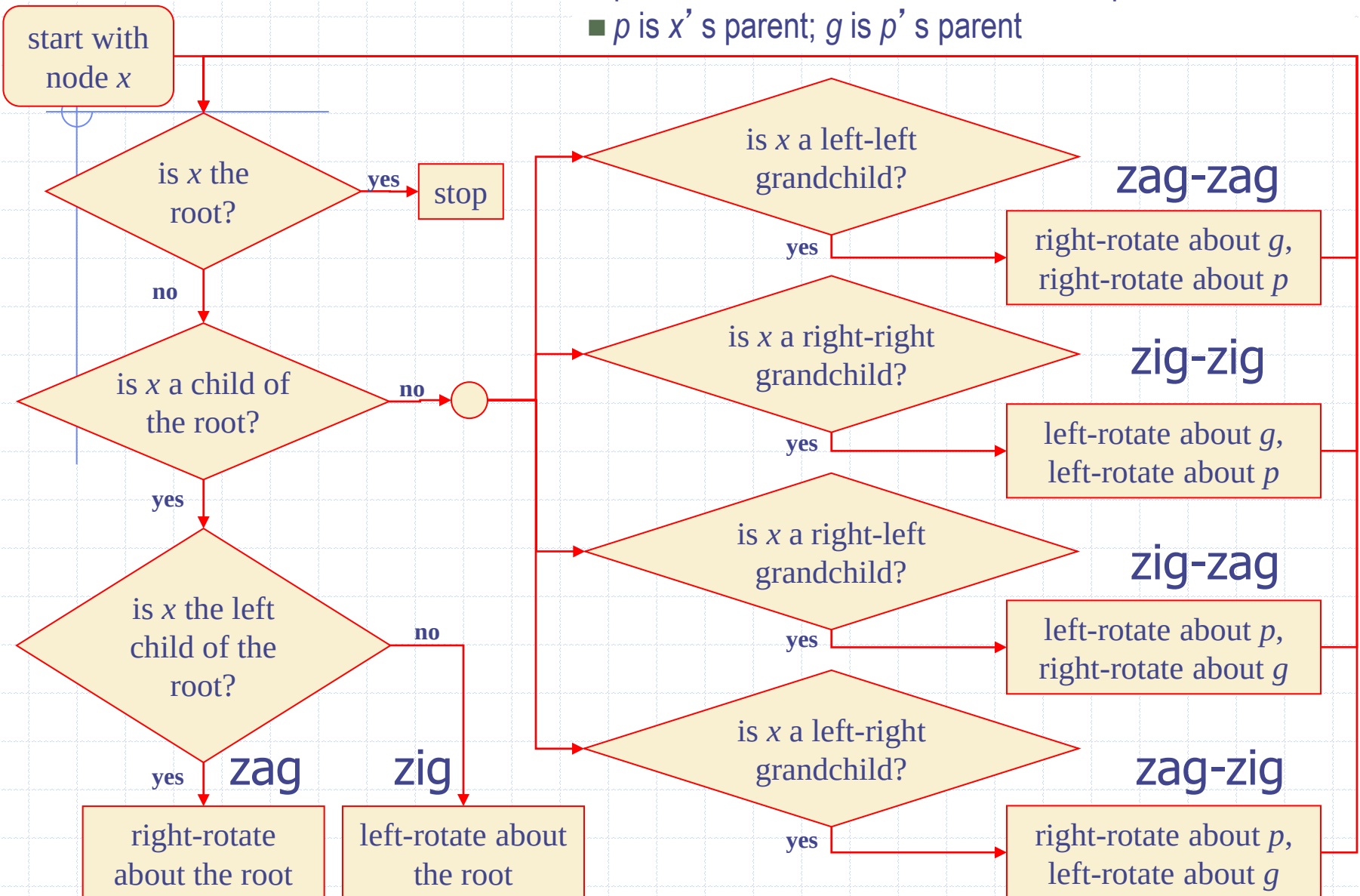
◆ What if the node isn't a child of the root?



And keep splaying until you get it to the root...
Don't forget zig-zag!

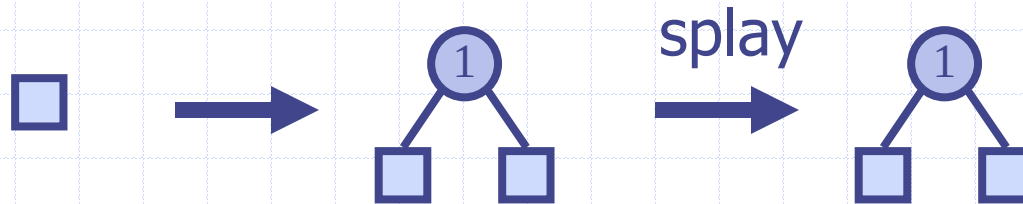
Splaying:

- “ x is a left-left grandchild” means x is a left child of its parent, which is itself a left child of its parent
- p is x 's parent; g is p 's parent



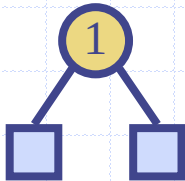
Insertion with Splaying: example

Insert 1



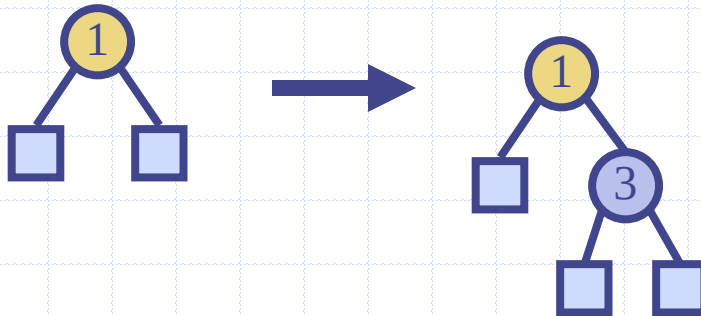
Insertion with Splaying: example

Insert 3



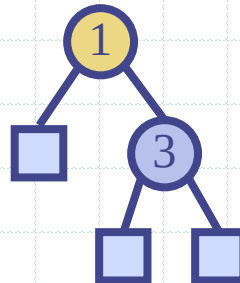
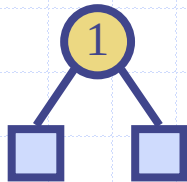
Insertion with Splaying: example

Insert 3

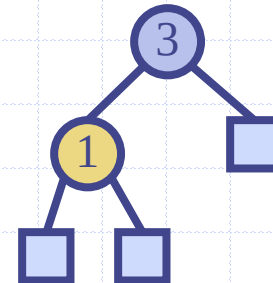


Insertion with Splaying: example

Insert 3

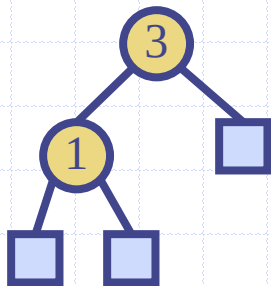


splay



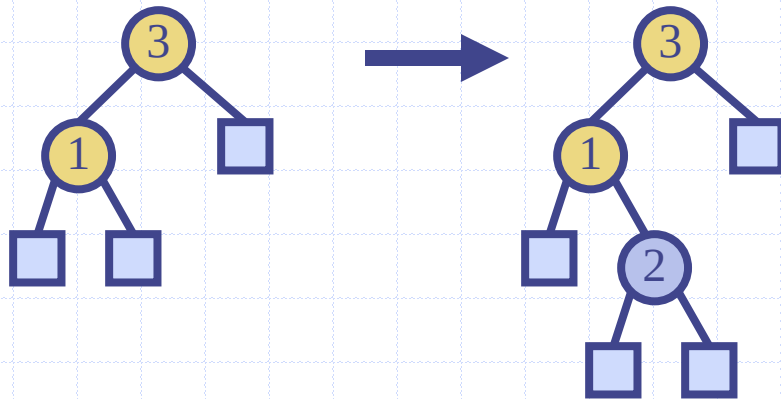
Insertion with Splaying: example

Insert 2



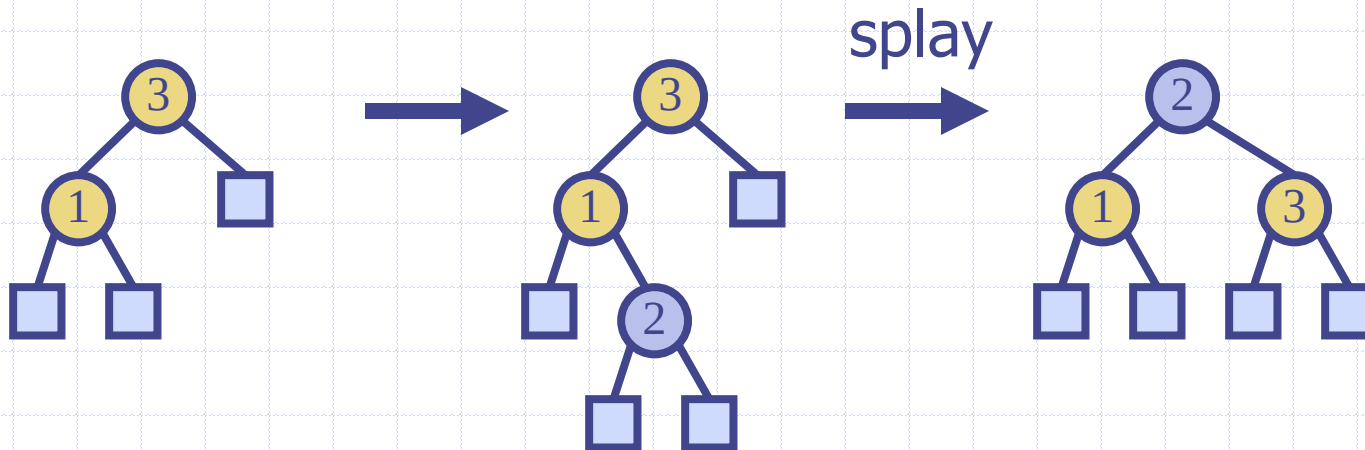
Insertion with Splaying: example

Insert 2



Insertion with Splaying: example

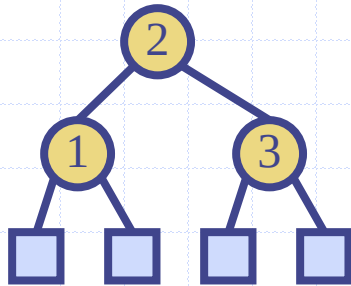
Insert 2



Splaying might decrease the height of the tree!

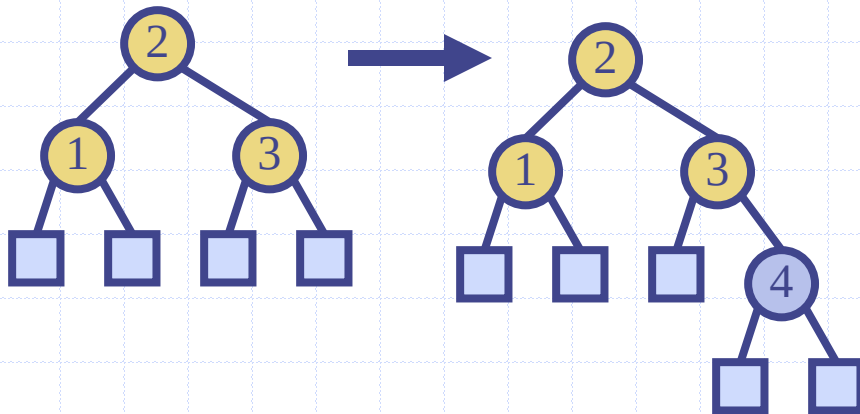
Insertion with Splaying: example

Insert 4



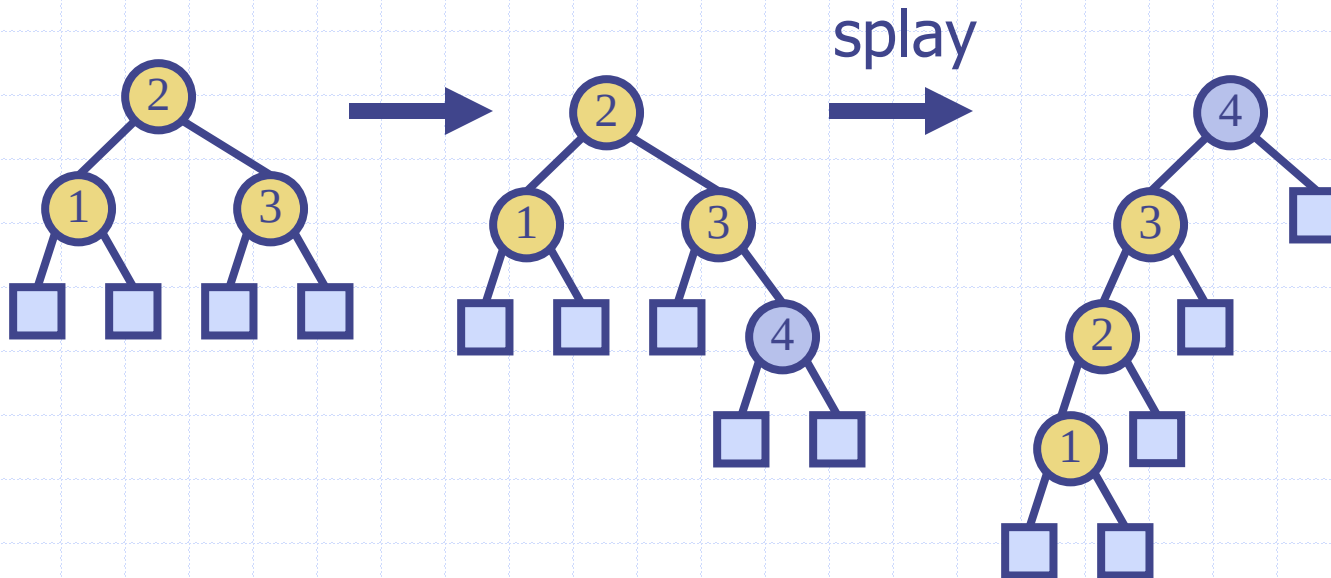
Insertion with Splaying: example

Insert 4



Insertion with Splaying: example

Insert 4



Splaying might increase the height of the tree!

Splaying Costs

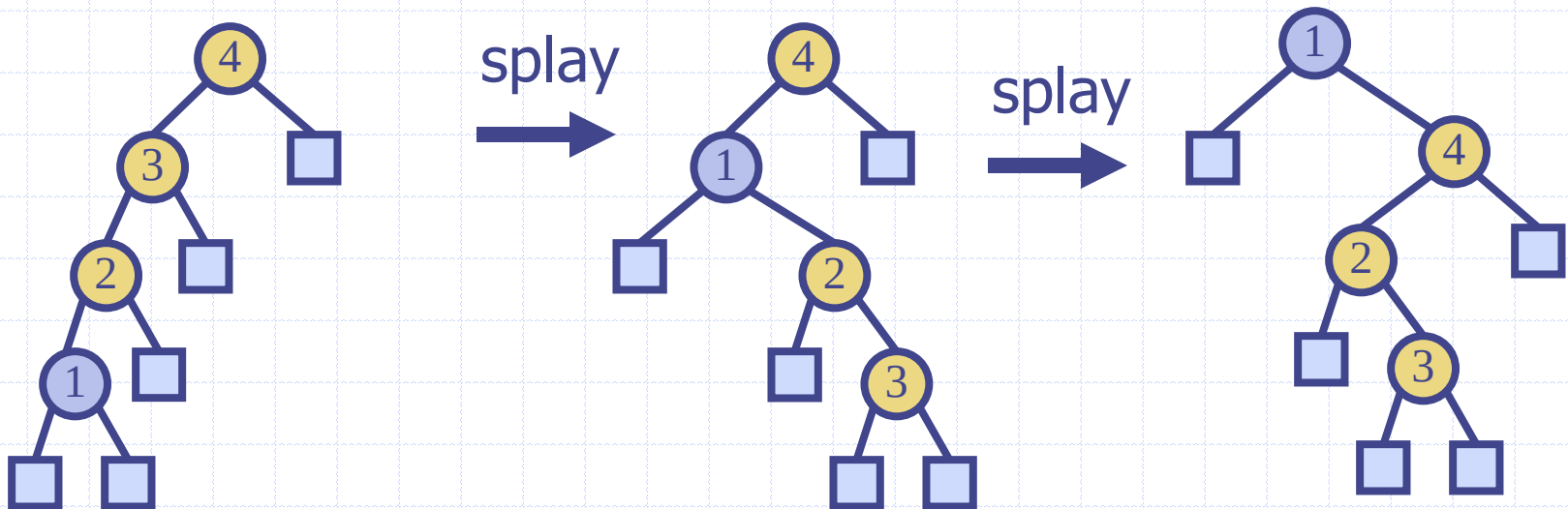
- ◆ Splaying costs $O(h)$

- where h is height of the tree
 - ◆ $O(n)$ worst-case

- ◆ $O(h)$ rotations, each of which is $O(1)$

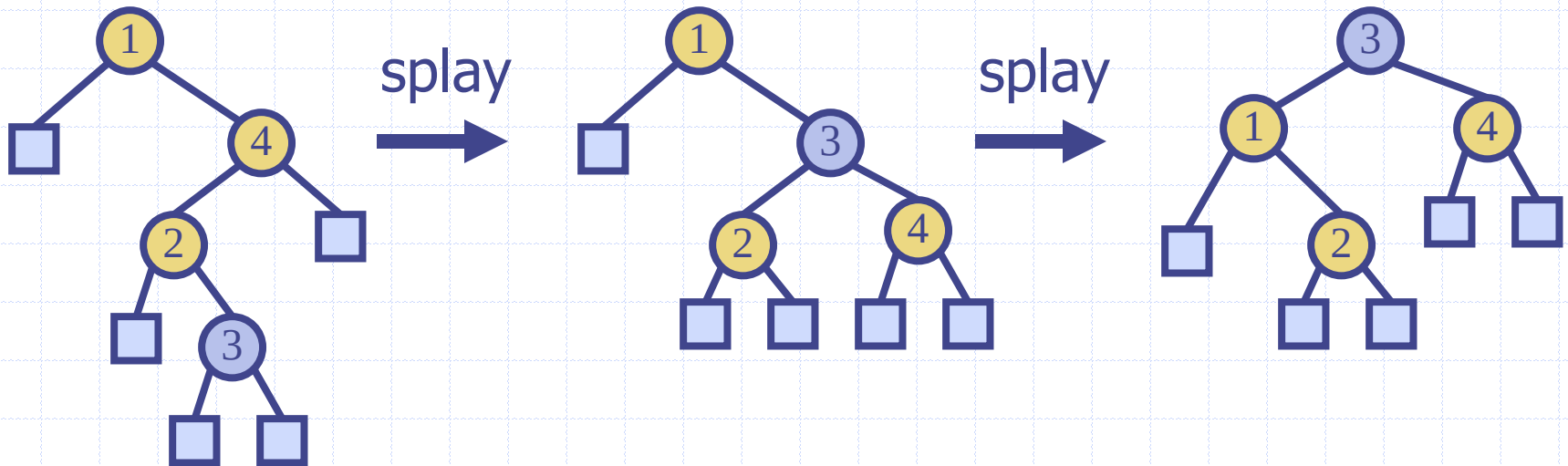
Splaying Costs

- ◆ But we can't do an $O(n)$ operation every time!
 - Example: search 1



Splaying Costs

- ◆ But we can't do an $O(n)$ operation every time!
 - Example: search 3



Splaying Costs

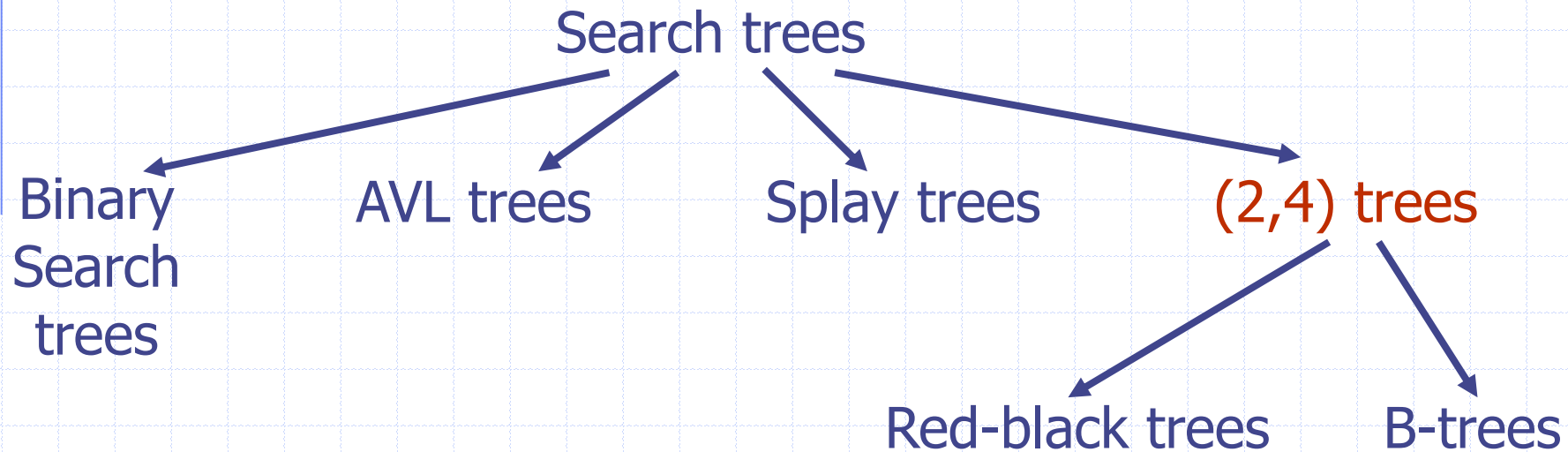
◆ Amortised cost of splaying any node is $O(\log n)$

- amortised cost of a splay operation is the time $T(n)$ needed to perform a series of n splay operations, divided by n
 - ◆ i.e. $T(n) \div n$

Splaying Costs

- ◆ Splay trees can adapt to perform searches on frequently-requested items much faster than $O(\log n)$ in some cases

Week 7 – Search Trees



Key Ideas

- ◆ There are probably ~ 10 - 20 balanced tree types in the literature
 - AVL, Red-Black, Scapegoat, AA, Treap, Tango, B, 2-3, 2-3-4 (aka 2,4), ...
 - Each has its own unique trade-offs
 - Most commonly seen: AVL, Splay, **Red-Black, B-Trees**

Red-Black Trees

- ◆ A very specific type of balanced tree
 - A specialisation of (2, 4)-trees
- ◆ Very similar in spirit to the AVL tree, but less stringent balance requirements
 - Implication: Faster inserts and deletions, but slower finds.

Comparison to AVL-trees

- ◆ AVL trees maintain tighter restrictions on balance than red-black trees
- ◆ AVL trees have **better** search performance
 - but more **expensive** insert/delete
 - Vice versa
- ◆ Red Black trees are used in Java's TreeMap and the C++ STL Ordered Map

B-Trees (B+ and other variants)

- ◆ Typically used in database applications
- ◆ Very high number of children
 - Allows for shallow trees
 - In databases, this means fewer disk seeks

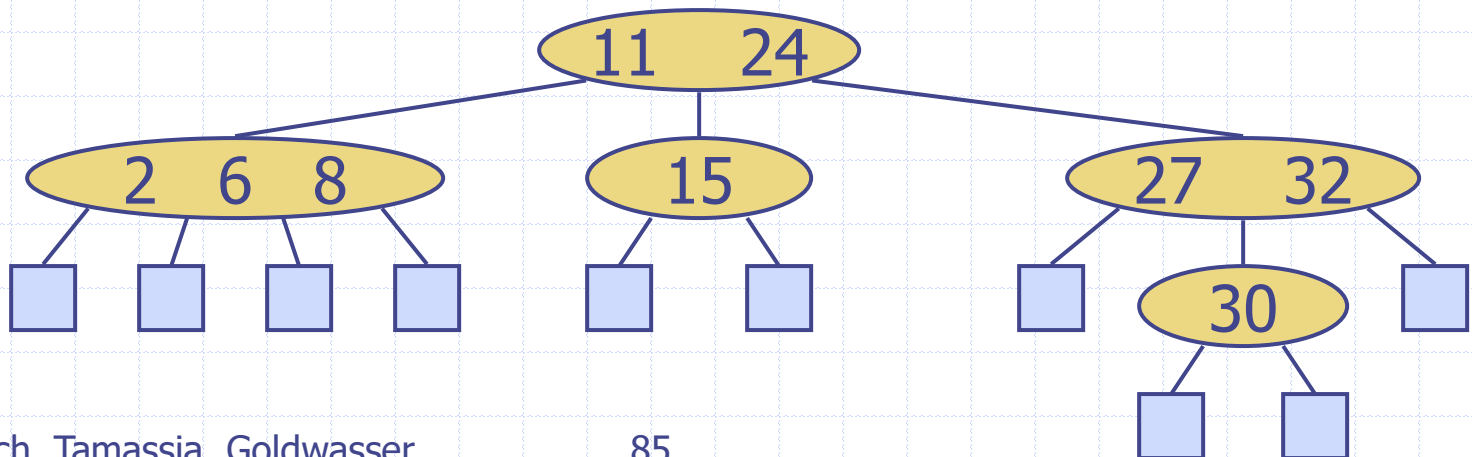
Further Reading

- ◆ Data Structures and Algorithms in Python
 - Chapter 11.1—11.6
- ◆ Introduction to Algorithms
 - Chapter 13
 - Chapter 18

Multi-Way Search Tree

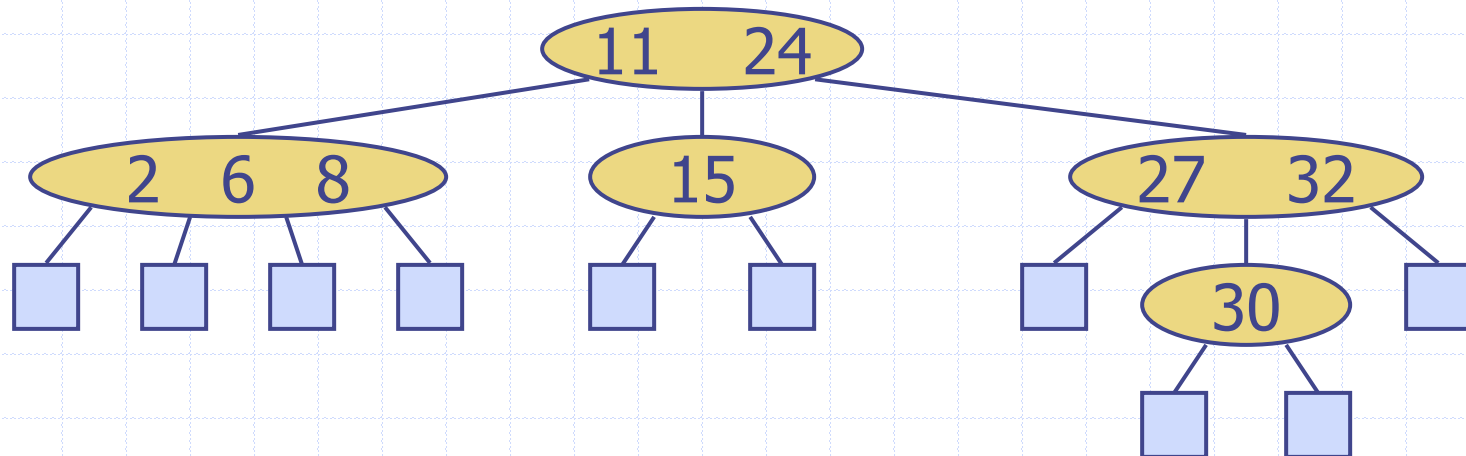
◆ Ordered Tree

- Each internal node has at least two children and stores $d - 1$ key-element items (k_i, o_i) , where d is the number of children
- For a node with children $v_1 v_2 \dots v_d$ storing keys $k_1 k_2 \dots k_{d-1}$
 - ◆ keys in the subtree of v_1 are less than k_1
 - ◆ keys in the subtree of v_i are between k_{i-1} and k_i ($i = 2, \dots, d - 1$)
 - ◆ keys in the subtree of v_d are greater than k_{d-1}
- Leaves store no items and serve as placeholders



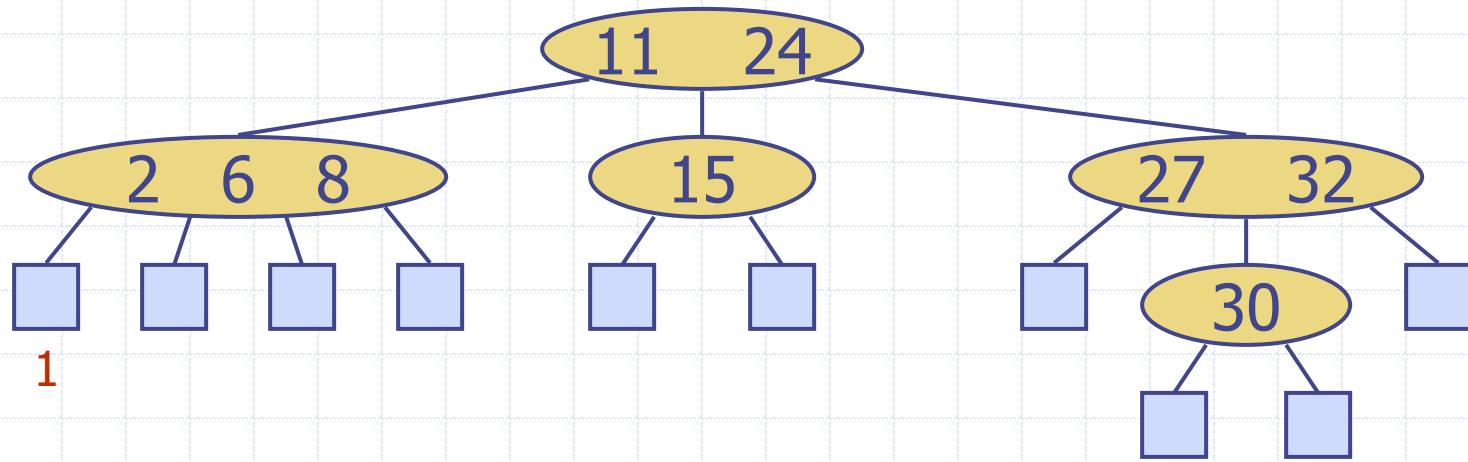
Multi-Way In-Order Traversal

- ◆ Extend the notion of in-order traversal from binary trees to multi-way search trees
- ◆ Namely, we visit item (k_i, o_i) of node v between the recursive traversals of the subtrees of v rooted at children v_i and v_{i+1}
- ◆ An in-order traversal of a multi-way search tree visits the keys in increasing order



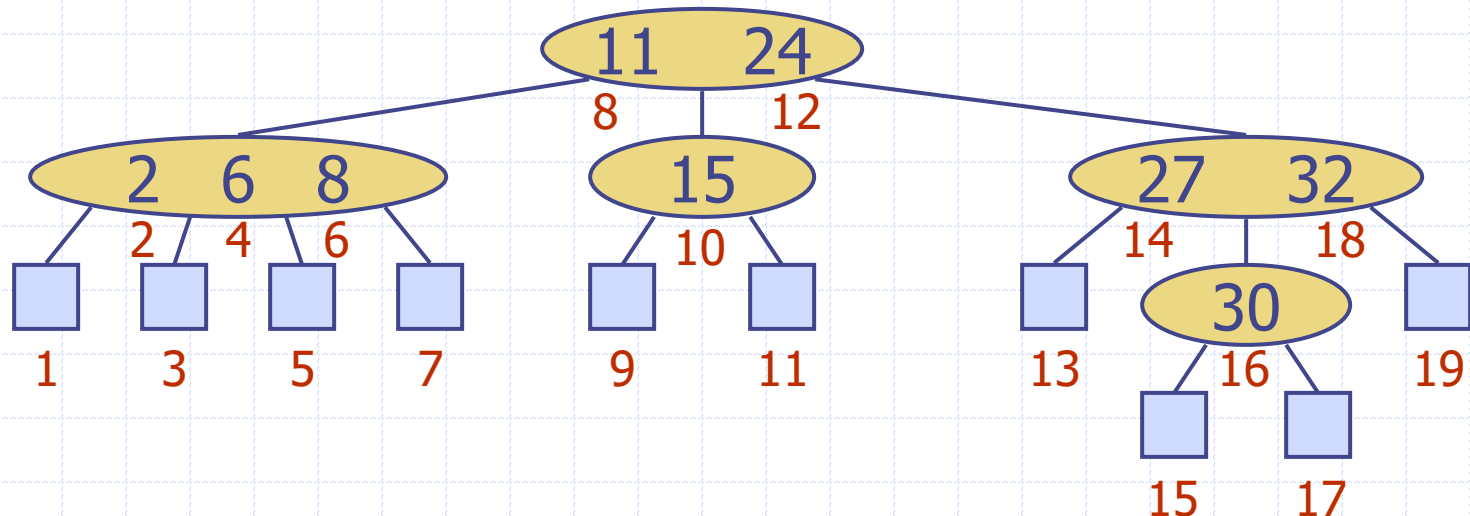
Multi-Way In-Order Traversal

- ◆ Extend the notion of in-order traversal from binary trees to multi-way search trees
- ◆ Namely, we visit item (k_i, o_i) of node v between the recursive traversals of the subtrees of v rooted at children v_i and v_{i+1}
- ◆ An in-order traversal of a multi-way search tree visits the keys in increasing order



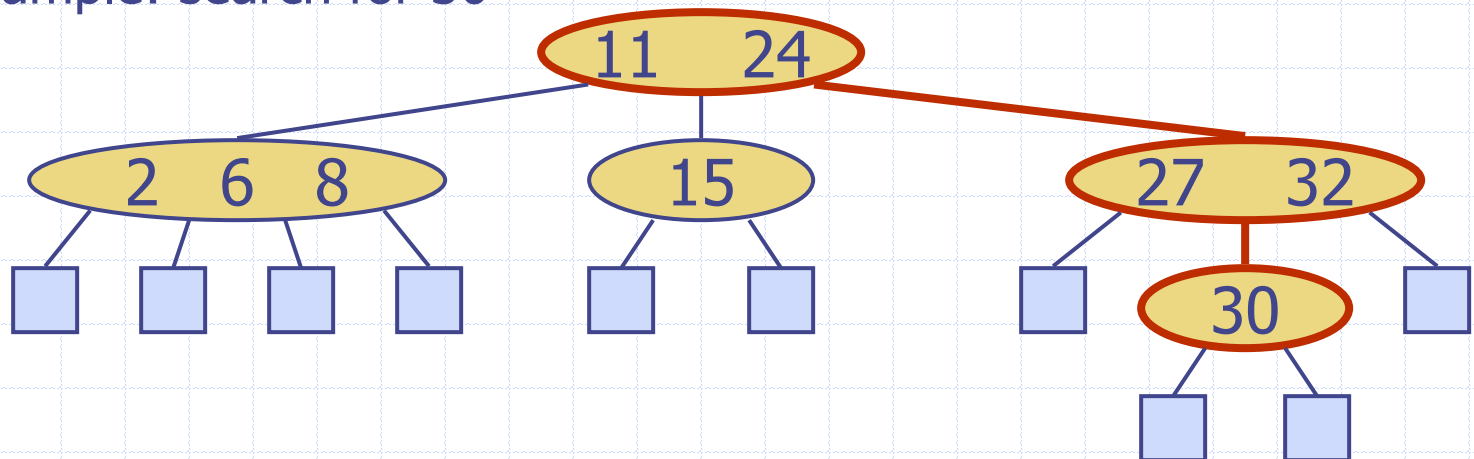
Multi-Way In-Order Traversal

- ◆ Extend the notion of in-order traversal from binary trees to multi-way search trees
- ◆ Namely, we visit item (k_i, o_i) of node v between the recursive traversals of the subtrees of v rooted at children v_i and v_{i+1}
- ◆ An in-order traversal of a multi-way search tree visits the keys in increasing order



Multi-Way Searching

- ◆ Similar to search in a binary search tree
- ◆ At each internal node with children $v_1 v_2 \dots v_d$ and keys $k_1 k_2 \dots k_{d-1}$
 - $k = k_i$ ($i = 1, \dots, d - 1$): search terminates successfully
 - $k < k_1$: continue the search in child v_1
 - $k_{i-1} < k < k_i$ ($i = 2, \dots, d - 1$): continue the search in child v_i
 - $k > k_{d-1}$: continue the search in child v_d
- ◆ Reaching an external node terminates the search unsuccessfully
- ◆ Example: search for 30

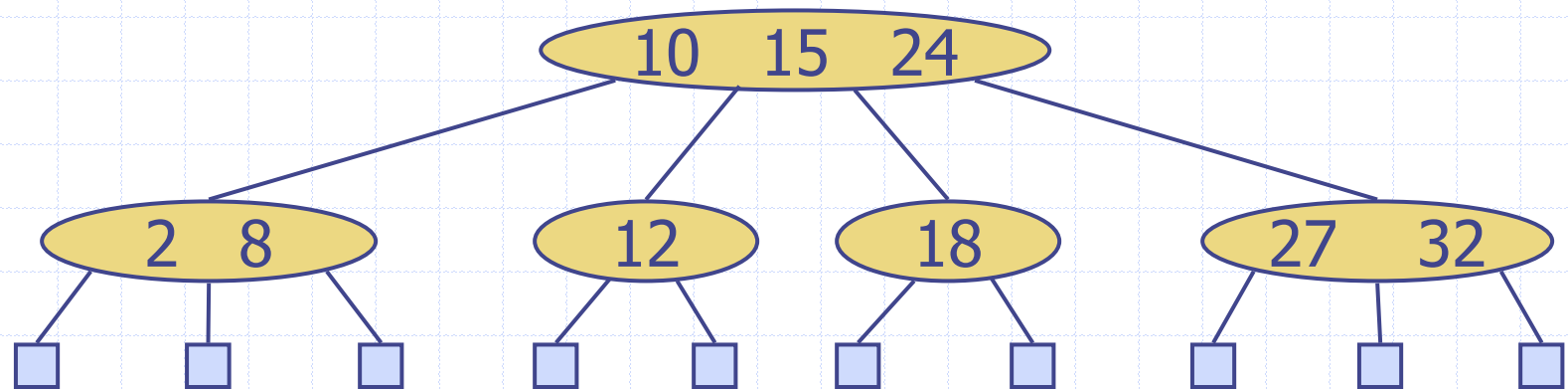


Bonus Material

The following material is considered “bonus” material and is for your self study only. It will not be assessed.

(2,4) Trees

- ◆ Multi-way search tree with the following properties
 - **Node-Size Property**: every internal node has at most **four** children
 - **Depth Property**: all the external nodes have the **same** depth
 - also called 2-4 tree or 2-3-4 tree
- ◆ Depending on the number of children, an internal node of a (2,4) tree is called a 2-node, 3-node or 4-node



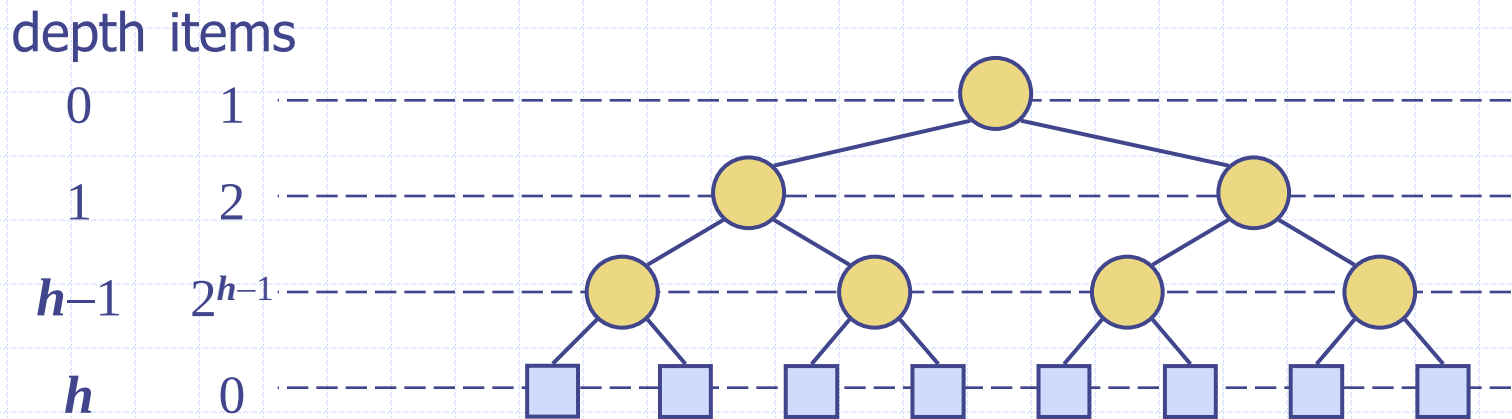
Height of a (2,4) Tree

- ◆ Searching in a (2,4) tree with height h takes $O(h)$ time
- ◆ What is the maximum height of a (2,4) tree with n nodes?
- ◆ Equivalently, what is the minimum number of nodes n in a (2,4) tree with height h ?

Height of a (2,4) Tree

◆ Since there are at least 2^i items at depth i and no items at depth h , we have

■ $n \geq 1 + 2 + 4 + \dots + 2^{h-1} = 2^h - 1$

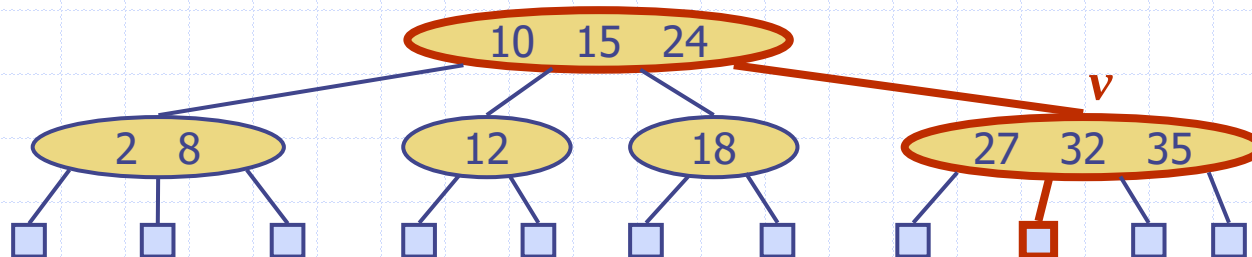


◆ Thus $h \leq \log(n+1)$

◆ Searching takes $O(\log n)$ time

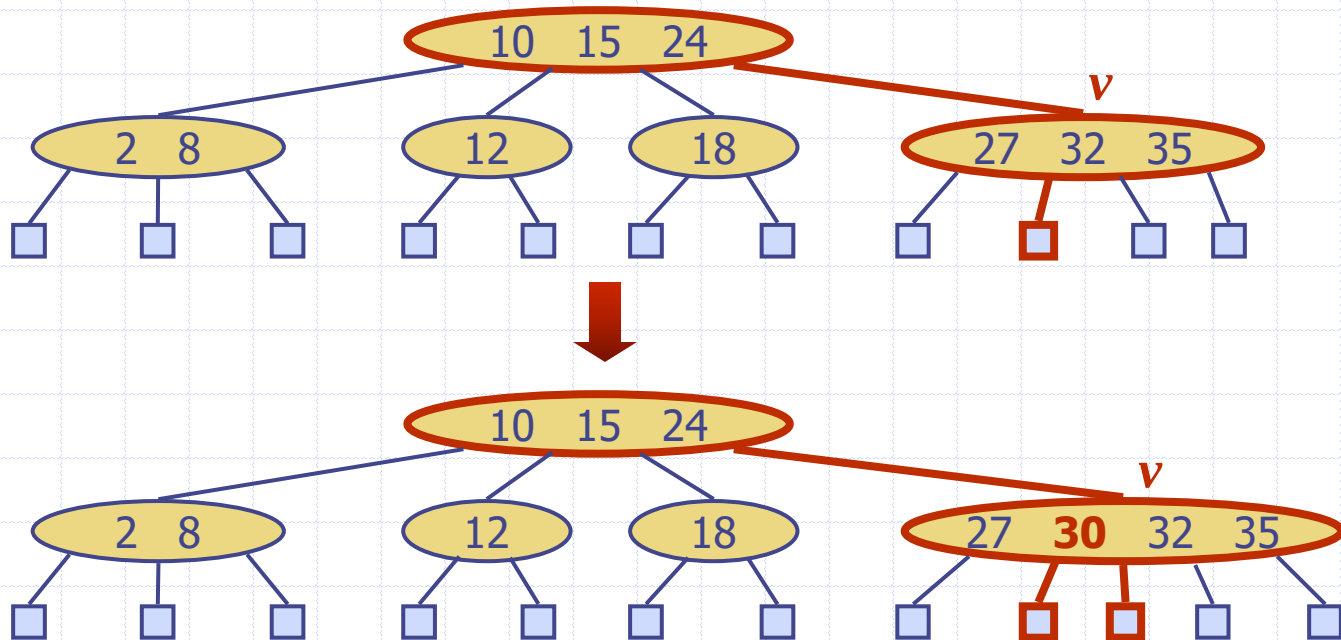
Insertion

- ◆ Insert a new item (k, o) at the parent v of the leaf reached by searching for k
 - preserve depth property, but
 - may cause an **overflow** (i.e. node v may become a 5-node)
- ◆ Example: inserting key 30 causes an overflow



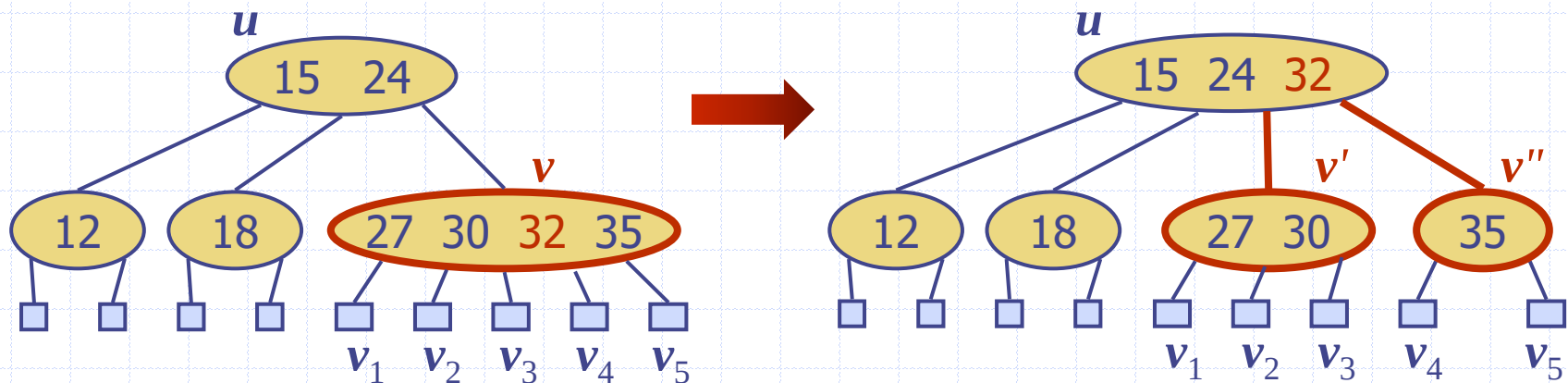
Insertion

- ◆ Insert a new item (k, o) at the parent v of the leaf reached by searching for k
 - preserve depth property, but
 - may cause an **overflow** (i.e. node v may become a 5-node)
- ◆ Example: inserting key 30 causes an overflow



Overflow and Split

- ◆ Handle an **overflow** at a 5-node v with a **split** operation
 - let $v_1 \dots v_5$ be the children of v and $k_1 \dots k_4$ be the keys of v
 - node v is replaced nodes v' and v''
 - ◆ v' is a 3-node with keys $k_1 k_2$ and children $v_1 v_2 v_3$
 - ◆ v'' is a 2-node with key k_4 and children $v_4 v_5$
 - k_3 is inserted into the parent u of v (a new root may be created)
- ◆ Overflow may propagate to the parent node u



Analysis of Insertion

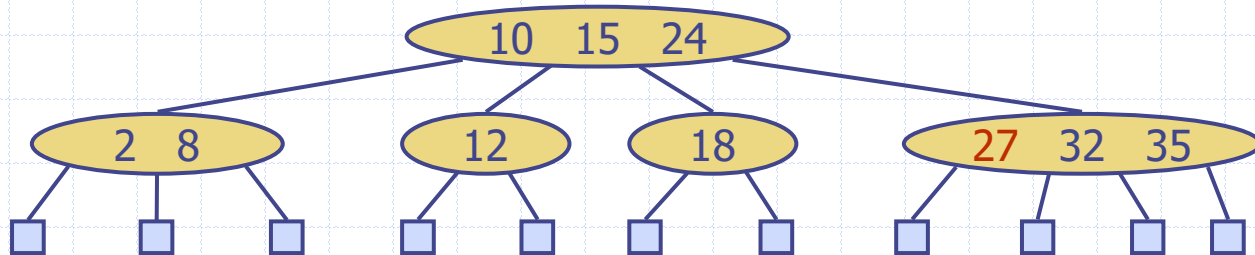
Algorithm *put*(k, o):

1. Search for key k to locate the insertion node v
2. Add new entry (k, o) at node v
3. **while** *overflow*(v)
 if *isRoot*(v)
 create a new empty root
 above v
 $v \leftarrow \textit{split}(v)$

- ◆ Let T be a (2,4) tree with n items
 - Tree T has $O(\log n)$ height
 - Step 1 takes $O(\log n)$ time because we visit $O(\log n)$ nodes
 - Step 2 takes $O(1)$ time
 - Step 3 takes $O(\log n)$ time because each split takes $O(1)$ time and we perform $O(\log n)$ splits
- ◆ Thus, an insertion in a (2,4) tree takes $O(\log n)$ time

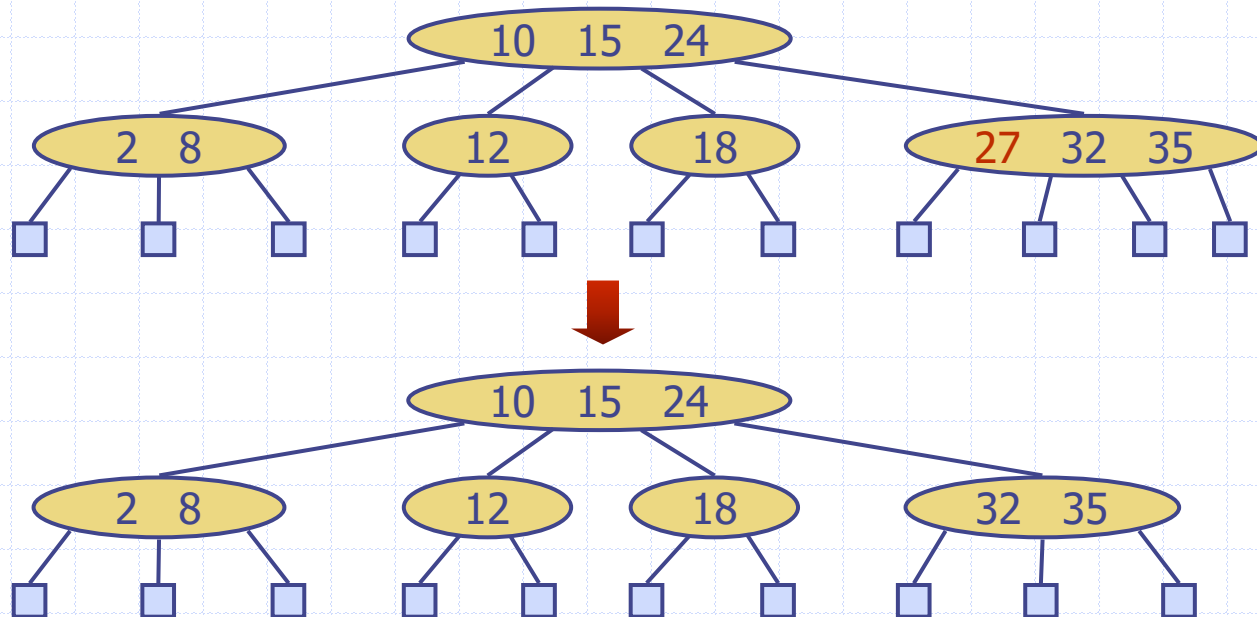
Deletion

- ◆ If the item to be deleted is at a node with leaf children, just delete it.
- ◆ Example: delete key 27



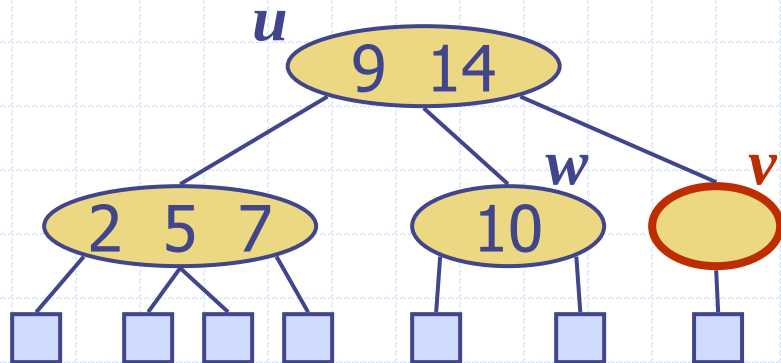
Deletion

- ◆ If the item to be deleted is at a node with leaf children, just delete it.
- ◆ Example: delete key 27



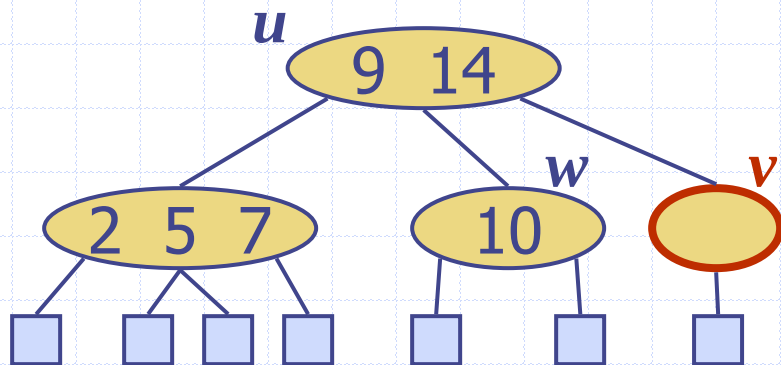
Underflow and Fusion

- Deleting an entry from a node v may cause an **underflow**, where node v becomes a 1-node with one child and no keys



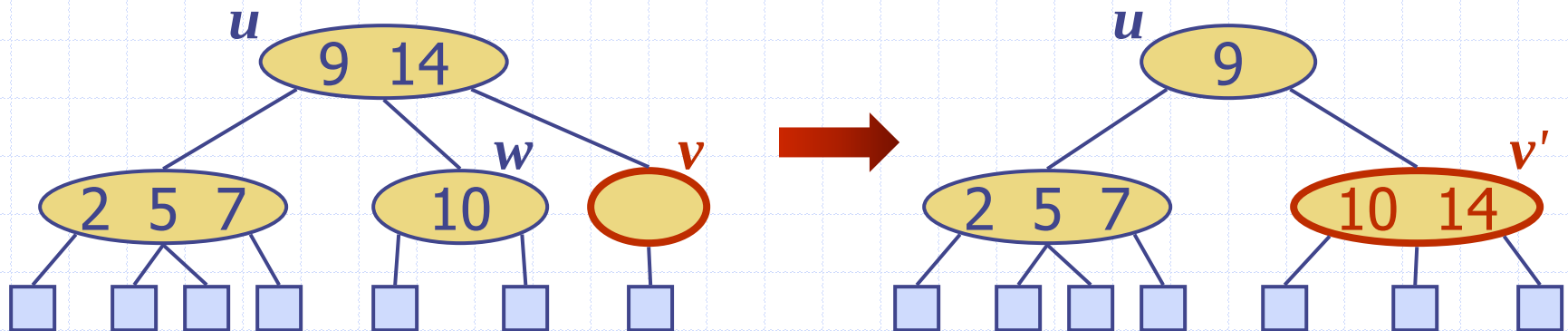
Underflow and Fusion

- ◆ Deleting an entry from a node v may cause an **underflow**, where node v becomes a 1-node with one child and no keys
- ◆ To handle an underflow at node v with parent u , consider two cases
- ◆ **Case 1:** adjacent siblings of v are 2-nodes
 - **Fusion operation:** merge v with an adjacent sibling w and move an entry from u to the merged node v'



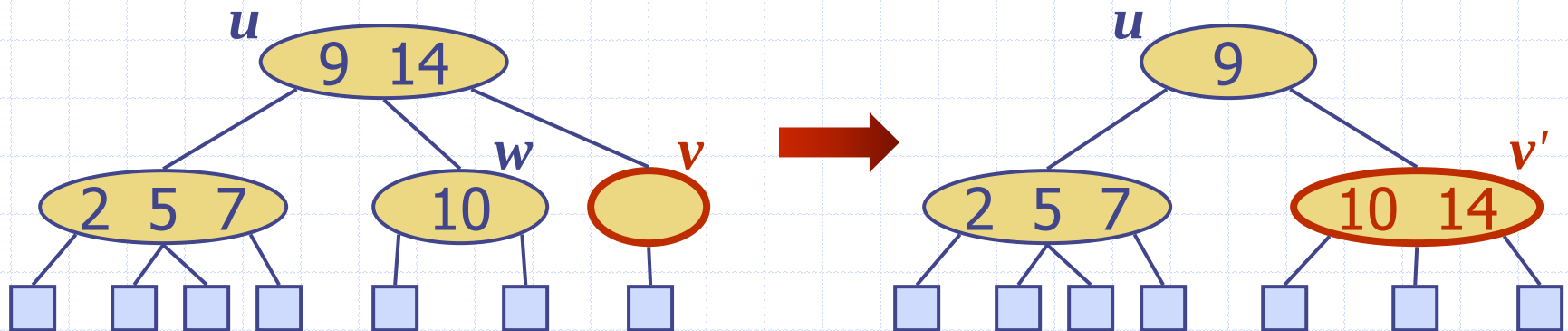
Underflow and Fusion

- ◆ Deleting an entry from a node v may cause an **underflow**, where node v becomes a 1-node with one child and no keys
- ◆ To handle an underflow at node v with parent u , consider two cases
- ◆ **Case 1:** adjacent siblings of v are 2-nodes
 - **Fusion operation:** merge v with an adjacent sibling w and move an entry from u to the merged node v'



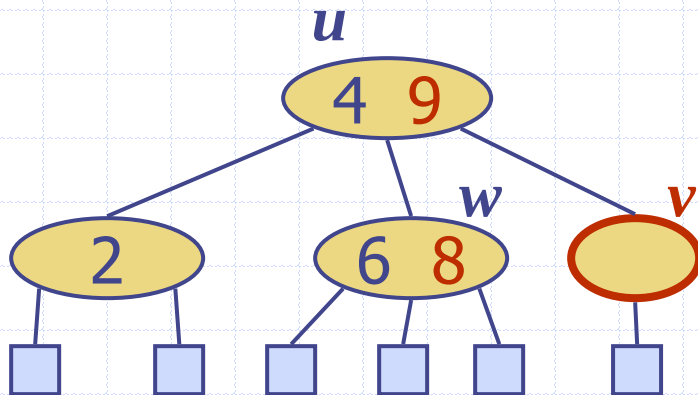
Underflow and Fusion

- ◆ Deleting an entry from a node v may cause an **underflow**, where node v becomes a 1-node with one child and no keys
- ◆ To handle an underflow at node v with parent u , consider two cases
- ◆ **Case 1:** adjacent siblings of v are 2-nodes
 - **Fusion operation:** merge v with an adjacent sibling w and move an entry from u to the merged node v'
 - After a fusion, the underflow may propagate to the parent u



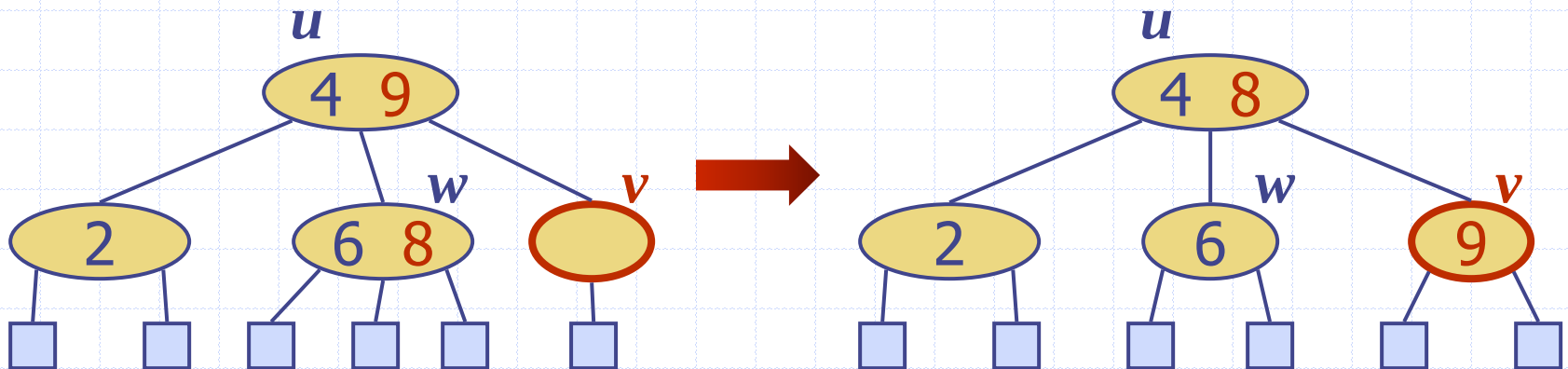
Underflow and Transfer

- ◆ To handle an underflow at node v with parent u , consider two cases
- ◆ **Case 2:** adjacent sibling w of v is a 3-node or a 4-node
 - **Transfer operation**
 1. move a child of w to v
 2. move an item from u to v
 3. move an item from w to u
 - After a transfer, no underflow occurs



Underflow and Transfer

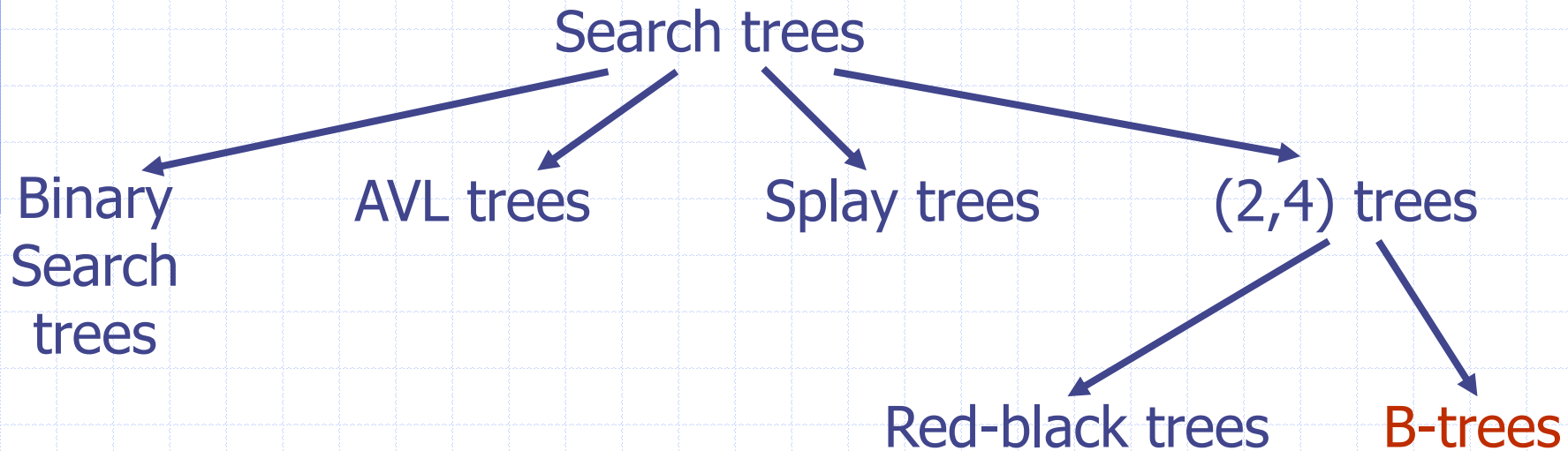
- ◆ To handle an underflow at node v with parent u , consider two cases
- ◆ **Case 2:** adjacent sibling w of v is a 3-node or a 4-node
 - **Transfer operation**
 1. move a child of w to v
 2. move an item from u to v
 3. move an item from w to u
 - After a transfer, no underflow occurs



Analysis of Deletion

- ◆ Let T be a $(2,4)$ tree with n items
 - $O(\log n)$ height
- ◆ In a deletion operation
 - visit $O(\log n)$ nodes to locate the node from which to delete the entry
 - handle an underflow with a series of $O(\log n)$ fusions, followed by at most one transfer
 - each fusion and transfer takes $O(1)$ time
- ◆ Thus, deleting an item from a $(2,4)$ tree takes $O(\log n)$ time

Week 7 – Search Trees



(a,b) Trees

- ◆ An (a,b) tree is a multi-way search tree where each node has between **a** and **b** children and stores between $a-1$ and $b-1$ entries
- ◆ By setting the parameters **a** and **b** appropriately, with respect to the size of disk blocks, we can derive a data structure that achieves good external-memory performance

Definition

- ◆ An **(a,b) tree**, where parameters **a** and **b** are integers such that $2 \leq a \leq (b+1) \div 2$, is a multi-way search tree, with the following additional restrictions
 - **Size Property:** Each internal node has at least **a** children, unless it is the root, and has at most **b** children
 - **Depth Property:** All external nodes have the same depth

Height of an (a,b) Tree

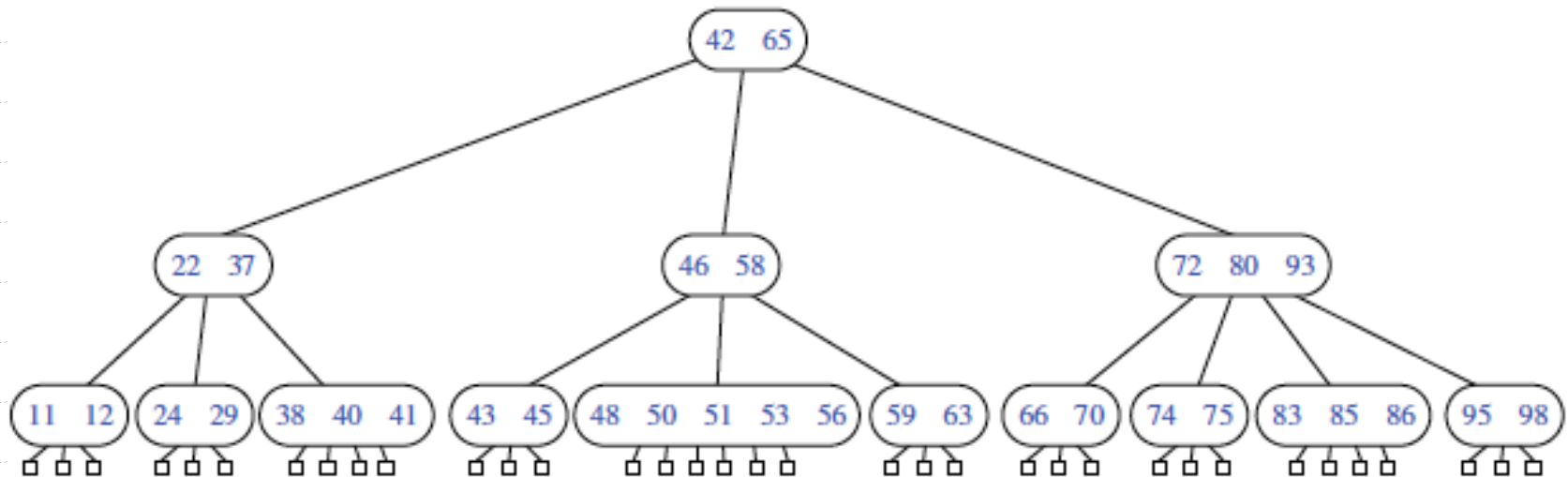
- ◆ Height of an (a,b) tree storing n entries
 - $\Omega(\log n \div \log b)$, which is $\Omega(\log_b n)$
 - ◆ i.e. at least
 - $O(\log n \div \log a)$, which is $O(\log_a n)$
 - ◆ i.e. at most

Search and Updates

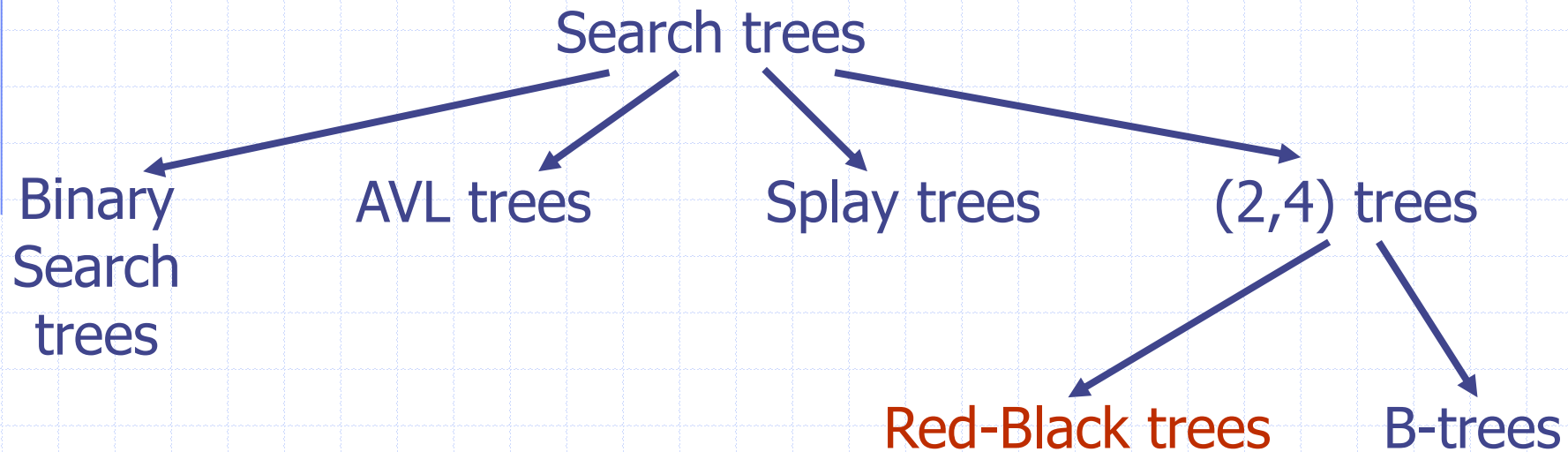
- ◆ **Search** algorithm in an (a,b) tree is same as for multi-way search trees
- ◆ **Insertion** algorithm for an (a,b) tree is similar to that for a $(2,4)$ tree
- ◆ **Removing** an entry from an (a,b) tree is similar to $(2,4)$ trees

B-Trees

- ◆ Version of the (a,b) tree data structure, which is the best-known method for maintaining a map in external memory, is a **B-tree**
- ◆ **B-tree of order d** is an (a,b) tree with $a = d \div 2$ and $b = d$



Week 7 – Search Trees

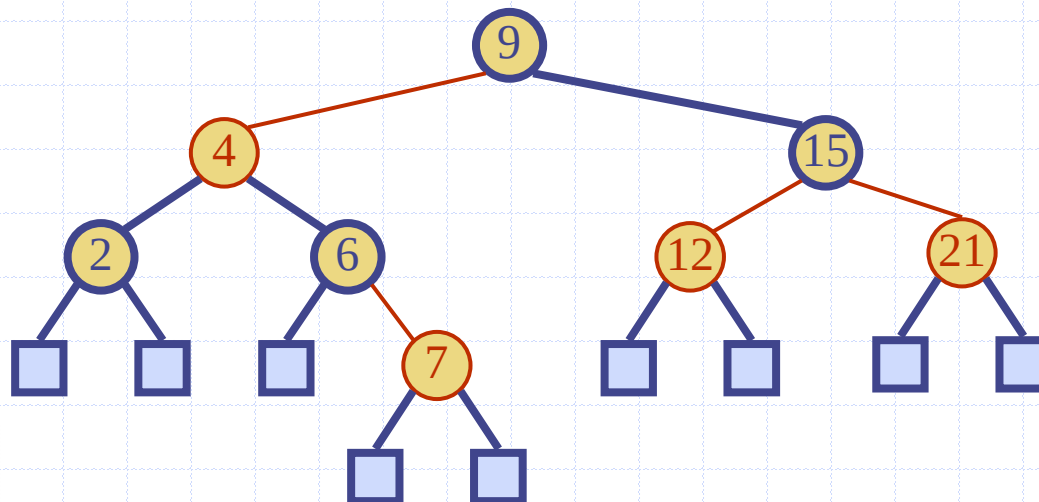


From (2,4) to Red-Black Trees

- ◆ Comparing with its associated (2,4) tree, a red-black tree has
 - same logarithmic worst-case complexity for search, insert, removal
 - simpler implementation
 - fewer restructurings required

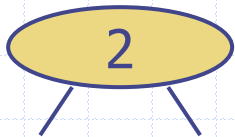
Red-Black Trees

- ◆ Binary search tree that satisfies the following properties
 - **Root Property:** root is black
 - **External Property:** every leaf is black
 - **Internal Property:** children of a red node are black
 - **Depth Property:** all leaves have the same black depth

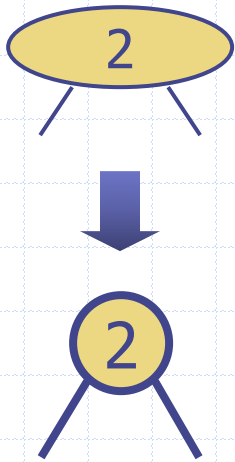


From (2,4) to Red-Black Trees

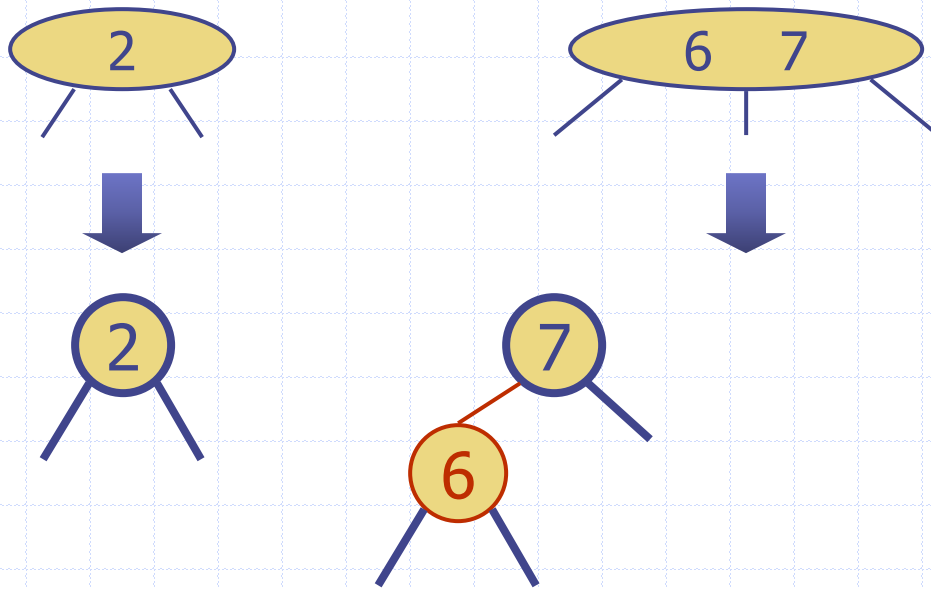
From (2,4) to Red-Black Trees



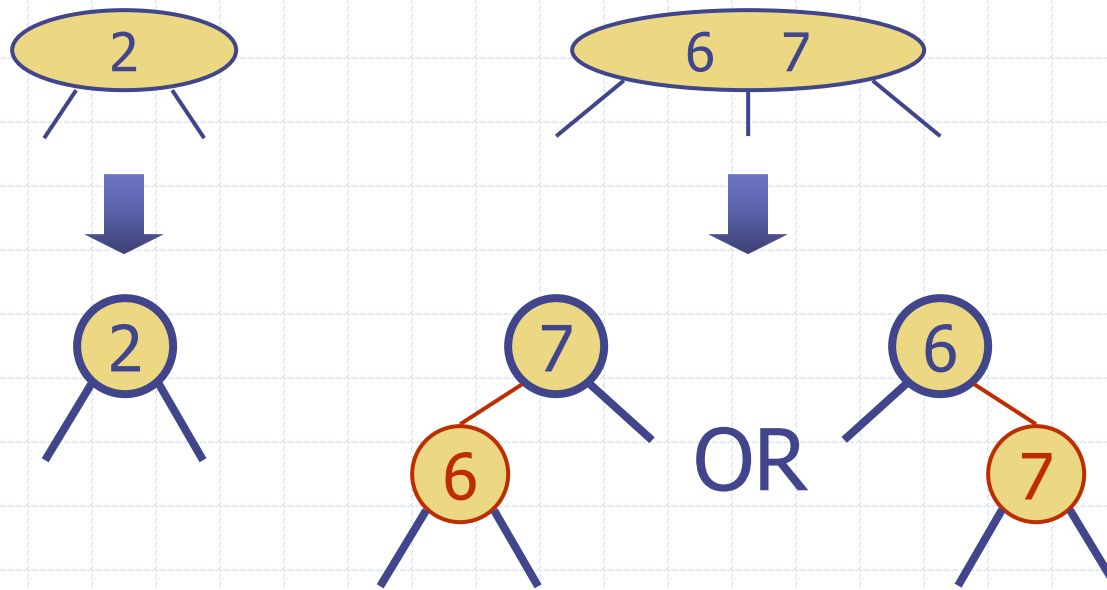
From (2,4) to Red-Black Trees



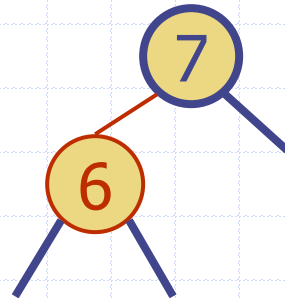
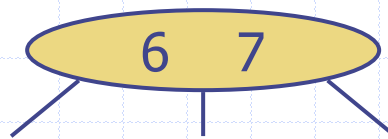
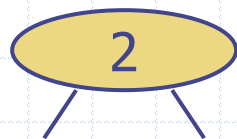
From (2,4) to Red-Black Trees



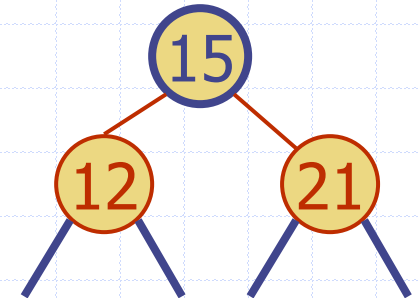
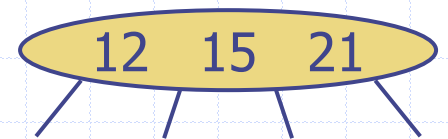
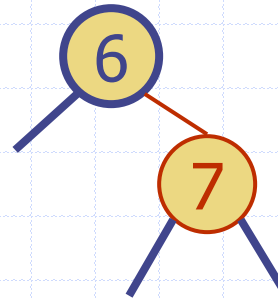
From (2,4) to Red-Black Trees



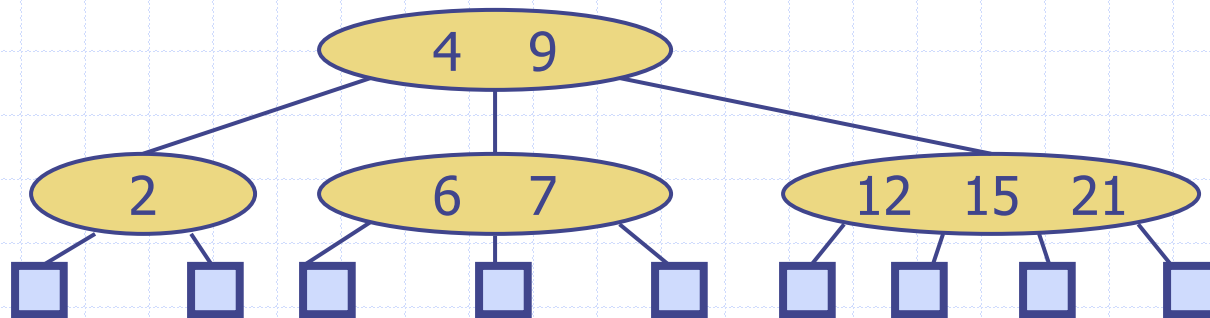
From (2,4) to Red-Black Trees



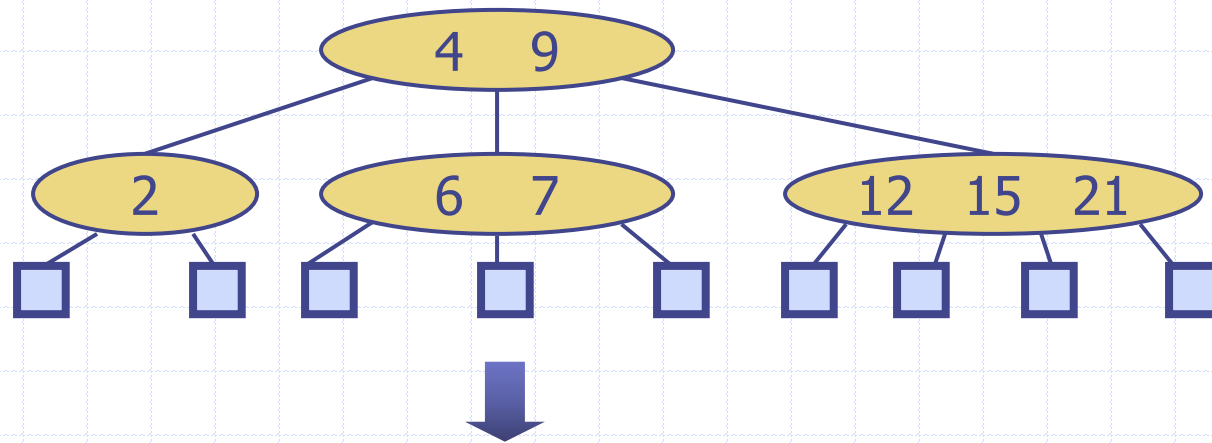
OR



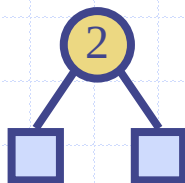
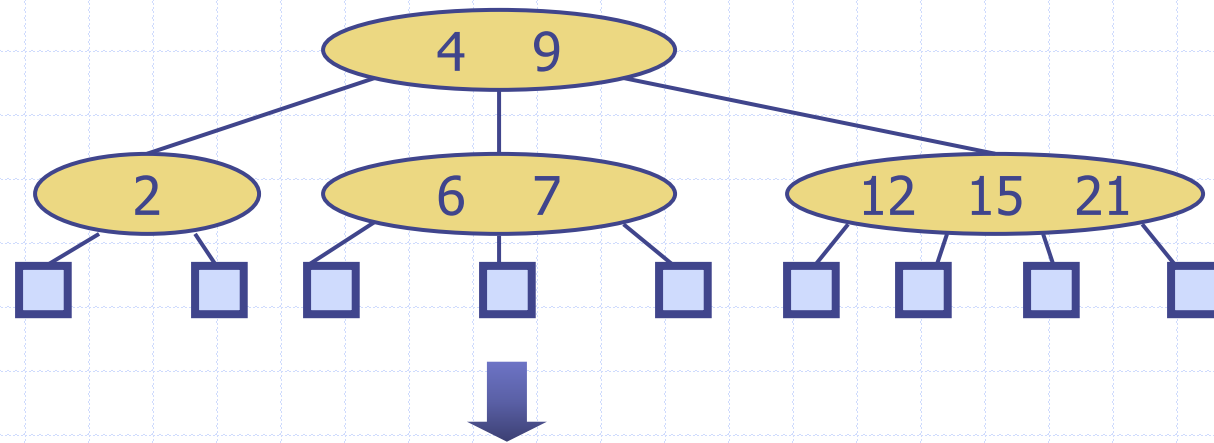
From (2,4) to Red-Black Trees



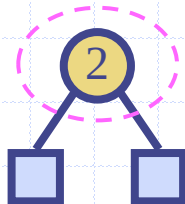
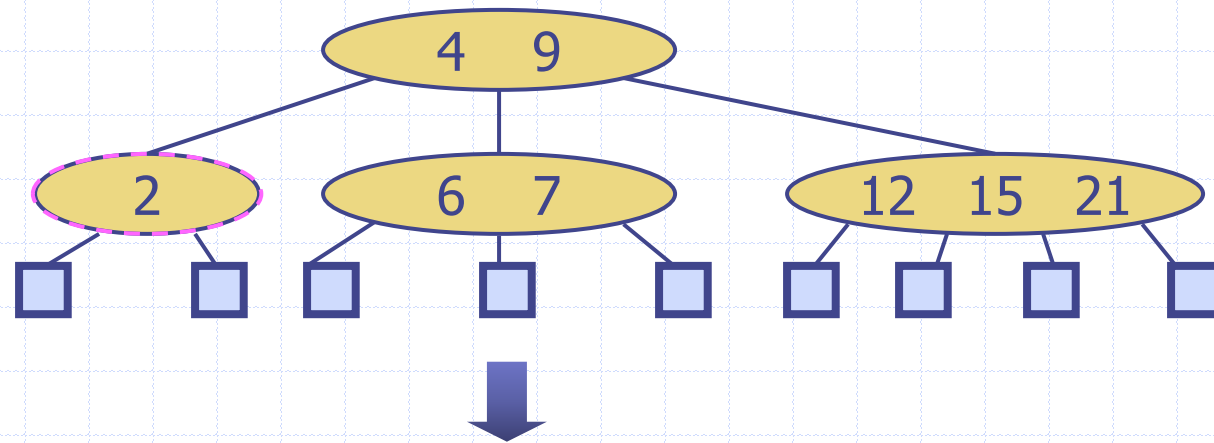
From (2,4) to Red-Black Trees



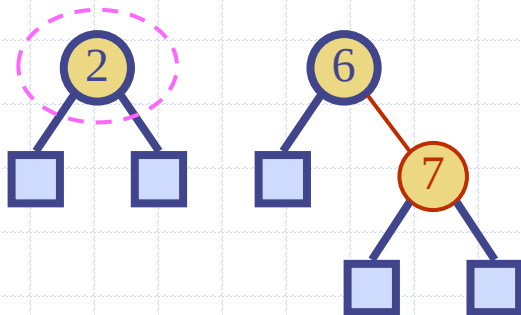
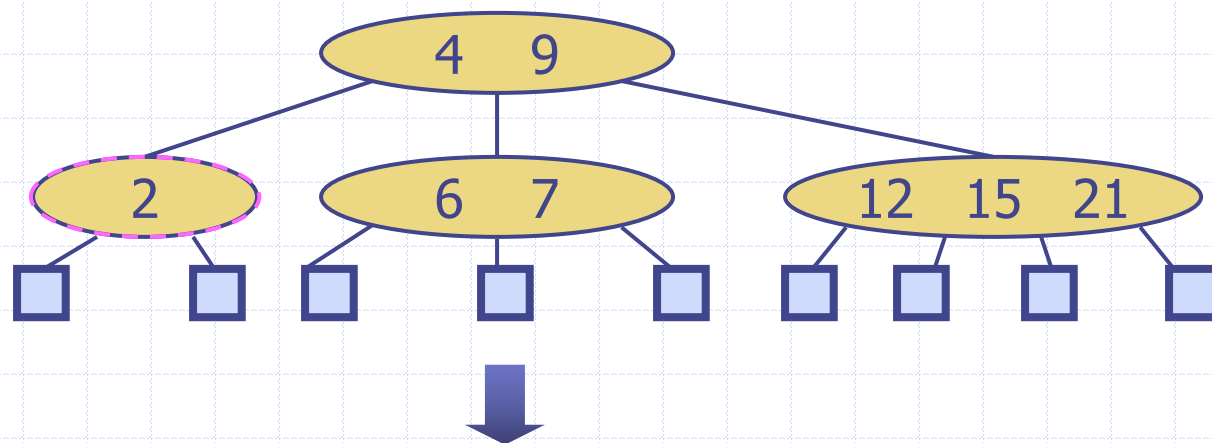
From (2,4) to Red-Black Trees



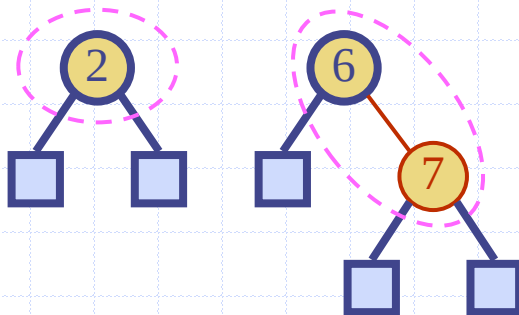
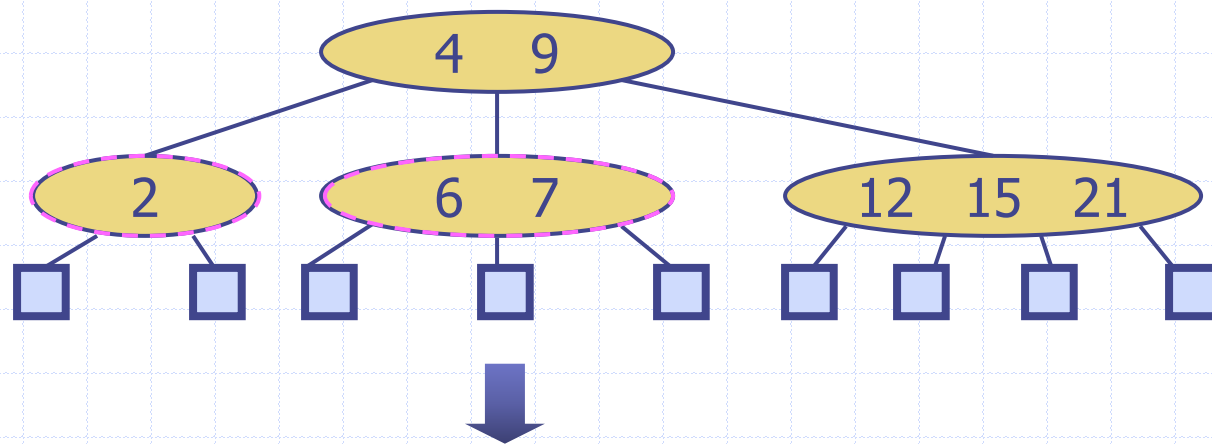
From (2,4) to Red-Black Trees



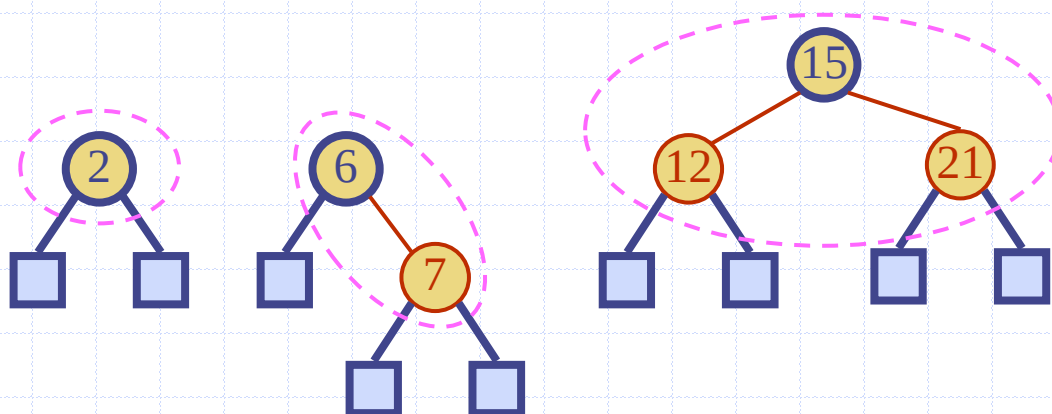
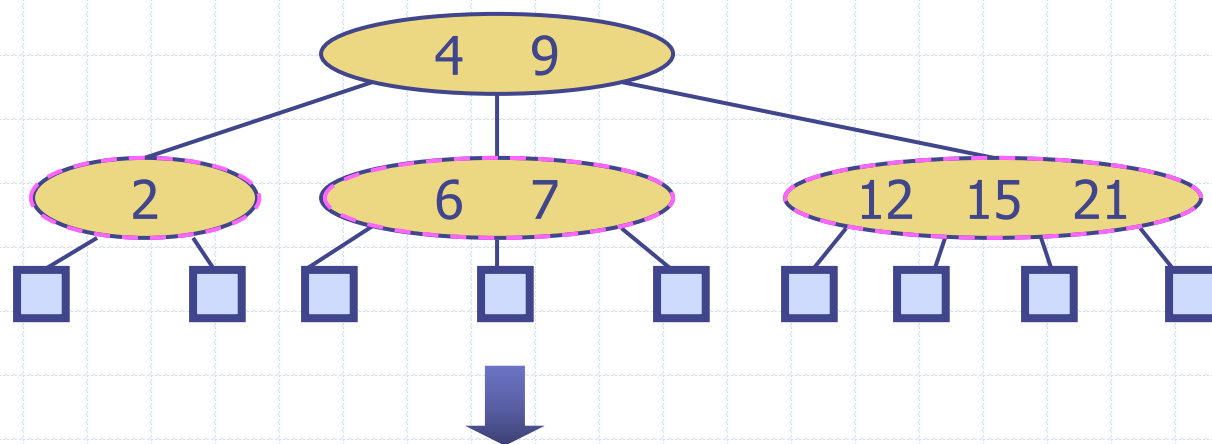
From (2,4) to Red-Black Trees



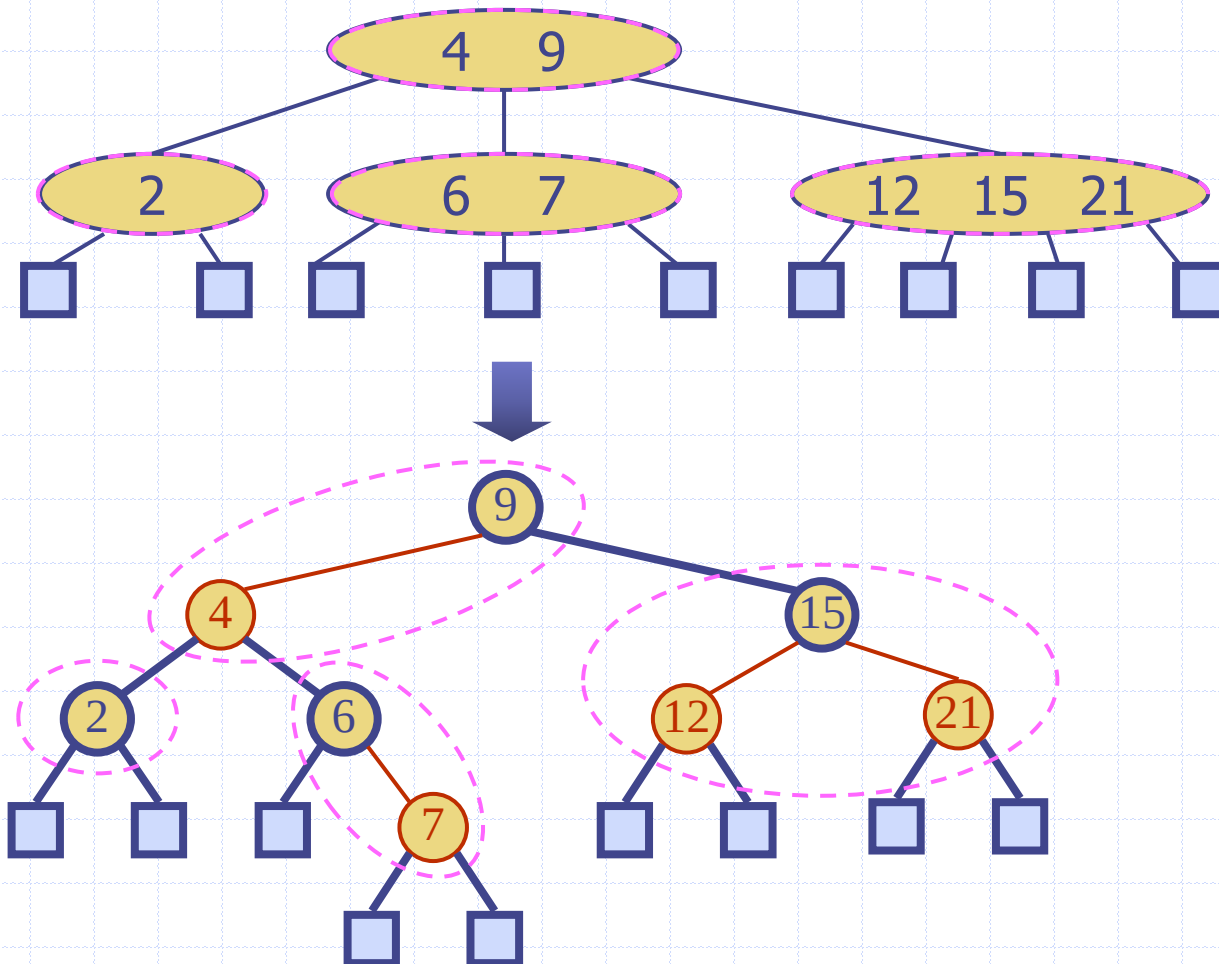
From (2,4) to Red-Black Trees



From (2,4) to Red-Black Trees



From (2,4) to Red-Black Trees



Height of a Red-Black Tree

◆ **Theorem:** Red-black tree storing n items has height $O(\log n)$

■ **Proof**

◆ height of a red-black tree is at **most twice** the height of its associated (2,4) tree

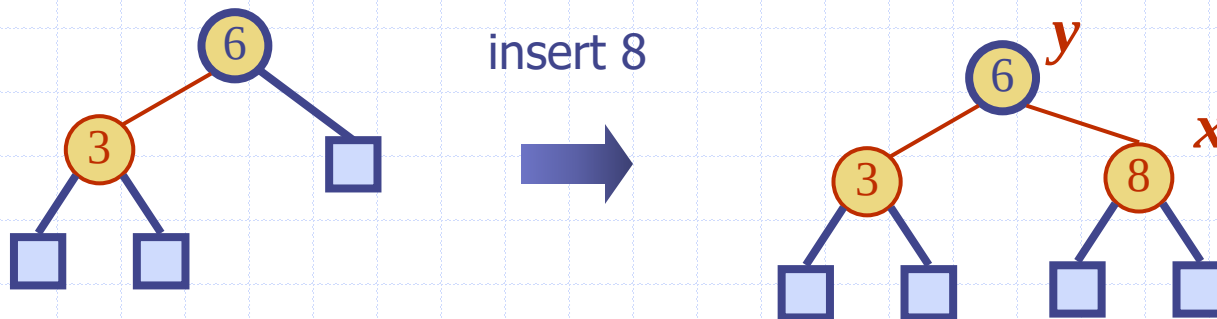
■ $O(\log n)$

◆ Search algorithm for a red-black tree is the same as for a binary search tree

■ $\therefore$ searching takes $O(\log n)$ time

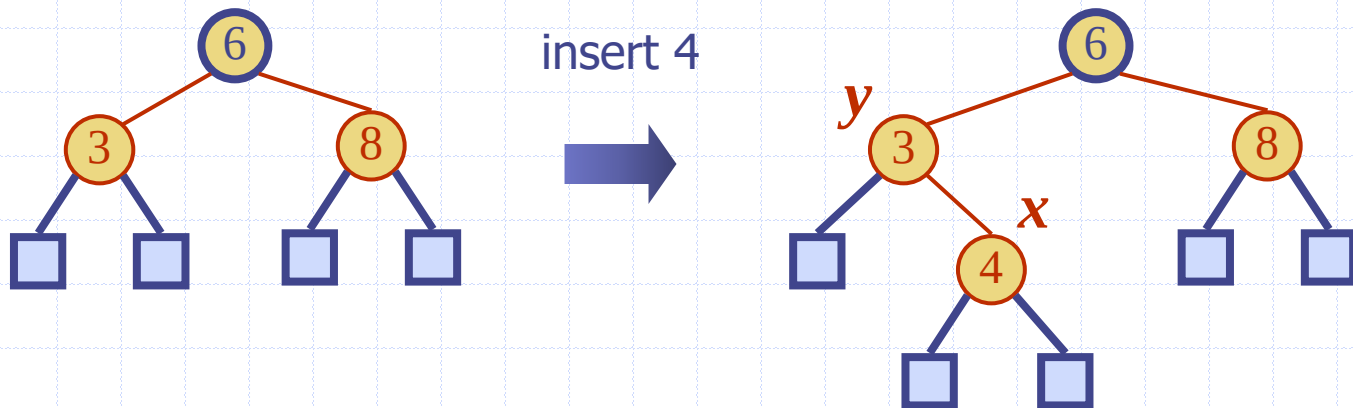
Insertion

- ◆ Insert (k, o) – execute BST insertion algorithm and colour newly inserted node (x) red, unless it is the root
 - Preserves the root, external and depth properties
 - If the parent y of x is black, it preserves the internal property
 - ◆ nothing else to do



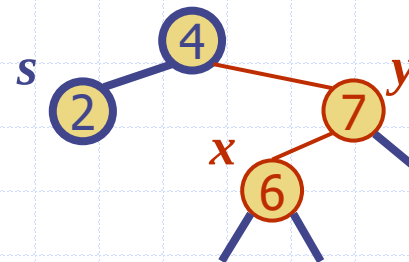
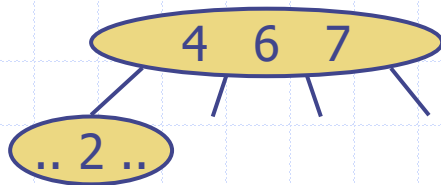
Insertion

- ◆ Insert (k, o) – execute BST insertion algorithm and colour newly inserted node (x) red, unless it is the root
 - Preserves the root, external and depth properties
 - If the parent y of x is black, it preserves the internal property
 - ◆ nothing else to do
 - Else (y is red) causing a **double red** (violation of internal property)
 - ◆ requires a reorganisation of the tree



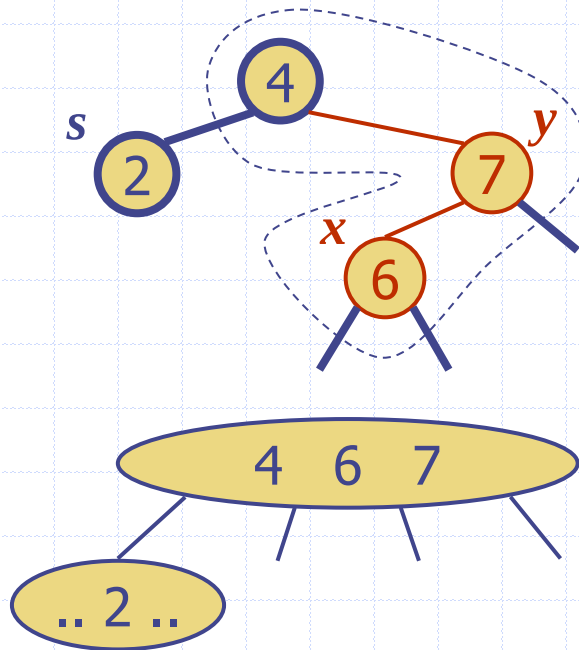
Remedying Double Red

- ◆ Consider double red with child x and parent y
 - let s be the sibling of y
- ◆ Case 1: s is black
 - double red is an incorrect replacement of a 4-node
 - tri-node restructuring: change the 4-node replacement



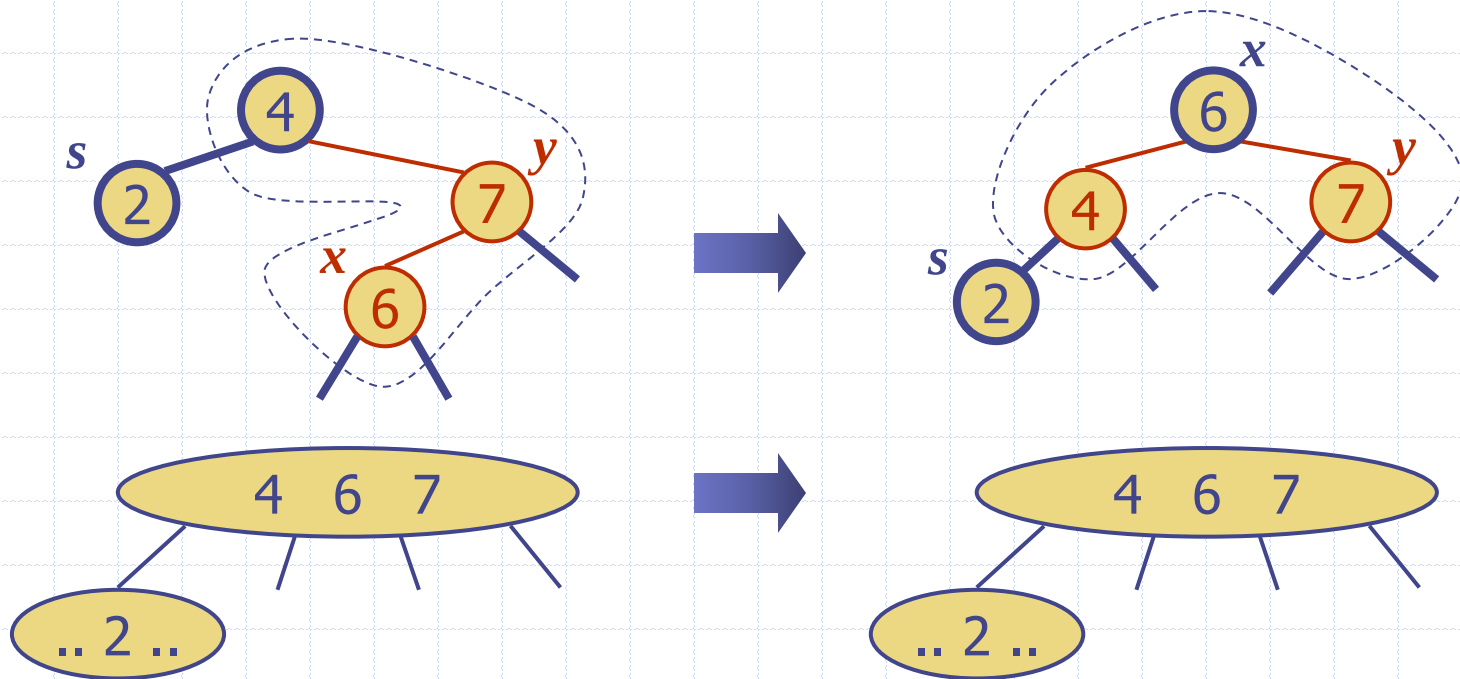
Case 1: Tri-Node Restructuring

- ◆ Remedies a child-parent double red when the parent red node has a black sibling
- ◆ Equivalent to restoring the correct replacement of a 4-node
- ◆ Internal property is restored and other properties are preserved



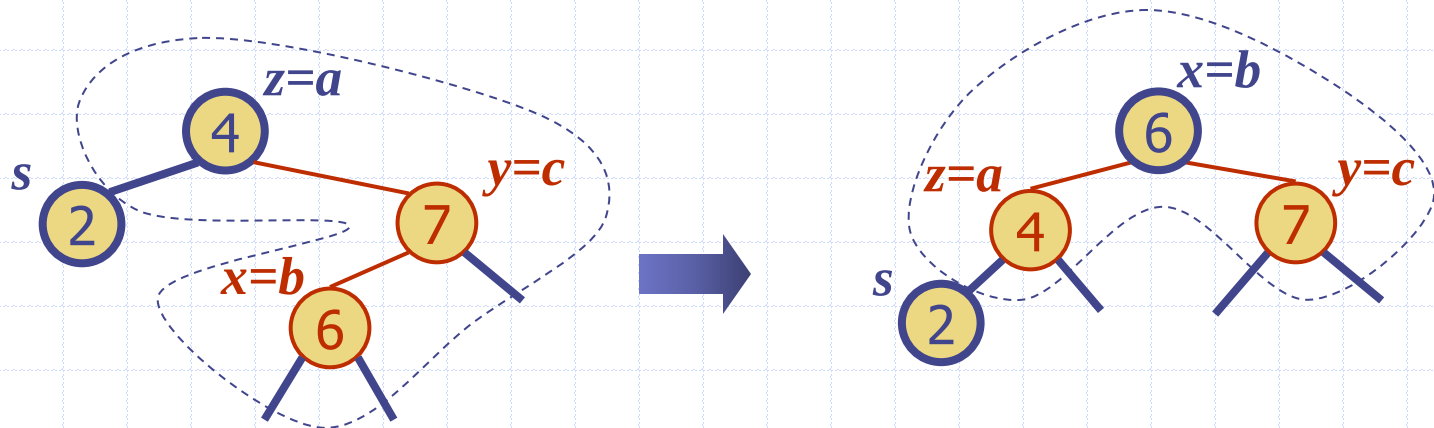
Case 1: Tri-Node Restructuring

- ◆ Remedies a child-parent double red when the parent red node has a black sibling
- ◆ Equivalent to restoring the correct replacement of a 4-node
- ◆ Internal property is restored and other properties are preserved

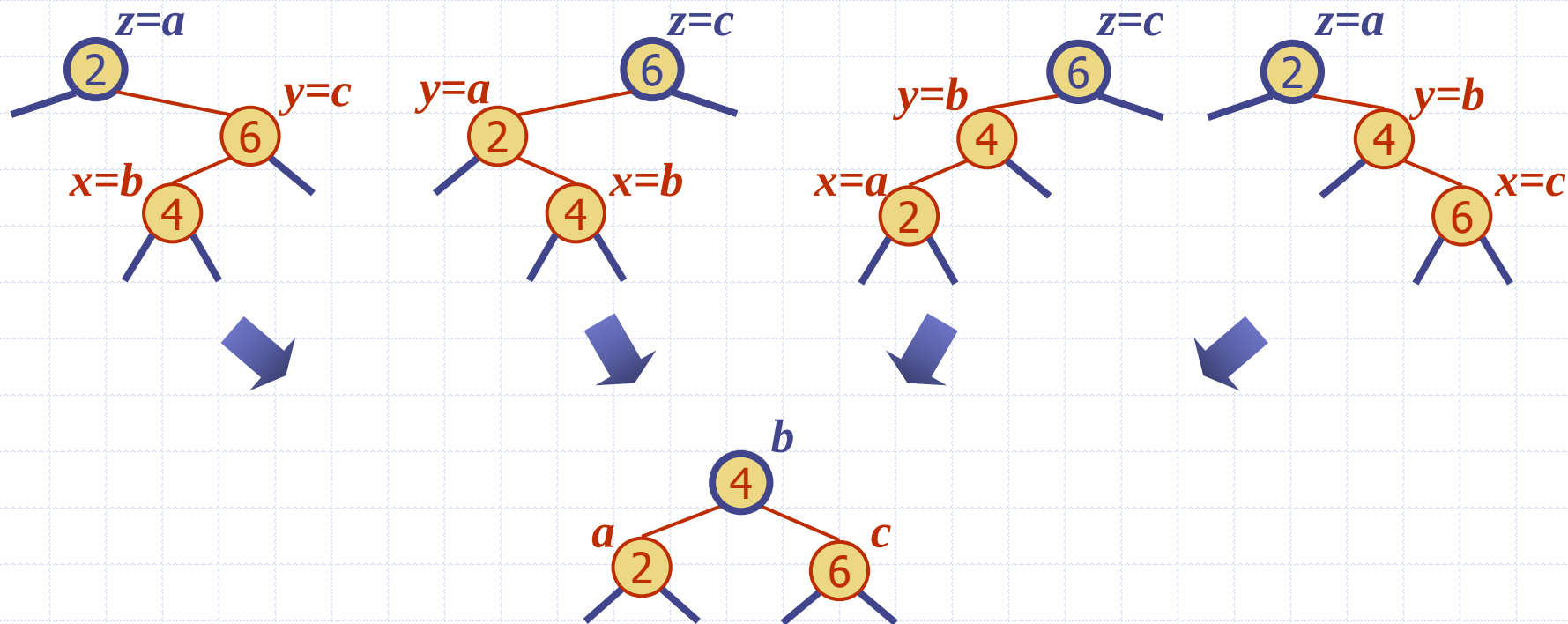


Case 1: Tri-Node Restructuring

- ◆ Label nodes x , y and z as a , b and c in left-to-right order
 - a , b and c will be visited in this order by in-order traversal
- ◆ Replace z with the node labelled b
- ◆ Make nodes a and c children of b
 - maintaining in-order relationship

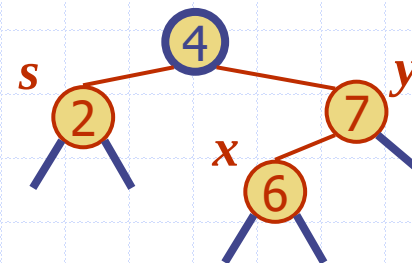
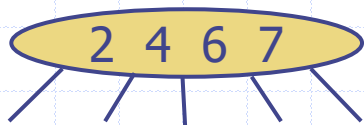


Case 1: Restructuring (cont.)



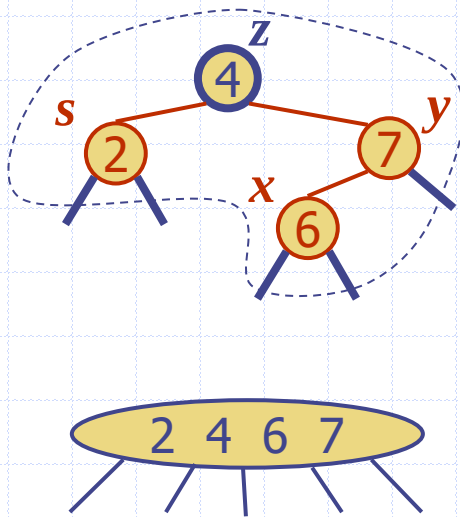
Remedying Double Red

- ◆ Consider double red with child x and parent y
 - let s be the sibling of y
- ◆ Case 2: s is red
 - double red corresponds to an **overflow**
 - **recolouring**: perform the equivalent of a **split**



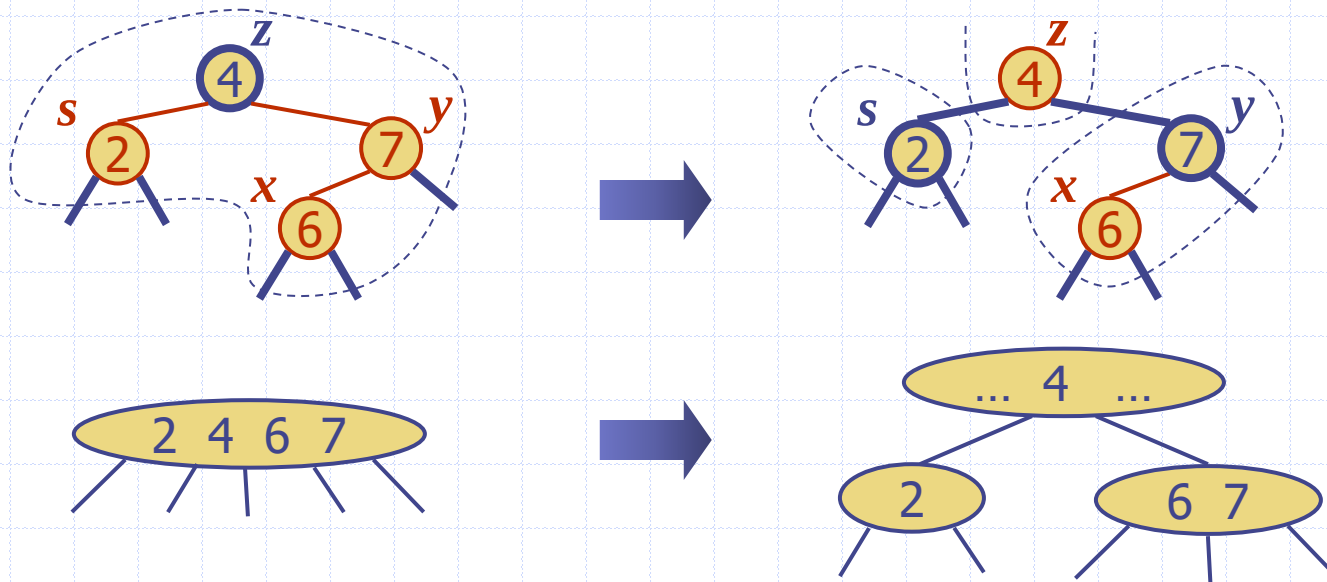
Case 2: Recolouring

- ◆ Remedies a child-parent double red when the parent red node has a red sibling
- ◆ Parent y and its sibling s become black and the grandparent z becomes red, unless it is the root
- ◆ Equivalent to performing a split on a 5-node
- ◆ Double red violation may propagate to the grandparent z



Case 2: Recolouring

- ◆ Remedies a child-parent double red when the parent red node has a red sibling
- ◆ Parent y and its sibling s become black and the grandparent z becomes red, unless it is the root
- ◆ Equivalent to performing a split on a 5-node
- ◆ Double red violation may propagate to the grandparent z



Analysis of Insertion

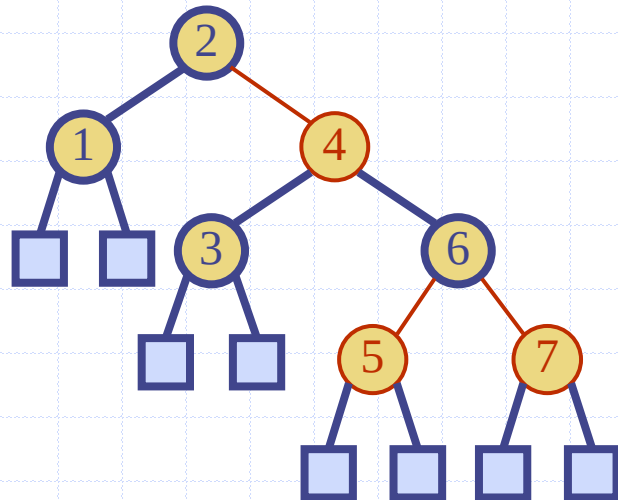
Algorithm *insert*(*k*, *o*)

1. Search for key *k* to locate the insertion node *z*
2. Add new entry (*k*, *o*) at node *z* and colour *z* red
3. **while** *doubleRed*(*z*)
 if *isBlack*(*sibling*(*parent*(*z*)))
 z $\leftarrow$ *restructure*(*z*)
 return
 else {*sibling*(*parent*(*z*)) is red}
 z $\leftarrow$ *recolour*(*z*)

- ◆ Recall that a red-black tree has $O(\log n)$ height
- ◆ Step 1 takes $O(\log n)$ time because we visit $O(\log n)$ nodes
- ◆ Step 2 takes $O(1)$ time
- ◆ Step 3 takes $O(\log n)$ time because we perform
 - $O(\log n)$ recolourings, each taking $O(1)$ time, and
 - at most one restructuring taking $O(1)$ time
- ◆ Thus, insertion in a red-black tree takes $O(\log n)$ time

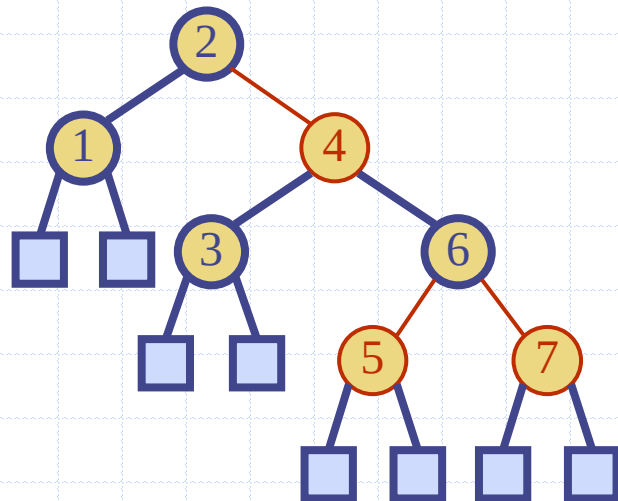
Deletion

- ◆ To perform operation **remove(k)**, first execute the deletion algorithm for binary search trees
 - node to be deleted has a right leaf

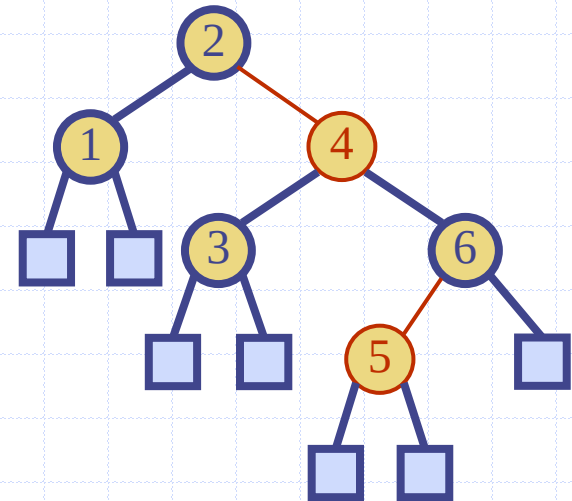


Deletion

- ◆ To perform operation **remove(k)**, first execute the deletion algorithm for binary search trees
 - node to be deleted has a right leaf

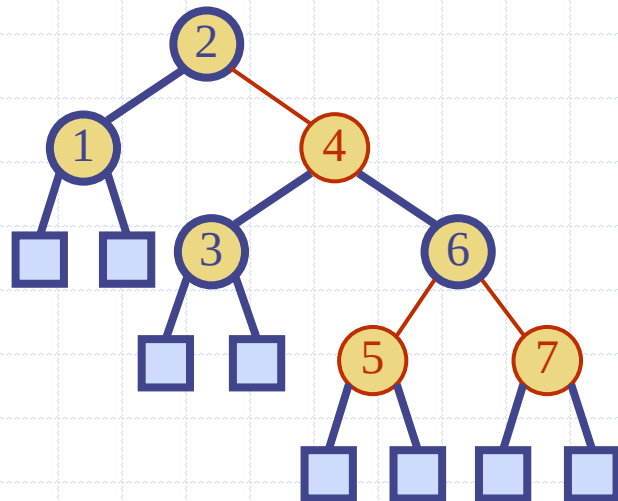


delete 7



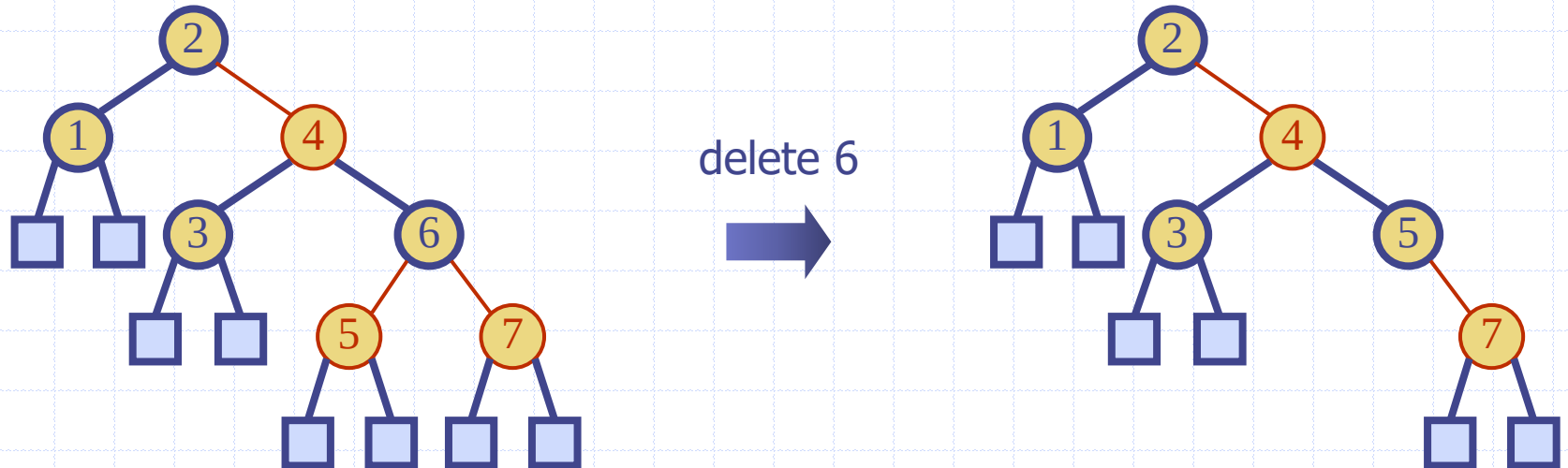
Deletion

- ◆ To perform operation **remove(k)**, first execute the deletion algorithm for binary search trees
 - node to be deleted has a right leaf, or
 - swap node to be deleted with its in-order predecessor
 - ◆ which will have a right leaf



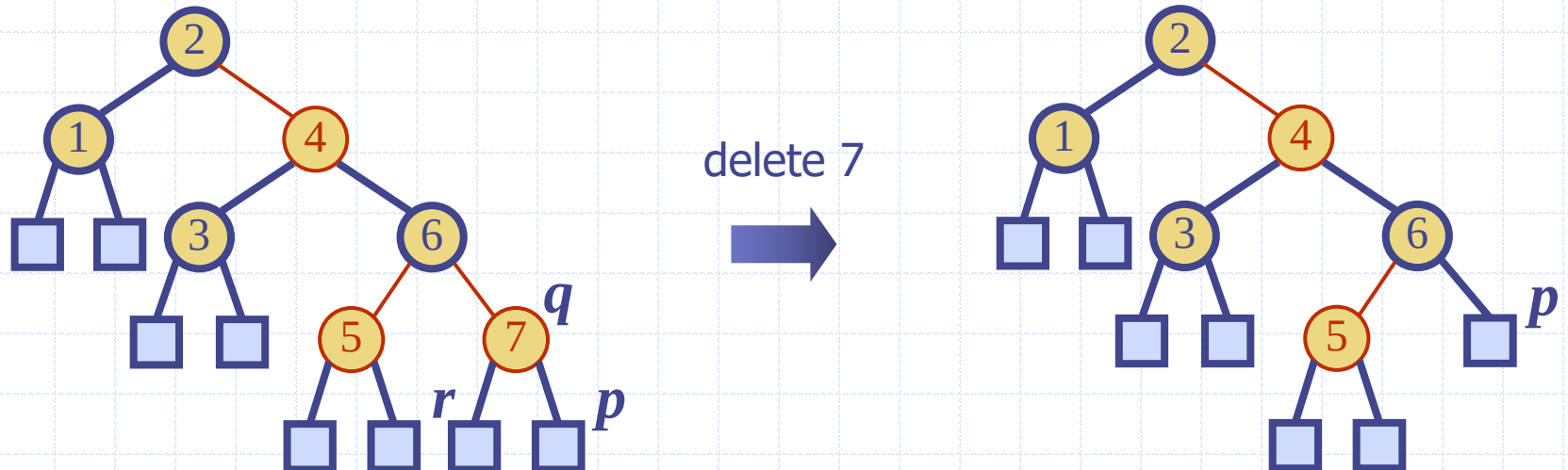
Deletion

- ◆ To perform operation **remove(k)**, first execute the deletion algorithm for binary search trees
 - node to be deleted has a right leaf, or
 - swap node to be deleted with its in-order predecessor
 - ◆ which will have a right leaf



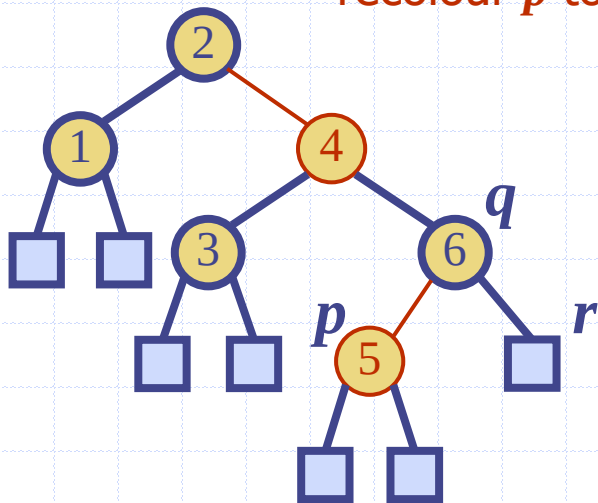
Deletion

- ◆ To perform operation **remove(k)**, first execute the deletion algorithm for binary search trees
- ◆ Let q be the internal node removed, r the external node removed, and p the sibling of r
 - If q was red, we are done

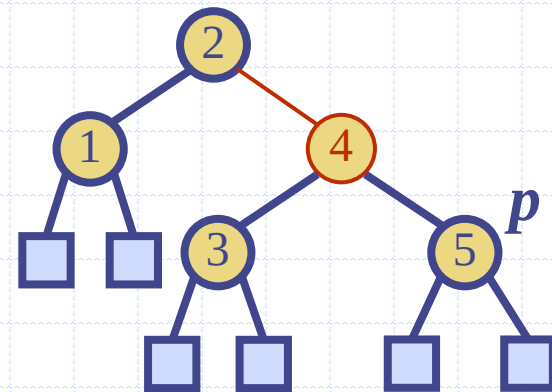


Deletion

- ◆ To perform operation **remove(k)**, first execute the deletion algorithm for binary search trees
- ◆ Let q be the internal node removed, r the external node removed, and p the sibling of r
 - If q was black, either both children were external, or q had one red child, p
 - ◆ recolour p to be black

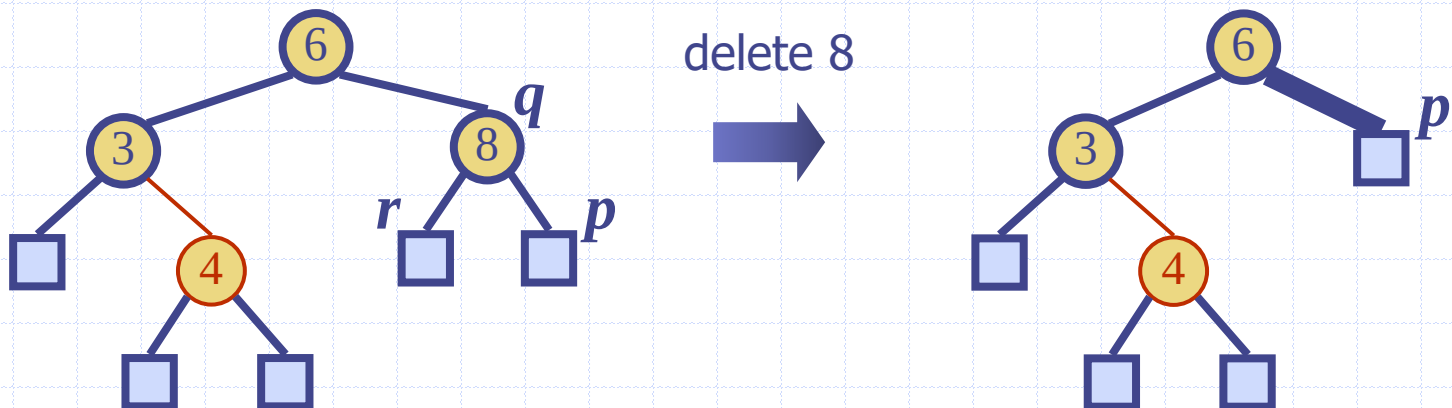


delete 6



Deletion

- ◆ To perform operation **remove(k)**, first execute the deletion algorithm for binary search trees
- ◆ Let q be the internal node removed, r the external node removed, and p the sibling of r
 - Else q was black, with two external nodes, colour p double black
 - ◆ corresponds to an underflow

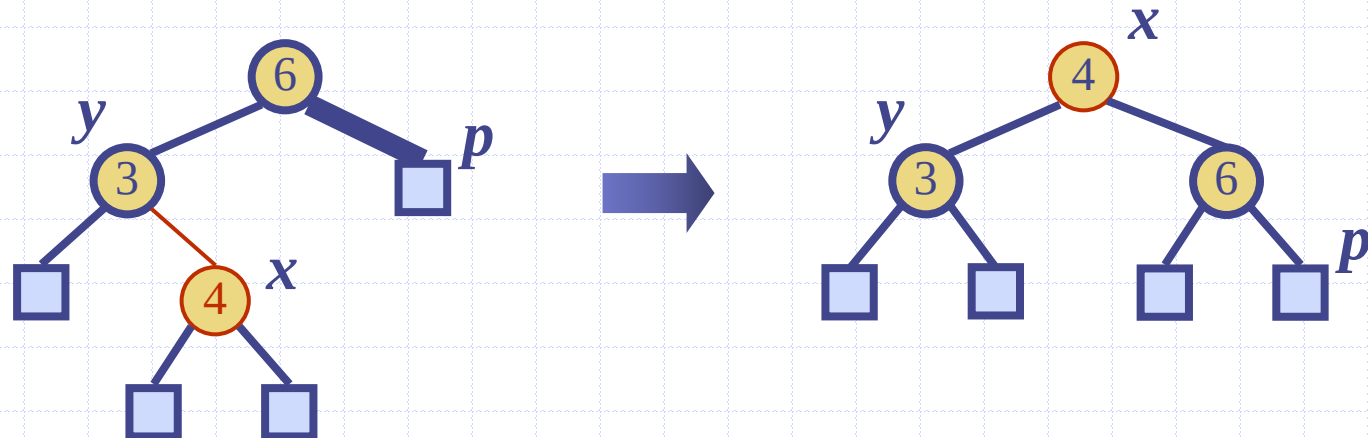


Remedying Double Black

- Algorithm for remedying double black node p with sibling y , considers three cases

Case 1: y is black and has a red child, x

- Perform a **restructuring**, equivalent to a **transfer**, and we are done

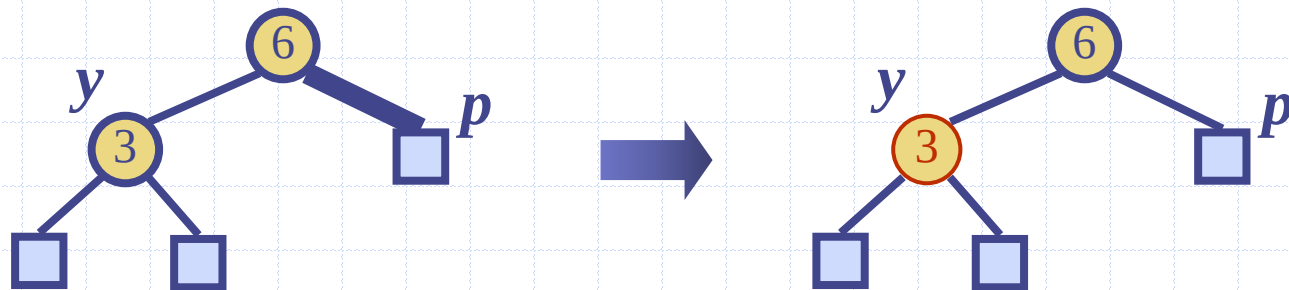


Remedying Double Black

- Algorithm for remedying double black node p with sibling y , considers three cases

Case 2: y is black and its children are both black

- Perform a **recolouring**, equivalent to a **fusion**

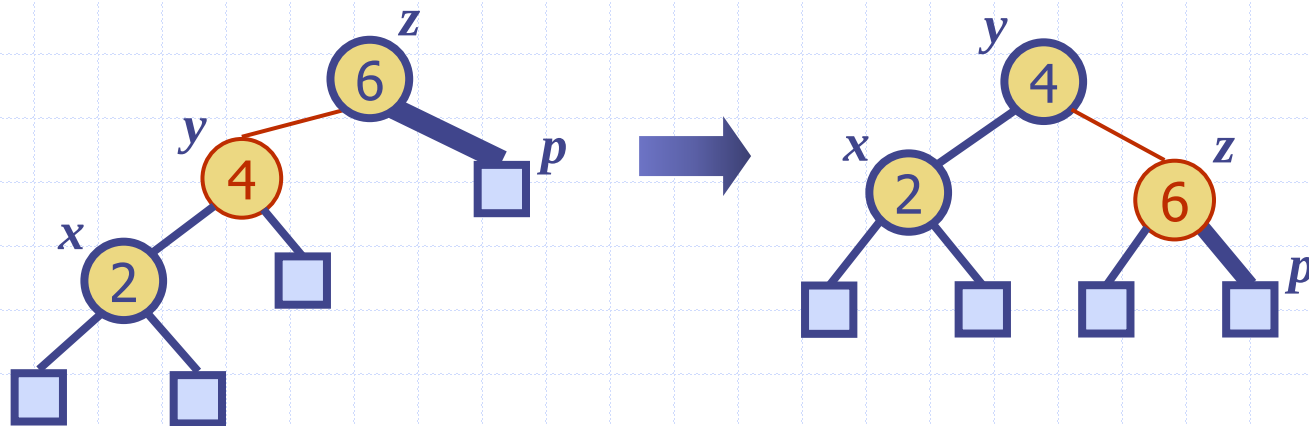


Remedying Double Black

- Algorithm for remedying double black node p with sibling y , considers three cases

Case 3: y is red

- Perform an **adjustment**, after which either Case 1 or Case 2 applies



- Deletion in a red-black tree takes $O(\log n)$ time

Red-Black Tree Reorganisation

Insertion remedy double red		
Red-black tree action	(2,4) tree action	result
restructuring	change of 4-node representation	double red removed
recolouring	split	double red removed or propagated up

Deletion remedy double black		
Red-black tree action	(2,4) tree action	result
restructuring	transfer	double black removed
recolouring	fusion	double black removed or propagated up
adjustment	change of 3-node representation	restructuring or recolouring follows